unooria®

PLATAFORMA UNOORIA DE APRENDIZAGEM INTERATIVA

Manual do Gerador de Missões

A esteira de 114 passos que transforma um briefing de meia página — ou o one-pager que o PO já escreveu — em um PRD pronto para codificação e em uma missão que a professora aprova.

PARA QUEM ESTE MANUAL É

Coordenação pedagógica e professoras consultoras · Product Owners · Designers instrucionais e de experiência · Engenharia de software · Quem opera a esteira no dia a dia

EDIÇÃO

4.ª edição — 8 de outubro de 2026
Repositório `gerador-missoes` · rubrica de profundidade v1.21

ÂMBITO

1.º ano do Fundamental ao 3.º do Médio
BNCC Computação (Resolução CNE/CEB n.º 1/2022) · LGPD, art. 14 · ECA

Unooria Educação e Tecnologia — documento de trabalho, uso interno.

Ficha técnica desta edição

O que este documento é	O manual único do Gerador de Missões. Nasce da fusão de dois documentos anteriores: um voltado à operação (como escrever a entrada, como ler o diagnóstico, o que cada portão cobra) e outro à fundamentação (por que a arquitetura é multiagente, o que sustenta a régua pedagógica, o que a lei exige). Os dois eram corretos e incompletos em direções opostas.
Como foi verificado	Toda descrição de agente, portão, variável de ambiente e artefato foi conferida contra o código do repositório — `agents/`, `config.py`, `prompts/`, `tools/` — e não contra os manuais anteriores. Onde os dois divergiam, o código decidiu. A contagem de passos vem da lista de agentes que o próprio código publica para a tela de progresso, e a da árvore, de montar o orquestrador e percorrê-lo.
Arquitetura descrita	**114 passos numerados** — 43 com modelo de linguagem e 71 em código —, treze ciclos de correção automática, **parecer humano entre as etapas**, **banca em quatro posições** e a **base da plataforma** lida de um *bucket* próprio, só para leitura. Orquestração em Google ADK, modelo padrão `gemini-3.8-flash`.
O que mudou nesta edição	A 3.ª edição (06/10/2026) descrevia 81 passos. Em dois dias entraram: o ***one-pager* do PO como base preservada**; o **conteúdo aprovado** conferido até o PRD; a **banca em quatro posições**; o Stitch gerado **fora do modelo**; o **PRD codificável**; gates de modelo que **só bloqueiam com citação que existe**; a **matriz de rastreabilidade**; o **corte do modelo** registrado; e a grade publicada da plataforma no catálogo. Quase tudo nasceu de missões reais (Parte VII). A 2.ª edição (25/09) descrevia 72 passos; a 1.ª (19/09), 55.
Identidade visual	Paleta e tipografia do *brandbook* Unooria: azul profundo `#0e1039` (estrutura), lavanda `#8b85ac` (inteligência), azul claro `#7fa5e2` (interface). A fonte institucional é a Garet; nesta composição usamos Montserrat, da mesma família de desenho geométrico, por disponibilidade tipográfica. Verde, âmbar e vermelho aparecem apenas como **cores funcionais de sinalização**, para reproduzir os três estados que os relatórios da esteira emitem; não são cores de marca e nunca se aplicam ao logotipo.

Uma convenção de leitura. Ao longo do manual, cada regra da esteira aparece marcada pela natureza da sua fundamentação:

EVIDÊNCIA deriva de pesquisa publicada, com citação. CONTRATO deriva de documento normativo — BNCC, LGPD, manifesto da plataforma; não se pesquisa, se confere. CONVENÇÃO a equipe escolheu por consistência de produto, sem base empírica.

Convenção é legítima. Convenção apresentada como achado, não.

unooria®

Sumário

Por onde começar **5**

1 Os documentos que saem **7**

1.1 A regra que muda tudo: a esteira não desiste 8

2 O que é uma missão Unooria **9**

2.1 O framework de seis etapas 9

2.2 Os ambientes, e por que a lista é fechada 10

2.3 As duas camadas: o que é fixo e o que a professora parametriza 11

2.4 O recorte da aula: a customização é transversal, não temática 12

2.5 A missão tem duas visões, e as duas são entregáveis 12

3 Como escrever a entrada **14**

3.1 O que o primeiro agente precisa 14

3.2 Um briefing que funciona 14

3.3 Briefings que produzem missão ruim 15

3.4 Anexos valem mais que adjetivos 15

3.5 Quando você já tem o *one-pager* 16

3.6 O checklist antes de apertar o botão 16

4 Rodar a esteira **17**

4.1 Entrar 17

4.2 O parecer humano entre etapas 17

4.3 Os pedidos de refazer se acumulam 18

4.4 O que você pede é conferido. 19

4.5 O azul é decisão do PO 20

4.6 O agente não decide no lugar do PO 20

4.7 Perguntar a uma etapa, sem mudar nada 20

4.8 Corrigir uma missão já gerada 21

4.9 Rodar uma etapa isolada 21

4.10 Os *prompts*, para ler 21

4.11 Configuração 22

5 O identificador da missão **23**

6 Como ler o diagnóstico **24**

6.1 Quando a etapa para com uma mensagem 25

7 Três recomendações de uso **25**

8 Por que muitos agentes, e não um prompt grande. **27**

8.1 Densidade de instrução degrada o cumprimento 27

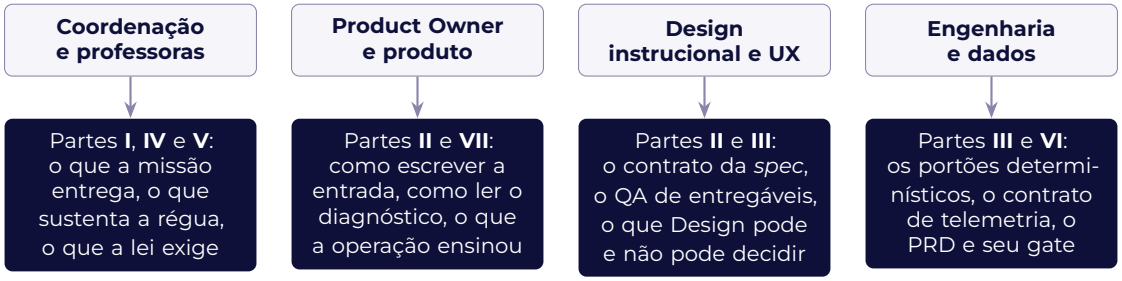
8.2 O critério de decomposição 27

8.3 Redundância não é cobertura . . . 27
8.4 O contraponto honesto . . . 28
9 As três etapas e o mapa dos agentes . . . 28
9.1 Produto (00b–16f): a missão pode ser construída? . . . 29
9.2 Design (16d–33c): como isso se constrói . . . 29
9.3 Dev (33a–49c): isso é implementável? . . . 30
9.4 Transversal (50–52): o que aconteceu, e o que fica . . . 30
10 Os treze ciclos de correção . . . 31
11 A banca de validação, em quatro posições. . . . 33
12 Os portões, e o que fazer quando reprovam . . . 34
13 O que impede o sistema de alucinar . . . 37
13.1 E os próprios verificadores, quem confere? . . . 38
14 Do *one-pager* aprovado ao PRD que dá para codar . . . 39
14.1 Três peças, todas em código . . . 39
14.2 O PRD que dá para codar . . . 39
15 As integrações, e por que nenhuma é caminho crítico . . . 40
16 A base da plataforma: lida do *bucket*, não copiada . . . 41
17 Descoberta ativa, e não instrucionismo . . . 44
18 A profundidade da missão: a anatomia do "ahá" . . . 44
18.1 Ler o material antes de inventar . . . 44
18.2 Os seis passos do "ahá" . . . 45
18.3 O juiz, e por que ele não decide sozinho. . . . 46
18.4 O que esta rubrica não faz . . . 46
19 Carga cognitiva, e o que a esteira faz com ela . . . 46
20 A régua por idade. . . . 47
20.1 Quantas opções o estudante compara de uma vez . . . 47
20.2 Quanto tempo dura cada ato . . . 48
20.3 O que se pode pedir que o estudante entenda . . . 48
20.4 O piso da escola pública brasileira . . . 49
21 O registro negativo: o que não sustenta a régua . . . 49
21.1 O que mudou na revisão de setembro de 2026 . . . 50
22 O papel do professor . . . 51
23 Alinhamento curricular e o primado do letramento digital . . . 53
23.1 Como o verificador de BNCC funciona . . . 53
23.2 Quando nenhuma habilidade do ano serve . . . 54
23.3 O contrato de evidência: o que o número sobre uma criança pode afirmar. . . . 54
24 Inteligência artificial com humano no controle . . . 55
25 Privacidade: o artigo 14 da LGPD . . . 55
26 Ética de acolhimento e não-rotulação . . . 56

27 Como ler este catálogo . . . 58
28 Produto (00b–16f) . . . 58
29 Design (16d–33c) . . . 63
30 Dev (33a–49c) e transversal (50–52) . . . 66
31 O refino dos entregáveis . . . 72
31.1 O *one-pager* resumido: quando o documento certo é o curto . . . 72
31.2 *One-pager*: o problema era onde as coisas estavam . . . 72
31.3 PRD: o problema era que nada verificava o que ele afirmava . . . 73
32 O que a primeira e a segunda execução ensinaram . . . 74
32.1 O efeito medido . . . 74
33 O que a terceira e a quarta execução ensinaram . . . 75
33.1 Um portão que julga precisa poder devolver . . . 75
33.2 Por que o redator errou com a regra escrita no *prompt* . . . 76
33.3 Um achado que não faz sentido é pior que nenhum achado . . . 76
34 A devolutiva que mudou a arquitetura . . . 76
34.1 Medir o que o time mede . . . 77
34.2 Pôr uma pessoa na passagem . . . 77
34.3 E uma coisa que a esteira passou a saber . . . 77
35 A missão que saiu verde e estava errada . . . 78
35.1 O que a leitura humana achou, e o que entrou por causa de cada coisa . . . 78
35.2 O juiz de fidelidade, com a IA de verdade . . . 79
35.3 As lições . . . 79
36 O *one-pager* do PO que a esteira reescreveu . . . 80
37 A resposta que o modelo cortou . . . 81
38 A primeira volta do padrão ouro . . . 81
39 A consequência de método . . . 82
A A árvore de artefatos de uma execução . . . 84
B As seções do PRD padrão ouro . . . 85
C Glossário . . . 87
D Fontes . . . 89

Por onde começar

Este manual tem sete partes e não precisa ser lido na ordem. O caminho mais curto depende do que você faz com a esteira.



Se você tem quinze minutos e precisa de um só capítulo, leia a Seção 3 — **como escrever a entrada**. É a parte do sistema com maior efeito sobre a qualidade do que sai, e a única que depende inteiramente de você.

Parte	Título	Responde a
I	**O que a esteira faz**	Que documentos saem, para quem, e o que significa a esteira não abortar.
II	**Como usar**	O que colocar no briefing, como rodar, como ler o diagnóstico quando algo trava.
III	**Como é construída**	Por que 114 passos e não um prompt grande; portões, ciclos, a banca, a base da plataforma e o controle anti-alucinação.
IV	**Por que funciona**	A jornada de seis etapas, a carga cognitiva e a régua por idade — com os números e as fontes.
V	**O que protege**	BNCC e o primado do letramento digital, LGPD art. 14, ética de acolhimento.
VI	**Os agentes, um a um**	O catálogo completo, com o que cada um faz e em que se apoia.
VII	**O que a operação ensinou**	As correções que só apareceram ao rodar a esteira inteira sobre material real.

PARTE I

O que a esteira faz

Meia página de briefing entra. Sete documentos saem — e uma pessoa responde por cada etapa antes da seguinte começar.



uncoria®

1 Os documentos que saem

Você escreve um briefing de meia página sobre a missão que quer, anexa o material do bimestre e aperta um botão. A esteira roda a etapa de **Produto** — cerca de quinze minutos — e **para, esperando o seu parecer**. Só depois de você dar o *de acordo* ela segue para Design, e de novo para Dev. Ao fim, a pasta tem tudo o que é preciso para construir aquela missão, mais o relatório honesto do que não deu certo.

Essa parada é a mudança mais importante desde a primeira edição deste manual, e a Seção 4.2 a descreve em detalhe. O resumo: a máquina julga o que é mecânico, e uma pessoa responde por cada etapa antes da seguinte começar.

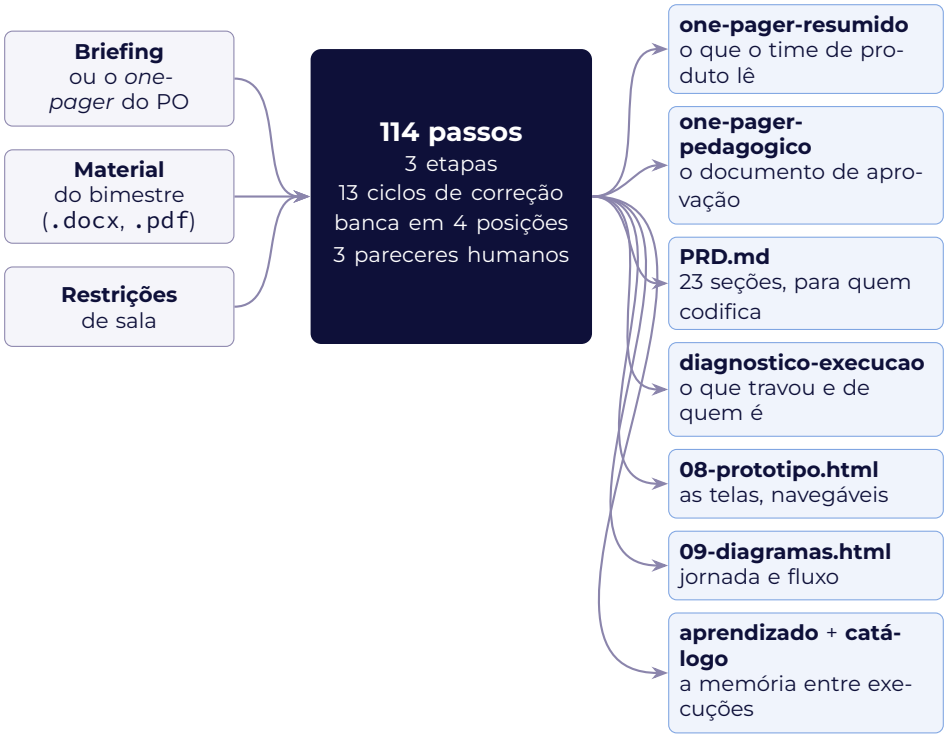


Figura 1: O contrato da ferramenta: o que ela exige de você e o que devolve.

Artefato	Para quem é
`produto/00d-one-pager-resumido` (`.md`, `.docx`, `.html`)	**O time de produto**, para a aprovação pedagógica. Formato curto: ficha, ideia, componente curricular do bimestre, socioemocionais, as seis etapas, o papel do professor, o contrato de evidência e o *takeaway*. Nasceu da comparação com o documento que o time escrevia à mão — ver Seção 31.1.
`one-pager-pedagogico` (`.md`, `.docx`, `.html`)	Coordenação e a professora consultora. Linguagem de sala de aula, zero jargão de software. É o documento de aprovação: se ele não convence, a missão não vai para a escola.
`PRD.md`	O agente — ou a pessoa — que vai codificar. 23 seções, com rotas em Zod, tabela de erros, DDL, eventos de telemetria, a ficha de evidência e o plano de implementação em caminhos reais do monorepo — e, no fim, as seções do *one-pager* aprovado coladas literais.
`diagnostico-execucao.md`	Você. Diz o que passou, o que passou com ressalva, o que travou, de quem é cada conserto e por onde começar. **É o primeiro arquivo a abrir.**
`design/08-prototipo.html`	Revisão humana. Arquivo único, sem dependências: abre com duplo clique e navega. É o que permite à professora simular a aula antes de aprovar.
`design/09-diagramas.html`	Quem quiser *ver* a missão: a jornada por raias e o grafo de telas, remontados do Mermaid.
`aprendizado.md`	O processo. O que esta execução ensinou, que padrões se repetiram e o que propor de mudança nas rubricas.
`matriz-de-rastreabilidade.md`	Quem quer saber se **o que o produto especificou chegou inteiro ao fim**: cada requisito do *one-pager* aprovado e da *spec*, procurado em cada documento de Design e de Dev, com o dono de cada perda.
`catalogo-ficha.md`	A memória entre execuções. A ficha desta missão entra no índice das já geradas e das publicadas na plataforma, e é esse índice que o micro-gate consulta para acusar missão repetida.

Mais cerca de setenta documentos intermediários na mesma pasta, consultáveis quando algo não fecha: o mapa conceitual do material, os três mecanismos da ideação, a *spec* da missão com o glossário canônico, a jornada, o fluxo, o dicionário de *copy*, a camada de acessibilidade, a especificação de telas, o *handoff* de dev, o *brief* de assets, o **banco de conteúdo** (o texto integral de cada documento da missão e as falas da IA simulada), a matriz de evidência, as perguntas de produto, o relatório da banca em cada posição e o de cada portão. Os arquivos baixados levam o identificador da missão no nome — `PRD_exp09ef1b.md`, `artefatos_exp09ef1b.zip` —, não o número da sessão. O Apêndice A traz a árvore completa.

1.1 A regra que muda tudo: a esteira não desiste

Um portão reprovado **não** interrompe a execução. Todos os documentos são produzidos até o fim; o que deriva de uma etapa reprovada sai marcado `[PROVISÓRIO]`, e o diagnóstico final diz o que é firme e o que é rascunho.

Isso é deliberado, e custou uma reescrita do orquestrador. Na versão anterior, a primeira reprovação parava tudo: não saía PRD, não saía *one-pager*, não saía relatório — e quem precisava entender o erro ficava sem material para analisar.

Um sistema que falha sem deixar evidência transfere para a pessoa o trabalho de reconstruir o que aconteceu. É o oposto do que uma esteira de produção deve fazer.

Antes — parada na primeira reprovação

Portão do gate estrutural reprova no critério 3.2.
→ Execução abortada.
→ Nenhum PRD, nenhum *one-pager*, nenhum relatório.
→ Quem lê precisa reabrir os *logs* para saber o que houve.

Resultado: 50 chamadas pagas, zero documento.

Hoje — a esteira segue e registra

Portão do gate estrutural reprova no critério 3.2.
→ Ciclo de correção tenta consertar (teto de rodadas).
→ Não resolvido, a etapa segue com herança degradada.
→ Tudo que deriva dali sai `[PROVISÓRIO]`; o PRD sai ●.

Resultado: documentos completos e um diagnóstico que nomeia o dono do conserto.

Figura 2: Por que a esteira deixa de abortar: a diferença não é de tolerância, é de evidência.

2 O que é uma missão Unooria

Antes de falar da esteira, é preciso ser preciso sobre o produto que ela fabrica. Uma **missão** é uma experiência imersiva de letramento digital, de uma aula, em que o estudante usa a própria conta dentro de uma simulação fiel de um ambiente digital real e **descobre agindo** algo que ninguém contou para ele.

A metáfora do estúdio

O Gerador de Missões não é uma máquina que escreve textos. É um **estúdio integrado de criação**, com três andares e uma sala de inspeção.
No primeiro andar, roteiristas e pedagogos (**Produto**) afiam a ideia até que o estudante viva um dilema real, e não uma aula expositiva disfarçada de jogo. No segundo, o estúdio de arte e cenografia (**Design**) projeta telas, fluxo, linguagem acessível e um protótipo que se abre no navegador. No terceiro, arquitetos e diretores técnicos (**Dev**) convertem tudo isso em especificação de engenharia, histórias de usuário e conformidade legal.
A sala de inspeção atravessa os três andares: a cada fronteira, um portão confere o trabalho contra um critério declarado — e quem inspeciona nunca é quem produziu.

2.1 O framework de seis etapas

Toda missão segue a mesma estrutura narrativa, e ela não é decorativa: é o que garante que a descoberta venha da ação, não do enunciado.



Figura 3: As seis etapas. O pico da experiência é o Resultado, e ele só funciona se a virada tiver sido guardada.

Etapa	O que o estudante faz	O que a etapa precisa garantir	Apoio teórico
Desafiar	Recebe uma tarefa concreta dentro da simulação: "monte uma seleção de três vídeos sobre o tema para mostrar à turma".	Um *job-to-be-done*, não um relato. O estudante sabe **o que** vai fazer — nunca **por quê**, porque o porquê é a descoberta.	*Merrill (2002)*
Navegar	Explora o ambiente com os verbos nativos da plataforma simulada: rolar, curtir, salvar, comentar, buscar.	O *gap* de informação. Nada no texto pode antecipar a virada — há inclusive uma lista de palavras-sentinela que o portão procura.	*Loewenstein (1994)*
Decidir	Executa a tarefa: escolhe, publica, envia, seleciona.	Escolha significativa: o estudante percebe que escolhe, a escolha muda algo perceptível, a missão a referencia depois e ele não pode desfazer depois de ver o efeito.	*Dewey (1938)*
Resultado	Vê a consequência da própria decisão e compara com a de colegas reais da turma.	O "ahá". É mediado pela professora, com dados de comparação no painel — não é uma tela de resposta certa.	*Bruner (1961)*
Explicar	Formula com as próprias palavras o que aconteceu e por quê.	Que o conceito seja **nomeado depois** da experiência, nunca antes.	*Vygotsky (1978)*
Refletir	Responde a uma provocação sobre uma situação nova, que não viveu na missão.	Aplicação, não confirmação. O teste é direto: alguém que *não* fez a missão consegue responder só pela pergunta? Se sim, a provocação não serve.	*Anderson (2001)*

2.2 Os ambientes, e por que a lista é fechada

A missão acontece dentro de uma simulação de plataforma real, e cada ambiente é uma **seção do *design system*** da Unooria. O briefing fala a língua do estudante — "ChatGPT", "Instagram", "WhatsApp"

—; a *spec* e o PRD falam a língua do código, que é o nome da seção. Desde outubro de 2026 a missão nova **não reproduz a identidade visual de aplicativo de terceiros**: ela usa a cópia estrutural do app com a identidade da Unooria, e o estudante vê o nome da Unooria, nunca o da marca imitada.

O briefing diz	Seção da missão nova	O que o estudante vê	Observação
ChatGPT, chat com IA	`unooriaai`	UnooriaAI	A seção `chatgpt` ficou para as missões antigas.
Instagram, feed de fotos	`unoovista`	Unoovista	A seção `instanooria` ficou para as missões antigas.
WhatsApp, grupo de conversa	`unoochat`	Unoochat	Tema claro. A `whatsappmobile` ficou para as antigas.
TikTok, vídeo curto	`tiktooria`	—	Existe no DS, mas ainda sem ponte para a plataforma: criá-la é pré-requisito.
YouTube, vídeo	`---`	—	O `unootube` foi arquivado.
Jornal, blog, galeria	`unoojornal`, `unooblog`, `unoogaleria`	—	Existem no DS, sem ponte para a plataforma.
A escola, o professor	`unooria`	Unooria	A própria plataforma.

A tradução vive num arquivo só (`prompts/references/ambientes.md`) e vale para todo briefing. A lista do que existe de fato — seções, pontes, componentes — não está escrita à mão em lugar nenhum: vem da **base da plataforma** que a execução leu (Seção 16).

Quem escolhe a seção é o PO, e a esteira respeita a escolha nos dois sentidos. A seção que o briefing fixa (`secao_ds: unoogaleria`) é **trava**: a ficha do manifesto e o *lint* do *one-pager* acusam a missão que a troca, e nenhum revisor a migra por conta própria. E a seção antiga em missão nova só **bloqueia** quando foi a esteira que a escolheu — a ficha aponta a sucessora. Quando a antiga está na linha "Seção do DS" do *one-pager* que o PO anexou, ou fixada no briefing, ela vira ressalva ● "mantida por decisão do PO". A mensagem do gate sempre disse que a antiga era decisão do PO; até 8 de outubro o código bloqueava assim mesmo, e empurrava a *spec* para a sucessora contra o *one-pager* aprovado.

Pedir uma missão "no Roblox" reprova de imediato, e a razão não é gosto: não existe seção de *design system* para construir aquilo, então não há o que especificar, nem o que prototipar, nem o que codificar. A lista fechada é um CONTRATO da plataforma, não uma preferência de produto. Nos anos iniciais, quando nenhum ambiente serve à intenção pedagógica, a esteira admite outra classe — o mini-jogo — decidida pelo agente de ambiente simulado (19).

2.3 As duas camadas: o que é fixo e o que a professora parametriza

Toda missão é projetada em duas camadas, e confundi-las é um erro caro de briefing.

1. Engine imersiva — estrutura fixa

O framework das seis etapas, a mecânica da plataforma simulada, os verbos nativos, os *checkpoints* com salvamento de estado e o banco curado padrão. Não muda de missão para missão, e não é parametrizável.

2. Parametrização e autoria assistida

Na preparação da aula, a professora insere tema, contexto e foco curricular; a IA da plataforma gera rascunhos de posts, comentários e mídias; e **ela cura, edita e aprova** antes de liberar para a turma.

A consequência prática aparece já no primeiro agente: onde o conteúdo é customizável, a missão escreve `[tema: parâmetro do professor]` em vez de fixar um assunto. Desenhar conteúdo genérico esperando personalizar depois é um defeito que a esteira detecta tarde e caro.

2.4 O recorte da aula: a customização é transversal, não temática

O que a professora parametriza não é "um tema". É o **recorte da aula**, e ele tem cinco campos — disciplina, tema, assuntos, contexto e foco. A diferença importa: significa que a professora de Português, a de Biologia e a de História podem preparar **a mesma missão** para conteúdos diferentes, porque a mecânica, as cardinalidades, a descoberta central e o currículo de letramento digital ficam fixos.

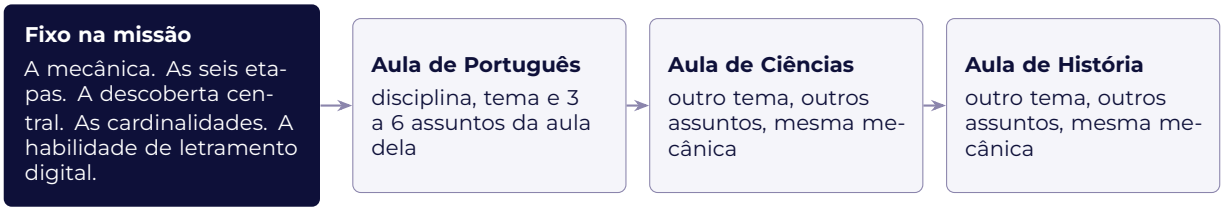


Figura 4: O recorte da aula. Toda missão sai com a tabela "a mesma missão em três aulas" como prova de que a transversalidade não é retórica.

Duas decisões sustentam isso, e as duas são CONVENÇÃO declarada:

- **A professora não declara habilidades BNCC da própria disciplina.** A missão responde pelo currículo de Computação; o conteúdo da aula dela entra como recorte, não como competência a comprovar. Isso evita que a esteira tenha de auditar uma matriz curricular que não é a dela.
- **Toda missão entrega a tabela das três aulas** — Português, Ciências/Biologia e História. Se a mesma mecânica não sobrevive às três, ela não era transversal; era temática, e o marcador `[LIMITE DE TRANSVERSALIDADE — dono: produto]` registra isso em vez de esconder.

2.5 A missão tem duas visões, e as duas são entregáveis

Um erro que a esteira cometeu em três missões seguidas: entregar só a visão do estudante. A tela em que a professora prepara a aula — escolhe o recorte, revisa o que a IA gerou, aprova e libera para a turma — era descartada pela própria regra de priorizar o pico e o "ahá".

Hoje a **visão do professor** é entregável obrigatório, com cinco telas fixas (`E0-PROF-01` a `E0-PROF-05`) que aparecem na jornada, num segundo grafo de fluxo, na folha do Stitch, no espelho do Miro e no protótipo. Faltando, o QA de design, a cobertura de telas por identificador e a banca cobram; e, como o Stitch é gerado pelo código a partir do inventário, as telas do professor não caem mais por "prioridade".

PARTE II

Como usar

A entrada é a única parte do sistema que depende inteiramente de você — e é a que mais muda o resultado.



uncoria®

3 Como escrever a entrada

A esteira é boa em **estruturar**, **auditar** e **formalizar**. Ela é ruim em adivinhar o que você não disse — e, por desenho, prefere marcar uma lacuna a inventar conteúdo. Toda a qualidade que ela pode acrescentar depende de haver matéria-prima para trabalhar.

3.1 O que o primeiro agente precisa

O agente 01 (Núcleo narrativo) é o único da esteira que lê os seus anexos. Ele produz quatro coisas, e precisa de matéria-prima para as quatro:

A descoberta central
O que o estudante vai saber depois que não sabia antes — escrito como descoberta, não como objetivo de aula.

A narrativa
Quem ele é na simulação, o que está em jogo e por que a tarefa importa.

O mecanismo
A cadeia de ações que produz a virada, distribuída nas seis etapas.

A identificação
Ano, bimestre e ambiente — os três entram no identificador da missão.

3.2 Um briefing que funciona

```
Missao para o 5o ano do Fundamental, 3o bimestre, numa rede social
de fotos como o Instagram.

Tema do bimestre (material Entre Redes anexado): como a recomendacao
de conteudo funciona e por que duas pessoas veem coisas diferentes.

O que quero que a crianca descubra: que o que ela ve no feed nao e
"o que existe", e sim o que o sistema aprendeu que ela costuma curtir.

Ideia de mecanismo: ela escolhe os posts que um personagem curte, a
partir de um banco de posts. Depois ve o feed que aquele personagem
passa a receber. Dois colegas com curtidas diferentes recebem feeds
diferentes, e isso fica visivel na tela.

Restricoes: 50 minutos, aula unica, dois estudantes por tablet.
A professora precisa conseguir pausar a turma antes da revelacao.

Em aberto: nao sei se o banco deve ter 8 ou 12 posts.
```

Cinco coisas fazem esse briefing funcionar:

1. **Ano e bimestre explícitos.** O identificador da missão codifica os dois (Seção 5); sem eles, o primeiro portão determinístico já reprova. O ambiente pode vir na língua do estudante (“como o Instagram”): a esteira o traduz para a seção atual, o Unoovista.
2. **A descoberta está escrita como descoberta.** Compare: “a criança vai aprender sobre algoritmos de recomendação” (objetivo de aula) contra “o que ela vê não é o que existe” (descoberta). A segunda pode ser revelada por uma tela; a primeira, não.
3. **Há um mecanismo, não só uma intenção.** “Curte os posts, vê o feed que resulta, compara com o colega” é uma cadeia de ações. “Trabalhar pensamento crítico sobre algoritmos” não é — e o gate estrutural tem um critério só para essa diferença.

4. **As restrições de sala aparecem.** Tempo, dispositivo compartilhado e controle da professora mudam o desenho, e a esteira audita os três.
5. **A dúvida está declarada.** Dizer "não sei se 8 ou 12" faz a esteira decidir e *registrar* a decisão, com base declarada. Omitir faria ela decidir sem que você soubesse.

3.3 Briefings que produzem missão ruim

Entrada problemática	O que acontece
"Missão sobre *fake news* para o Fundamental."	Sem ano, sem bimestre, sem ambiente, sem mecanismo. A esteira inventa os quatro, e o que ela inventa é o mais genérico possível. **É o pior uso da ferramenta.**
"O estudante navega por um feed com posts falsos e descobre que foram manipulados."	O briefing **entrega a virada**. Há um portão dedicado a isso (agente 02) e ele vai reprovar — mas o estrago já aconteceu: a ideia nasceu sem segredo, e refazer o núcleo custa rodadas do orçamento.
"Uma missão que desenvolva pensamento crítico e cidadania digital."	Intenção sem mecanismo. O teste do gate estrutural é literal: se você remover a frase, a missão continua igual? Então a frase não descreve nada construível.
"O estudante avalia 15 perfis e classifica cada um como confiável ou não."	Passa no teste de decisão significativa e reprova no volume e na monotonia: quinze decisões do mesmo tipo excedem o que a faixa sustenta. Vira **ressalva** ou reprovação no agente 09.
"Fazer no Roblox, que as crianças amam."	Ambiente fora da lista fechada. Reprovação imediata — não existe seção de *design system* para construir.
"A criança conversa com a IA e reflete sobre o que aprendeu."	Reflexão genérica. O critério pede uma *provocação concreta*: uma situação nova em que ela aplique o que viveu. O teste é direto — alguém que não fez a missão consegue responder?
Briefing de oito páginas com todas as telas já desenhadas.	A esteira vira transcritora. Os portões detectam "eco" — quando um agente repete a entrada em vez de construir — mas você terá pago cinquenta chamadas para formatar o que já tinha.

3.4 Anexos valem mais que adjetivos

Anexe o material do bimestre — o *Entre Redes* em `.docx`, o PDF da coleção, o plano de aula. Ele é a fonte primária de código curricular e de contexto: a esteira lê os códigos da BNCC dali e os confere contra o anexo oficial do MEC. **Dois parágrafos de material real valem mais que uma página de descrição.**

O que a esteira aceita como anexo

PDFs entram **em binário**, aproveitando a capacidade multimodal nativa do modelo — diagramas, tabelas e *layout* visual são interpretados, não apenas o texto. Documentos Word são convertidos para Markdown estruturado, preservando tabelas, listas e títulos. O limite do lote é de **10 arquivos** e **25 MB** agregados.

3.5 Quando você já tem o *one-pager*

Às vezes o PO não chega com meia página: chega com o *one-pager* inteiro, já discutido com o time, e quer que a esteira o leve até o PRD. Anexe-o numa conversa nova. A partir de 8 de outubro, **o *one-pager* anexado pelo PO é a missão**: a esteira completa e confere, não redesenha.

- **A ideação é dispensada.** Não há três mecanismos candidatos quando o PO já escolheu um.
- **Os agentes de Produto recebem o *one-pager* do PO** — seções de apoio, gate estrutural, revisor, os três pareceres e o gate final — com uma regra só: o que eles acharem que deve mudar e o PO não pediu vira `[PROPOSTA — validar com PO: …]` **ao lado** do texto dele, que fica.
- **Cada trecho é conferido em código.** Cada linha e cada célula de tabela do *one-pager* do PO tem de estar no gerado: igual, ou com 85% dos trigramas e os números e nomes intactos. Saem da conta só duas coisas — a habilidade que o pedido manda remover, e a alteração que o autor declara na tabela "Alterações sobre o *one-pager* do PO" citando, entre aspas, o trecho do pedido que a manda fazer. A citação também é conferida.
- **A conferência roda duas vezes.** Logo depois das seções de apoio, devolvendo ao autor o que ele reescreveu; e no fim, depois do gate estrutural, dos pareceres e dos revisores — os que mais reescreviam. Trecho perdido ali vai a um **restaurador**, que o devolve literal; se ainda faltar, Produto fica ● e o *de acordo* é recusado.

Como pedir

Na mensagem, diga o mínimo: *"use o one-pager anexado; não alterar nada"*, mais as correções que você quer, se houver, no formato de pedido da Seção 4.4. Não repita na mensagem o que o *one-pager* já diz — habilidades, tempos, seção do DS: a mensagem vira briefing, e o que ela listar a fidelidade passa a cobrar dela também. Na missão do 6.º ano, uma conversa nova com o *one-pager* corrigido e esse pedido curto preservou os 304 trechos do PO (Parte VII).

3.6 O checklist antes de apertar o botão

Cinco perguntas que evitam a maioria das reprovações

- ☐ **Descoberta sem *spoiler*.** O conceito é algo que o estudante descobre agindo, ou já está escrito no enunciado?
- ☐ **Ofício claro.** Qual é o papel que ele assume na narrativa, e esse papel justifica as ferramentas que ele usa?
- ☐ **Letramento digital ativo.** Em qual tela ele *pratica* segurança, ética ou discernimento de informação — e não apenas lê sobre isso?
- ☐ **Piso de realidade escolar.** Isso roda numa escola sem dispositivo próprio e com sinal instá-

vel em sala?

- [ ] **Roda de conversa.** O fechamento permite à professora aprofundar o debate com a turma depois da tela?

4 Rodar a esteira

4.1 Entrar

O aplicativo exige login, e não existe modo aberto: sem as credenciais configuradas, ele não sobe. São **quatro contas fixas, uma por papel** — `produto`, `design`, `dev` e `admin` —, sem provedor de identidade externo. A sessão vale quatro horas contadas do login e sobrevive ao recarregar a página.

As contas são papéis, não pessoas

O registro diz "Design", não qual designer. Isso é deliberado: o que precisa ficar registrado é *quem respondia pela etapa*, e a conta `admin` que age por outro papel aparece como "admin (como Produto)" — nunca como o papel que ela imitou.
Quem age em cada etapa: **Produto** — `produto` e `design`; **Design** — `design`; **Dev** — `dev`; o `admin` em todas. Missão nova é criada por `produto`, `design` ou `admin`.
O login é da aplicação, não da rede: os endereços temporários de download continuam acessíveis a quem tiver o link exato.

4.2 O parecer humano entre etapas

Esta é a mudança mais importante desde a primeira edição. A esteira **não roda mais do briefing ao PRD sozinha**: ela para ao fim de cada etapa e espera quem responde por ela.

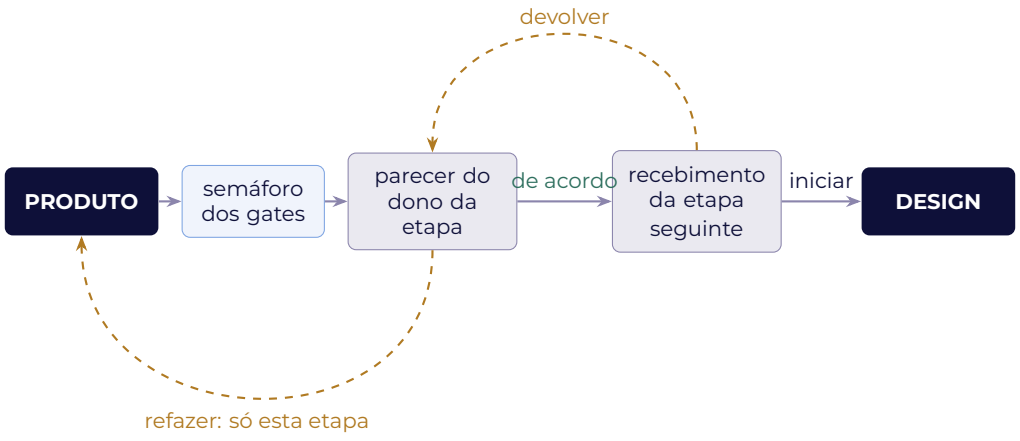


Figura 5: A passagem entre duas etapas. O mesmo desenho se repete de Design para Dev; o *de acordo* do Dev fecha a missão.

São quatro momentos, nesta ordem:

1. **O semáforo, que é da máquina.** Os gates da etapa, ● ● ●, como sempre. Ele não decide nada

sozinho — informa.

2. **O parecer do dono da etapa.** *De acordo* libera, e exige um comentário — que não é formalidade: ele segue para os agentes das etapas seguintes, no bloco "Notas de passagem". *Refazer* transforma o comentário em pedido de correção e refaz **só aquela etapa**, voltando ao parecer da mesma pessoa. A etapa refeita parte da **versão anterior** dos documentos, não de uma folha em branco: o que não foi pedido continua como estava.
3. **O recebimento pelo dono da etapa seguinte.** *Iniciar* dispara a etapa dele. *Devolver*, com comentário, reabre o parecer anterior — e **não refaz nada sozinho**: quem recebeu a devolutiva decide entre refazer, reafirmar o *de acordo* ou, se vê que a causa vem de antes, **repassar a devolução** à etapa anterior sem refazer o que não é dele — Design devolve a Produto o que o Dev lhe devolveu. O *Refazer* que vem depois leva a cadeia inteira de devolutivas (Dev → Design → Produto), não só a última.
4. **O fechamento.** O *de acordo* do Dev encerra a missão: diagnóstico, aprendizado e registro no catálogo rodam só aí. Missão entregue ainda pode voltar: quem responde por Dev (ou o `admin`) a *devolve* a Design, com comentário, e ela reabre ali sem rodar nada sozinha.

De acordo não muda a missão. _Refazer_ muda.

O *de acordo* só registra que a etapa está pronta **como está**. O comentário dele vai para os agentes das etapas seguintes como contexto, mas **não entra em documento nenhum** — e o do Dev, a última etapa, não tem etapa seguinte para lê-lo.
Por isso a esteira **recusa** o *de acordo* cujo comentário parece trazer conteúdo: "Pergunta 1", "Alternativas", "Cenário:", "AJUSTE 2", "TRAVAS" — ou, na última etapa, mais de 400 letras. A mensagem diz o caminho: use **Refazer** com o mesmo texto, e a etapa é refeita aplicando-o. Na missão do 9.º ano, as perguntas reescritas de um canal inteiro foram escritas num *de acordo* do Dev e não chegaram a documento nenhum (Parte VII).

O _one-pager_ aprovado só muda em Produto

Design e Dev são conferidos contra o *one-pager* que o PO aprovou (Seção 14). Por isso um *Refazer* de Design ou de Dev que mexe no que ele decide — texto de tela, Canal B, debate, podcast, habilidade, tempo — seria desfeito pelos próprios gates. O caminho é **devolver até Produto**, e a tela diz isso no cartão do parecer. Pedido ou parecer que contradiz o *one-pager* aprovado não é aplicado: o agente registra `[CONFLITO — pedido do PO × one-pager aprovado: …]` e a decisão volta ao PO.

Gate vermelho e a exceção do admin

Com um gate ●, o botão *De acordo* **não aparece**. Só a conta `admin` libera, e a liberação entra no dossiê marcada como **exceção** — a esteira não esquece que alguém passou por cima de um bloqueio.
As etapas rodam em segundo plano: dá para fechar a aba, e a missão volta para a fila *Aguardando você* de quem dá o próximo parecer. Um batimento de 30 segundos detecta execução morta — reinício ou *deploy* no meio — e restaura o backup. **Não faça _deploy_ com missão gerando.**

4.3 Os pedidos de refazer se acumulam

Cada *Refazer* é um pedido do PO — ou de quem responde pela etapa — e **todos continuam valendo**, do mais antigo ao mais novo. Um pedido não apaga o anterior; onde dois se contradizem, vale o mais novo. Eles chegam a **todas as etapas seguintes**, não só à que foi refeita: Design e Dev leem o que o

PO pediu ao Produto.

Até 6 de outubro não era assim. O pedido de correção era uma caixa só, e o segundo *Refazer* substituía o primeiro. Na missão do 9.º ano o primeiro pedido definia quem era o "caminho A" e quem era o "caminho B" e trazia a frase da descoberta inteira; o segundo pedido apagou o primeiro, e a missão saiu com os caminhos trocados.

O pedido mais recente do PO vence o briefing original — em todo gate. Na missão do 6.º ano, o briefing de 28 de setembro marcava a EF06CO07 como obrigatória; o PO pediu três vezes para tirá-la, e a esteira a recolocava a cada *Refazer*, porque o *lint* do *one-pager* lia só o briefing e mandava devolver a habilidade "sumida". Hoje, código que um pedido do PO manda retirar (remover, desvincular, descartar…) não conta como sumido.

4.4 O que você pede é conferido

Pedir não basta: a esteira **confere se o pedido foi atendido**. Um verificador de fidelidade lê todos os pedidos de *Refazer* e o briefing, transforma o que eles dizem numa lista de itens e pergunta a um juiz, item a item, se os documentos fazem aquilo. O juiz só pode responder "atendido" **citando o trecho do documento que prova** — e o código confere que a citação existe mesmo, letra por letra. Ele roda em quatro pontos: dentro do ciclo do núcleo narrativo, onde o autor ainda reescreve; no fim de Produto, sobre o núcleo, o *one-pager* e a *spec*; e, quando o *Refazer* foi pedido a Design ou a Dev, no fim de cada uma delas, sobre os documentos daquela etapa. Cada etapa confere o que foi pedido a ela, e Produto deixou de cobrar o que foi pedido ao Design.

Pedido do PO sem prova é ●, e com ele o *de acordo* daquela etapa não aparece. Elemento do briefing sem prova é ●. O verificador lê o briefing também quando ele vem em prosa ou com títulos em maiúsculas — e uma ordem de "não alterar" vira trava — e lê **só o texto do pedido**: até 8 de outubro ele lia junto o cabeçalho que a esteira põe no pedido, citava "Esta execução refaz a etapa…" como se fosse item, e contava o mesmo pedido duas vezes.

Como escrever um pedido que a esteira consegue conferir

O verificador reconhece três blocos no texto do *Refazer*, além de frases entre aspas. Escrever nesse formato é o que transforma um desejo numa conferência:

TRAVAS: O que **não pode mudar**. A frase entre aspas dentro das travas é conferida **letra por letra**, sem juiz: *TRAVAS: a frase da descoberta é "…que cada um tinha no sistema — e quase deixei de fora quem mais precisava de ajuda". Não trocar.*

AJUSTE 1 — Uma mudança por bloco, numerada: *AJUSTE 1 — Caminho A é quem não perguntou; caminho B é quem perguntou.* Nomes, números e definições valem ao pé da letra: trocar o rótulo é violar.

VALIDAÇÃO: As perguntas que **você** faria ao ler a missão pronta: *VALIDAÇÃO: o caminho A é o da maioria? A frase aparece inteira na tela do "ahá"?* O juiz responde cada uma, com citação.

Uma ideia por frase, com o nome das coisas. "Melhore o ápice" não é conferível; "no ápice, o estudante vê o projeto que ele recusou atendendo 24 famílias" é. "Algo como" dá liberdade de redação, não de sentido: a reescrita tem de manter sujeito, consequência e a parte final.

4.5 O azul é decisão do PO

Às vezes o pedido esbarra numa regra da própria esteira. No 9.º ano, o briefing pedia que o estudante "confirme ou mude a escolha" depois da virada, e a regra do núcleo exigia escolha irreversível. Nesse caso o juiz não escolhe um lado, e o autor também não: o item sai ● — **conflito entre o pedido e uma regra** — e vai para a lista do que só uma pessoa decide, com os dois trechos citados.

> ● não é defeito da missão nem do pedido. É a esteira dizendo que a autoridade para resolver aquilo é do PO. Leia, decida e registre a decisão num *Refazer*.

A regra da escolha irreversível, aliás, foi esclarecida pelo mesmo caso: ela proíbe **apagar** a primeira escolha, não fazer uma **segunda** depois da virada. "Confirma ou muda", com a primeira registrada e travada, e a segunda também travada e justificada, é permitido.

4.6 O agente não decide no lugar do PO

O azul cobre o conflito declarado. Há um caso mais sorrateiro: o agente que decide e escreve que foi o PO. As três gerações da missão do 3.º ano escreveram "Candidato A aprovado pelo PO", "Quem decidiu: PO (via *One-pager* v2.2)" — e nenhuma delas tinha parecer do PO. O PO lê "aprovado" e não revisa.

O que o documento diz	O que a esteira faz
`[PROPOSTA — validar com PO: …]`	Decisão do agente, declarada como dele. Não bloqueia, mas **tira a etapa do verde**: ela sai ● e a proposta chega ao parecer. É o único jeito de um agente decidir — e só quando há ganho pedagógico ou funcional claro.
`"aprovado pelo PO", "decidido pelo PO"`	Se não há nenhum registro do PO na sessão — pedido de *Refazer*, parecer, pedido de correção, *one-pager* anexado —, **bloqueia**: o autor reescreve como proposta. Com registro, fica ●: o código não sabe se o parecer diz aquilo, e quem confere é uma pessoa.
`[CONFLITO — critério × frase do PO]`	A rubrica pede uma coisa e o PO decidiu outra. O agente segue o PO e registra o conflito; quem resolve é o PO. Restrição explícita do briefing — seção do DS fixada, "é proibido", habilidade obrigatória — não se troca.

4.7 Perguntar a uma etapa, sem mudar nada

Com o fluxo de pareceres ativo, o campo de mensagem da missão não fica travado: ele vira **conversa com as etapas**. Você escolhe Produto, Design ou Dev e pergunta — "por que este gate ficou vermelho?", "onde está a decisão sobre o banco de posts?" — e um agente de fora da esteira responde lendo os documentos daquela etapa.

- A conversa **só lê**. Mudar a missão continua sendo só pelo *Refazer* do cartão de parecer, que passa pelo dono.
- Ela fica no histórico da missão, mas **fora dos artefatos** — não entra no ZIP, no manifesto nem no PRD.
- **Todos que abrem a missão veem, e qualquer papel pode perguntar.** Uma dúvida do dev sobre a decisão de produto é exatamente o tipo de coisa que precisa ficar visível.

- O botão *Rascunhar o comentário do parecer* leva a conversa para o campo do comentário, para o dono revisar antes de assinar.

4.8 Corrigir uma missão já gerada

Cada mensagem numa conversa *nova* cria uma missão do zero. Para corrigir uma que já existe, abra-a no histórico: o campo de mensagem passa a pedir correções nela.

1. Você escreve o pedido, com anexos se precisar.
2. Um agente de triagem, **fora da esteira**, decide de qual etapa recomeçar — *spec*, design, dev ou nenhuma — e mostra a decisão com o motivo. Você confirma ou troca. O que o *one-pager* fixa — habilidades, Canal B, debate, podcast, tempos, parâmetros — a triagem manda sempre para Produto.
3. A esteira roda **só dali em diante**, sobre o estado salvo. Refazer da *spec* custa cerca de vinte minutos; do design, quinze; só o dev, sete.
4. O pedido entra numa chave de estado que **todo agente lê**: os autores aplicam no que é seu e registram "o que mudou nesta versão"; o QA e a banca conferem se foi atendido.
5. O que seria perdido é copiado para uma pasta oculta antes de sobrescrever — e há **desfazer a última correção**, que restaura documentos, estado e conversa.

Com o fluxo de pareceres ativo, a correção livre pelo chat fica desligada, porque o *Refazer* do cartão faz o mesmo passando pelo dono.

4.9 Rodar uma etapa isolada

Quando você está iterando sobre um *prompt* ou investigando um defeito, rodar a esteira inteira é desperdício. Há duas formas:

- `PIPELINE_STAGES` roda só as etapas pedidas sobre um estado já salvo;
- `scripts/rerodar_etapa.py` reexecuta Design, Dev ou só o diagnóstico sobre uma sessão salva — inclusive uma que veio do *bucket* do Cloud Run.

> **Rodar só a etapa de Dev sobre um estado salvo custa cerca de dois minutos, contra vinte da execução completa** — e, o que importa mais, a entrada fica *fixa* entre as tentativas, o que é a única forma de saber se a mudança que você fez teve efeito.

4.10 Os *prompts*, para ler

A seção *Prompts e referências* do aplicativo mostra cada *prompt* e cada referência que os agentes leem — **só para leitura**. Os *prompts* vão na imagem do aplicativo, e mudá-los é mudar o código, com revisão e teste; editar pela tela durava até o próximo *deploy*. Um botão prepara um `.zip` com todos eles em duas formas: como estão no repositório, com as diretivas de inclusão, e como o modelo os recebe, com as inclusões resolvidas. E este manual, em PDF, fica na barra lateral, logo abaixo do horário em que a sessão expira.

4.11 Configuração

Os padrões foram calibrados na operação real e raramente precisam mudar. As exceções são `PIPELINE_STAGES`, que muda a cada sessão de trabalho, e os três interruptores de rigor.

Variável	Padrão	O que faz
FLUXO E ACESSO		
`PARECER_HUMANO`	true	A esteira para ao fim de cada etapa e espera o parecer do dono.
`AUTH_CREDENCIAIS`	—	**Obrigatória.** Hashes das quatro contas. Sem ela o aplicativo não sobe.
`AUTH_SEGREDO_JWT`	—	**Obrigatória.** Chave da sessão. Trocá-la derruba todas as sessões.
`AUTH_HORAS_SESSAO`	4	Validade absoluta da sessão, contada do login.
`PIPELINE_STAGES`	todas	Roda só as etapas listadas, sobre um estado salvo.
`BASE_PLATAFORMA_DIR`	—	Onde está a base da plataforma e do DS (Seção 16). No servidor, o *bucket* próprio da base, montado só para leitura em `/app/base-plataforma`; o *deploy* manual recusa subir sem ela. Vazia, a esteira a procura em `<OUTPUT_DIR>/base-plataforma`. Sem base, a execução não começa.
RIGOR DOS PORTÕES		
`ESTEIRA_ESTRITA`	true	Reprovação marca herança degradada e o PRD sai 🔴. Desligado, vira ressalva.
`PRODUTO_MODO_ESTRITO`	true	Gate de produto reprovado impede a liberação para as etapas seguintes.
`BNCC_CHECK_ESTRITO`	true	Currículo não conforme bloqueia Design e Dev.
`PROFUNDIDADE_ESTRITA`	true	Profundidade reprovada na última rodada do núcleo fica 🔴 e o *de acordo* de produto é recusado. Em `false`, rebaixa a 🟡 — para usar **enquanto os cortes da rubrica são calibrados**.
`KPIS_CHECK_ESTRITO`	true	Incoerência entre indicador, evento e tela impede o PRD.
`BANCA_ENTRADA_ESTRITA`	true	Banca de entrada do Dev 🔴 impede todo agente de Dev de rodar: construir sobre o que vai mudar é desperdício. Em `false`, o Dev roda e o 🔴 fica no semáforo.
TETOS DE CORREÇÃO		
`GATE_ESTRUTURAL_MAX_RODADAS`	3	Ciclo gate estrutural ↔ revisor do *one-pager*.
`SPEC_MAX_RODADAS`	2	Ciclo *spec* ↔ currículo, manifesto, evidência, perguntas, conteúdo aprovado e banca de produto. Vale também para o ciclo do *one-pager* resumido.
`AVALIACOES_MAX_RODADAS`	2	Ciclo do parecer de adequação: o revisor aplica o ajuste exigido e as avaliações refazem. Em 1, o parecer entra como ressalva.
`QA_DESIGN_MAX_RODADAS`	2	Ciclo QA de entregáveis ↔ retrabalho de design.
`HANDOFF_MAX_RODADAS`	2	Ciclo *handoff* ↔ contrato de telemetria.
`BANCA_MAX_RODADAS`	2	Ciclo banca ↔ dono do achado.
`PRD_MAX_RODADAS`	2	Ciclo redator do PRD ↔ conferências do documento. Vale também para o ciclo do *one-pager* pedagógico.
`ORCAMENTO_REEXECUCOES`	8	Teto **global** de reexecuções na execução inteira. Conta o autor de cada conserto e quem chama modelo — não a cadeia inteira de passos em código que vem atrás dele.

Variável	Padrão	O que faz
MODELO		
`GEMINI_MODEL`	—	Modelo padrão de todos os agentes: `gemini-3.8-flash`. A variável `GEMINI_MODEL_PRD` permite outro só para o redator do PRD.
`THINKING_BUDGET`	8192	*Tokens* de raciocínio por chamada.
`STITCH_PARALELO`	4	Telas geradas ao mesmo tempo no Stitch, fora do modelo (Seção 15). O limite é a cota do Stitch.
`TEMPERATURA_JUIZ`, `SEED_JUIZ`	0,1 e fixa	Os juízes rodam com temperatura baixa e semente fixa: dois julgamentos do mesmo texto têm de dar o mesmo veredito.

Qualquer teto em 1 desliga o ciclo correspondente: a esteira avalia e reporta, sem corrigir. É útil para medir a qualidade bruta de um *prompt*, e péssimo como configuração de produção.

5 O identificador da missão

O código da experiência carrega ano e bimestre, e é ele que nomeia tabela, pasta e *branch*. Por isso um portão determinístico confere que os campos concordam entre si — divergência trava.

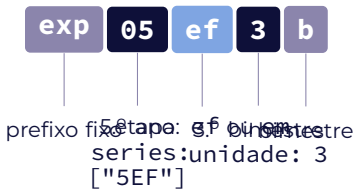


Figura 6: Anatomia de `exp05ef3b`. O portão confere que `series` e `unidade` concordam com o código.

Código	Significa	Campos que têm de concordar
`exp01ef1b`	1.º ano EF, 1.º bimestre	`series: ["1EF"]` · `unidade: 1`
`exp05ef3b`	5.º ano EF, 3.º bimestre	`series: ["5EF"]` · `unidade: 3`
`exp02em2b`	2.º ano EM, 2.º bimestre	`series: ["2EM"]` · `unidade: 2`

Missão transversal (sem série definida) e ferramenta de professor não têm ano nem bimestre: seguem no formato antigo `exp-NN-<slug>`. No histórico da interface, os registros passaram a começar pelo ano e bimestre — `2EF-1B · O Detalhe que Muda Tudo` — lidos do próprio identificador. Quando não há como deduzir, o título sai sem prefixo, em vez de inventar um.

6 Como ler o diagnóstico

O `diagnostico-execucao.md` é o primeiro arquivo a abrir, e tem sempre a mesma ordem. Lê-se em trinta segundos e decide se vale ler o resto.

1. **Veredito, contagem e base** — impedimentos, ressalvas e decisões humanas, em números; e de qual retrato da plataforma e do DS a missão foi escrita.
2. **Como cada portão se saiu** — tabela com resultado e rodadas gastas.
3. **O que reprovou, na ordem** — a sequência dos fatos, não a lista de culpados.
4. **O que precisa ser consertado, por dono** — agrupado por responsável.
5. **O que só uma pessoa decide** — o que nenhuma rodada de correção resolve.
6. **O que saiu provisório** — os documentos que dependem de algo reprovado.
7. **Documentos ausentes** — sintoma de agente que não produziu nada.
8. **Por onde começar** — a ação que destrava mais coisa de uma vez.

Figura 7: As oito seções do diagnóstico, sempre nessa ordem.

A linha da base. Logo abaixo do veredito, o diagnóstico diz de qual versão da plataforma a missão foi escrita — o *commit*, a data e a versão do *design system*: *"Base da plataforma: plataforma-unooria@89e0f9ae (…, 2026-10-06) · design system 0.2.18"*. Se a plataforma mudou muito depois disso, a missão precisa ser relida contra a plataforma de hoje. Execução antiga, de antes de 6 de outubro, diz "não registrada".

"Nada travando" significa nada travando. Quando algum portão pediu uma pessoa — uma decisão ●, uma colisão de identificador, um juiz que marcou "precisa de humano" —, o diagnóstico lista a decisão e **não** diz mais "nada travando". O ● que pede pessoa também abre expandido na tela, como o ●.

As três severidades, e o que cada uma pede de você

Impedimento O que impede construir. Tem conserto certo e, quase sempre, dono dentro da esteira. Se sobrou, é porque o teto de rodadas ou o orçamento global estourou.

Ressalva Risco aceito e registrado. Não bloqueia, mas **nenhuma ressalva sobrevivente pode ser omitida do *one-pager***, mesmo com veredito verde — há uma regra explícita contra isso.

Decisão humana O que a esteira não tem autoridade para resolver: escolha de produto, licença de imagem, homologação de autoria, validação pedagógica, conflito entre o pedido do PO e uma regra (●), identificador de missão que colide com outro já registrado. Não é defeito da missão; é informação que faltou de quem forneceu o insumo, ou uma escolha que é de alguém.

Verde é uma afirmação sobre o que o portão confere — não sobre a missão

Cada portão responde a uma pergunta estreita, e o verde dele quer dizer só isso. O rastro de lacunas, por exemplo, procura os marcadores que os agentes deixaram (`[ABERTO — dono]`, `[DECISÃO PENDENTE]`); o verde dele dizia "sem lacunas" e hoje diz o que de fato mede: **"nenhum marca-**

> **dor de lacuna”**. O que falta e ninguém marcou não aparece ali — a fidelidade ao pedido e a coerência entre documentos são conferidas em outros portões. A missão do 9.º ano saiu com “0 lacunas” e com defeitos que só a leitura humana achou (Parte VII).

6.1 Quando a etapa para com uma mensagem

A esteira não aborta por gate reprovado — mas uma etapa pode parar porque um documento obrigatório não existe. O caso que ensinou isso foi o modelo que **corta a própria resposta**: o filtro de recitação do Gemini devolve uma resposta vazia quando ela transcreve texto publicado por extenso, como o texto oficial de uma habilidade da BNCC. Hoje três coisas acontecem, nesta ordem:

1. Todo agente registra o corte e o motivo, e a tentativa seguinte do mesmo agente recebe o aviso no topo da instrução — “não transcreva texto publicado por extenso”, no caso da recitação; “seja enxuto”, no do limite de tamanho. Repetir com a mesma instrução dava o mesmo corte.
2. O *one-pager* vazio reprova a conferência das seções de apoio, e o ciclo chama o autor de novo, já com o aviso.
3. Se continuar vazio depois das duas tentativas, a etapa para **antes** do gate estrutural, com uma mensagem que diz qual agente ficou mudo e por quê. Antes, ela quebrava adiante com “*Context variable not found: one_pager*”, que nomeia a variável e não quem deixou de escrevê-la (Parte VII).

7 Três recomendações de uso

Comece pelo diagnóstico, não pelo PRD.
Ele diz em trinta segundos se vale ler o resto, e dá a ordem de ataque. E leia a missão mesmo com tudo verde: o semáforo confere o que é mecânico, e a pessoa que dá o parecer responde pelo resto.

Trate `[PROVISÓRIO]` como rascunho, nunca como entrega.
O marcador existe porque o documento foi construído sobre algo que um portão reprovou — ele pode estar certo, mas ninguém verificou.

Rode uma etapa isolada quando estiver iterando.
`PIPELINE_STAGES` sobre um estado salvo custa minutos em vez de uma execução inteira, e mantém a entrada fixa entre as tentativas.

PARTE III

Como é construída

Cento e catorze passos não são um capricho de arquitetura. São a resposta a um modo de falha medido.



uncoria®

8 Por que muitos agentes, e não um prompt grande

A pergunta aparece sempre, e a resposta não é “porque especialização é boa”.

8.1 Densidade de instrução degrada o cumprimento

A etapa de Produto sozinha carrega cerca de **74 mil *tokens* de instrução e 51 critérios tabelados**. No *benchmark* IFScale, que mede exatamente isso, os melhores modelos de fronteira chegam a 68% de acurácia quando precisam seguir 500 instruções ao mesmo tempo [24], com viés sistemático em favor das instruções apresentadas primeiro. Liu et al. [30] documentam o efeito irmão em contexto longo: o que está no meio é usado pior que o que está nas pontas.

Há evidência interna, e ela é mais convincente que a externa. O *prompt* do agente 09 abre justificando a própria existência:

> *“Esta avaliação existe porque a rubrica original errava sistematicamente aqui: os critérios de faixa etária competiam por atenção com ∼30 critérios de outra natureza.”*

A versão agregada falhava de modo reprodutível; a decomposta corrigiu. Não é teoria importada — é um defeito que se observou e se consertou.

8.2 O critério de decomposição

Não se divide por etapa cronológica. Divide-se por **decisão encapsulada**, no sentido clássico de Parnas [36]: cada agente responde a uma pergunta que pode ser respondida — e errada — independentemente das outras, e emite um envelope JSON que um portão consegue ler.

Agente	A pergunta que ele responde
05 Gate estrutural	A missão tem estrutura construível?
08 Avaliação pedagógica	Cumpre currículo e protege o estudante?
09 Experiência do estudante	Alguém dessa idade consegue viver isso?
30 QA de entregáveis	A construção é fiel ao que foi especificado?
48 Consistência do PRD	O documento final se contradiz?

8.3 Redundância não é cobertura

Três critérios do agente 09 eram testes idênticos a três critérios do agente 05 — mesma pergunta, mesmo modo de falha. Foram removidos. Knight e Leveson [27] mostraram, em 27 implementações independentes submetidas a um milhão de testes, que falhas correlacionadas anulam o ganho da redundância. Quando os dois “avaliadores independentes” são *prompts* diferentes do **mesmo modelo**, sobre o

mesmo texto, a correlação é ainda maior.

8.4 O contraponto honesto

A literatura recente é cética quanto a sistemas multiagente: sob orçamento fixo de *tokens*, um agente único frequentemente iguala ou supera arquiteturas multiagente [42]. Vale registrar isso num manual da casa, e não escondê-lo.

A condição de contorno declarada pelos próprios autores é que o multiagente vence *"quando a utilização efetiva de contexto de um agente único está degradada"* — que é exatamente o regime desta esteira, com 74 mil *tokens* de instrução só na primeira etapa. E Cemri et al. [12], sobre mais de 1.600 execuções anotadas, mostram que decompor cria uma classe nova de falha: **contratos de interface**. É por isso que cada fronteira aqui tem envelope JSON verificado por teste — a decomposição não sai de graça, e o preço dela é conhecido.

9 As três etapas e o mapa dos agentes

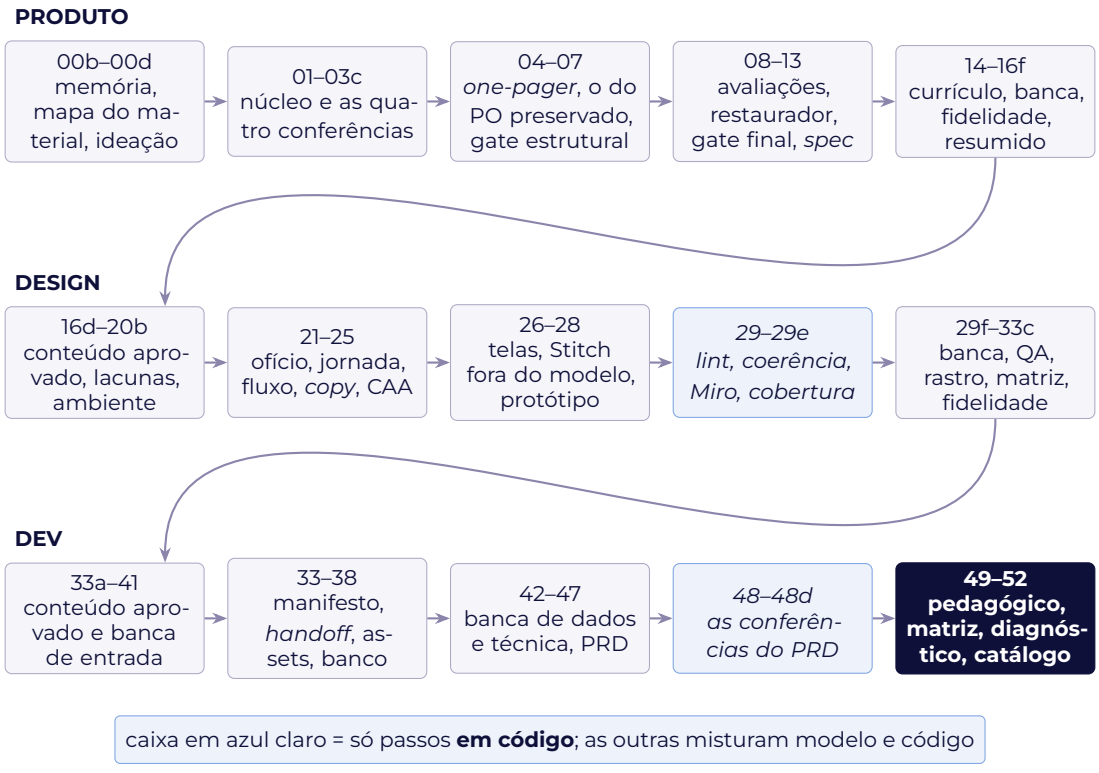


Figura 8: Os 114 passos, na ordem de execução. A numeração é a da tela de progresso, que não é a ordem: 03aa e 03a rodam dentro do ciclo do núcleo, e a banca de entrada do Dev usa os números 39 a 41 antes do 33. A última caixa inclui os três passos transversais. A Parte VI descreve cada um.

De onde vem a contagem

114 passos é a lista que o próprio código publica para a tela de progresso: 48 em Produto, 31 em Design, 32 em Dev e 3 transversais — 43 com modelo de linguagem e 71 em código. Seis dos passos em código — os quatro de fidelidade e os dois de coerência com o conteúdo aprovado

— chamam por dentro um juiz de modelo, mas quem dá o veredito é o código, que confere a citação.
Montada de verdade, a árvore de agentes que o orquestrador executa tem **160**: os 114 passos, mais **24 cópias** de autores e verificações que os ciclos de retrabalho reexecutam, mais **22 auxiliares internos** sem número próprio — nove juízes de modelo que moram dentro de verificações em código (os quatro de fidelidade, os dois de contradições e três que separam o que um texto proíbe do que ele afirma) e treze passos de código que gravam o escopo de cada banca, leem vereditos, aplicam o que os juízes decidiram ou separam um documento de outro.

9.1 Produto (00b–16f): a missão pode ser construída?

Decide *o que* a missão é, e é de longe a etapa mais densa — 48 dos 114 passos. Ela começa **antes** do núcleo narrativo:

- a **memória** (00b) recupera do índice as missões mais próximas do ano do briefing — as já geradas e as publicadas na plataforma —, para a nova não repetir uma anterior;
- o **mapa conceitual** (00c) lê o material do bimestre e extrai o conceito limiar, os conceitos de apoio, as crenças que o bimestre derruba e o vocabulário — *antes* de alguém inventar mecanismo;
- a **ideação** (00d) produz três mecanismos distintos para aquele conceito, pontuados na rubrica de profundidade. O núcleo desenvolve o escolhido. Com o *one-pager* do PO anexado, a ideação é dispensada: o mecanismo já foi escolhido.

Só então vem o núcleo narrativo, que sai do seu ciclo só quando passa em **quatro** conferências: o micro-gate de *spoiler*, o juiz de profundidade (Parte IV), a **conta da virada** — se a virada é um ranking que se inverte, o código refaz o ranking — e a **fidelidade ao pedido do PO**. Depois, o *one-pager* de 18 seções — que, havendo *one-pager* do PO, parte dele e é conferido contra ele trecho a trecho (Seção 3.5) —, os critérios do gate estrutural, as avaliações em paralelo, a conferência final do *one-pager* do PO com o restaurador, o gate final e a ***spec***: o contrato de 12 blocos, mais o **glossário canônico** com o nome único de cada coisa.

A *spec* **traduz o *one-pager* aprovado, não o revisa**. Até 6 de outubro ela era um segundo autor: aplicava as ressalvas ● dos pareceres que o PO tinha recusado, e tudo a jusante a seguia (Seção 14). Hoje a ressalva não incorporada vai para uma tabela própria do bloco 12, e o ciclo da *spec* confere, além de currículo, manifesto, evidência, perguntas e lacunas, a *spec* contra o *one-pager* aprovado e o parecer da **banca no fim de produto**. Fecha com a fidelidade sobre núcleo, *one-pager* e *spec* juntos, e com o *one-pager* resumido — em ciclo próprio, conferido contra o canônico.

Por que a conformidade curricular é conferida ainda em Produto

A ficha do manifesto (agente 15) e a conformidade BNCC (14) poderiam rodar em Dev, junto com o resto da verificação técnica. Rodam em Produto de propósito: um campo errado ali custa **uma reescrita da *spec***; o mesmo campo errado descoberto em Dev custa **o descarte do Design inteiro**, porque tudo que Design construiu se apoiou na *spec* errada.

9.2 Design (16d–33c): como isso se constrói

Começa gravando o **conteúdo aprovado** (16d): as seções do *one-pager* aprovado, literais, para os agentes de design que não leem o *one-pager*. Fecha as lacunas da *spec*, decide o ambiente simulado,

confere a *spec* consolidada contra o *one-pager* aprovado, escreve o *briefing* de ofício, e então constrói: jornada e fluxo em **diagramas Mermaid**, *copy* em duas passadas, camada de acessibilidade, especificação de telas — que o código gera no Stitch, todas, fora do modelo — e um **protótipo HTML navegável** que abre com duplo clique. Antes do QA, o código confere o arquivo do protótipo, a **coerência entre os documentos** contra o glossário canônico e a **cobertura de telas por ID** em cada documento. Depois vem a **banca no fim de design**, o QA de entregáveis e as afirmações do QA conferidas em código. Os achados voltam só para quem tem *ticket*, e cada um recebe a própria versão anterior e o *ticket* completo. Por último, o rastro cobra as lacunas que são de Design, a **matriz de rastreabilidade** mostra onde cada requisito do produto chegou, e a fidelidade confere o *Refazer* pedido a Design.

A regra que governa esta etapa inteira cabe numa linha: **Design preenche ausência, nunca altera presença.** Onde a *spec* decidiu, Design constrói; onde a *spec* calou, Design decide e registra a decisão com base declarada. Se a decisão pertence a produto, vira um pedido de re-*spec*, não uma escolha silenciosa.

9.3 Dev (33a–49c): isso é implementável?

Começa com o conteúdo aprovado e com a **banca de entrada** (39–41 e 33b): a segunda verificação de produto e de design já aprovados, com a visão cruzada que nenhuma etapa tem sozinha. Ela **não reescreve nada**; se bloqueia, nenhum agente de Dev roda, e o relatório aponta o dono de cada bloqueio (Seção 11). Passando, o Dev confere a ficha do manifesto, produz o *handoff* e o *brief* de assets, valida o contrato de telemetria, escreve o **banco de conteúdo** — o texto integral de cada documento fictício, tabela e roteiro da missão, e as falas da IA simulada — em ciclo com a conferência dele, e submete os documentos de Dev à banca de dados e de técnica. Escreve o PRD, que passa por quatro conferências em ciclo e recebe, colado no fim, o conteúdo aprovado literal. Por último, o *one-pager* pedagógico — que é o último documento justamente porque precisa dizer a verdade sobre tudo o que veio antes —, também em ciclo, também conferido contra o aprovado. Fecham a etapa a matriz de rastreabilidade final e a fidelidade ao *Refazer* pedido a Dev.

9.4 Transversal (50–52): o que aconteceu, e o que fica

O diagnóstico determinístico monta o dossiê a partir dos envelopes que os portões gravaram — roda **sempre**, inclusive quando tudo o mais falhou, e é essa garantia que sustenta a decisão de não abortar. O agente de aprendizado escreve a leitura em prosa.

O terceiro resolve um problema antigo: **o registro no catálogo** (52). Até ele existir, duas missões parecidas só eram percebidas por acaso, ou nunca. Agora a ficha entra no índice automaticamente ao fim da esteira, com o *status* derivado dos gates. Missão bloqueada **entra de propósito**: o que se quer evitar é reinventá-la, não escondê-la. **Promover é ato humano** — nenhum agente marca uma missão como aprovada. E identificador que colide com o de outra missão já registrada é **decisão humana**, não aviso: o diagnóstico confere a colisão antes de fechar, para que ela apareça no dossiê.

Desde 8 de outubro o índice conhece também **a grade publicada da plataforma**: uma ficha por missão do catálogo do *hub*, com *status* "em produção", importada pelo comando `importar-plataforma` de `scripts/catalogo.py`, que lê o repositório da plataforma pelo *git* sem tocar a pasta de trabalho. Eram 9 missões publicadas, 4 delas nascidas fora do gerador, que a verificação de semelhança não via. E a verificação deixou de comparar a missão com ela mesma: o identificador é ano mais bimestre, então **o mesmo identificador é a mesma vaga**, e a versão nova de uma missão não é repetição dela — a do 3.º ano tinha sete tentativas no *bucket*.

10 Os treze ciclos de correção

Quando um portão reprova, quem decide repetir é o **orquestrador, em código** — não uma instrução em linguagem natural dentro do *prompt*, que o modelo pode ou não seguir.

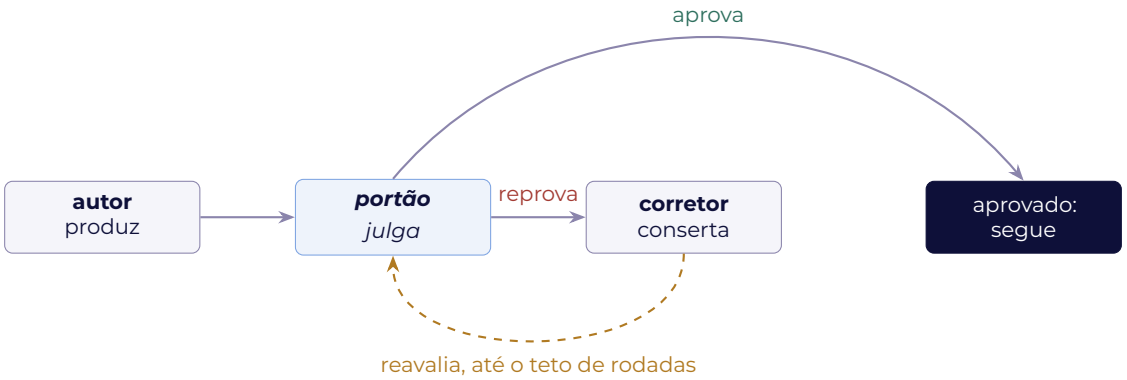


Figura 9: O padrão comum aos treze ciclos: quem julga não conserta, quem conserta não julga.

Ciclo	Portão → corretor	O que resolve
PRODUTO		
Núcleo	02, 02b, 03aa e 03a → 01	*Spoiler*, falta de segredo, escolha sem consequência, profundidade, a conta da virada que não fecha e o pedido do PO que o núcleo não cumpre. O núcleo só sai do ciclo quando passa nas quatro conferências; o autor recebe os relatórios e reescreve.
Seções de apoio	04d → 04	Trecho do *one-pager* do PO que o autor reescreveu sem pedido. Sem *one-pager* do PO, aprova na primeira rodada, sem custo. Também é aqui que o *one-pager* vazio, cortado pelo modelo, faz o autor ser chamado de novo.
Gate estrutural	05 → 07	Os critérios de estrutura, mais o *lint* de contrato de plataforma (04b), que roda a cada rodada. O gate escreve o texto de correção pronto para colar — é o ciclo mais barato da esteira.
Avaliações	10b → 10c	Os ajustes que o parecer de adequação exigiu: o revisor aplica e as três avaliações refazem. Até 7 de outubro vinha desligado e o parecer entrava só como ressalva; parecer com ajuste exigido também não encerra mais o ciclo como se fosse aprovação.
One-pager do PO	10d → 10f	A conferência final, depois de gate, pareceres e revisores: o que se perdeu do *one-pager* do PO vai ao restaurador, que o devolve literal. O que um parecer pedia vira proposta ao PO.

Ciclo	Portão → corretor	O que resolve
Spec	14, 15, 15b, 15e, 15f, 15g e 15h → 13	Código da BNCC inexistente ou de outro ano; campo do manifesto fora do vocabulário; habilidade sem evidência; pergunta de produto sem ação que a responda; lacuna de produto deixada na *spec*; a *spec* divergindo do *one-pager* aprovado; e o que a banca no fim de produto achou.
Resumido	16e → 16b	Habilidade, tempo em tela ou Canal B do resumido diferentes do canônico aprovado.
DESIGN		
QA de design	29f, 30, 30b e 29a → dono do *ticket*	Cada achado tem dono declarado; só os endereçados rodam de novo, e o protótipo, a coerência, a cobertura de telas e o *lint* são refeitos depois deles. Entram aqui a banca no fim de design, o *lint* do protótipo — que lê o arquivo HTML, não o texto —, as afirmações do QA conferidas e a *spec* consolidada que piorou em relação ao *one-pager* aprovado.
DEV		
Handoff	36 → 34	Contrato de eventos, papéis, canal de sala. A correção vem escrita na mensagem do portão.
Banco	35c → 35b	Item do *brief* sem texto no banco, podcast reescrito, árvore de intenções do chat ausente. Até 7 de outubro o banco rodava uma vez, e o achado ia direto ao dossiê.
Banca	42–44 → dono do achado	Dados, LGPD e técnica. Conserta **só documento de Dev** — *handoff*, *brief*, banco —, reconferidos depois. Achado sobre a *spec* ou o design vira pendência nomeada para a etapa dona: o Dev não reescreve Produto nem Design.
PRD	48, 48a, 48c e 48e → 47	Consistência interna, coerência dos KPIs, o PRD contra a plataforma, o design, o *handoff* e o banco, o PRD que ainda não dá para codar, e o corpo do PRD contra o *one-pager* aprovado (Seção 14.2).
Pedagógico	49b e 49e → 49	Habilidade, tempo, podcast ou caminho de arquivo diferentes do aprovado, e contradição comprovada com ele.

O autor recebe a própria versão. Nos ciclos da *spec*, do *handoff*, do PRD, do pedagógico e do resumido, e no retrabalho de design, quem conserta recebe a versão que escreveu na rodada reprovada, com a instrução de corrigir só o apontado. Antes, cada volta reescrevia o documento do zero — e consertava um achado criando outros.

Dois freios, e uma coisa que não volta

Dois freios. Cada ciclo tem teto próprio, e existe um orçamento global de reexecuções para a execução inteira (`ORCAMENTO_REEXECUCOES`, padrão 8). Estourado qualquer um dos dois, a esteira para de tentar, marca pendência humana e segue até o fim — e o conserto pulado vai ao dossiê com o dono. O orçamento conta o autor de cada conserto e quem chama modelo. Até 8 de outubro ele contava a cadeia inteira de passos em código atrás de cada dono: na primeira execução real do design novo, um conserto custava 11 de um orçamento de 8, e nenhum rodou em duas rodadas (Parte VII).

O que não volta. Achado cujo dono está numa etapa já aprovada não reabre aquela etapa sozinho: reabri-la invalidaria tudo que foi construído sobre ela. Vira pendência nomeada no diagnóstico, e o caminho é a devolução. Por isso cada etapa tem hoje o seu próprio rastro de lacunas (15f, 33 e 38) e a sua banca: a lacuna é cobrada onde ainda há quem a conserte.

11 A banca de validação, em quatro posições

A banca é a mesma régua — cinco dimensões com critérios numerados: produto e narrativa, design e imersão, pedagogia e BNCC, dados e LGPD, técnica e arquitetura. Até 8 de outubro ela rodava uma vez, dentro do Dev, depois do *handoff*; e de lá **reescrevia** jornada, fluxo, telas e *copy* depois que o Design já tinha recebido o *de acordo*. O Dev era validado sobre produto e design que ainda iam mudar.

Hoje a banca roda em quatro posições, com os mesmos critérios e o mesmo modelo. O que muda é o escopo, que um passo em código grava logo antes de cada uma.

Posição	Dimensões	O que pode fazer
Fim de produto (15h)	produto e pedagogia	Julga a *spec* com o conteúdo aprovado do *one-pager* ao lado, antes do parecer do PO. O achado volta ao autor da *spec*, dentro do ciclo dela.
Fim de design (29f)	produto realizado em tela, e design e imersão	Antes do parecer de Design. De produto, só o que se realiza em tela: o fluxo realiza o contrato de estados, a conclusão tem tela, o debate está nas telas do professor, cada modo de erro tem tela. O bloqueio vira *ticket* para o dono refazer nesta etapa.
Entrada do Dev (39–41, 33b)	produto, design e pedagogia	A **segunda verificação**, com a visão cruzada: cada estado da *spec* com tela e aresta no fluxo, o professor com as telas dele, os textos aprovados na *copy* como foram aprovados. **Não manda reescrever nada**: se bloqueia, o Dev não roda, e o relatório separa o que "passou na etapa e reprova aqui".
Dentro do Dev (42–44)	dados e técnica	Depois do *handoff*, do *brief* e do banco. Conserta só documento de Dev.

Por que manter a segunda verificação

A entrada do Dev repete critérios que já rodaram no fim de produto e no fim de design. É de propósito: ela existe para pegar **quem deu o *de acordo* sem ler** e **a correção que não pegou**. A visão é outra — produto e design lidos juntos, o que nenhuma etapa vê sozinha — e o modelo é o mesmo; usar outro modelo para ela foi considerado e recusado. O custo é visível: mais chamadas

> de modelo em Produto e Design, e uma economia no Dev quando a entrada bloqueia, porque nada é construído sobre o que vai mudar.

O veredito de cada posição continua sendo **a pior das dimensões**, contado em código com os bloqueantes lidos dos próprios arquivos de critério. E, como todo gate de modelo desde 8 de outubro, a banca só bloqueia com citação que existe no documento que ela julgou (Seção 13).

12 Os portões, e o que fazer quando reprovam

Há dois tipos, e a diferença entre eles é a espinha dorsal do projeto.

Determinístico — é função Python

Não gasta *token*, não erra, não muda de opinião entre duas execuções e não pode ser convencido. Compara literalmente contra um documento: a lista oficial da BNCC, o vocabulário fechado do manifesto, o contrato real da API de telemetria.

Toda regra que *pode* ser mecânica foi tirada dos avaliadores e trazida para cá.

Avaliador — é agente com rubrica

Julga o que não se pode conferir mecanicamente: se há segredo, se a escolha importa, se a linguagem cabe na idade. Avaliadores baseados em modelos de linguagem têm vieses documentados de posição, verbosidade e leniência [45].

Por isso a rubrica sempre traz teste prático e contraexemplo [25] — é o que sustenta concordância entre avaliações.

Portão	Tipo	Reprova quando
Micro-gate (02)	rubrica	O briefing ou o núcleo entrega a virada; a escolha não muda nada; a missão repete uma já gerada.
Cobertura conceitual (01c)	**código**	O conceito limiar não vive no Decidir nem no “ahá”; conceito do material sem destino; anatomia do “ahá” incompleta; trecho citado que não existe no material.
Profundidade (02b e 03b)	rubrica + **código**	Nota abaixo de 3 em conceito vivido, aposta comprometida ou ápice do “ahá”. O juiz dá as notas; **quem conta o veredito é código**, não a prosa do juiz.
Conta da virada (03aa)	**código**	O ranking que o núcleo declara não fecha: refeita a conta com os pesos da tabela, o vencedor não é o declarado, ou é o mesmo antes e depois — e então não há virada.
Fidelidade ao pedido do PO (03a e 16c)	rubrica + **código**	Pedido de *Refazer* violado ou sem prova; frase entre aspas das TRAVAS alterada. O juiz cita; **o código confere a citação**. Conflito com regra da esteira vira ●.
One-pager do PO preservado (04d e 10d)	**código**	Linha ou célula do *one-pager* que o PO anexou que não está no gerado — igual, ou com 85% dos trigramas e números e nomes intactos —, sem pedido que mande tirá-la. Na conferência final, ● recusa o *de acordo*.

Portão	Tipo	Reprova quando
Contrato de plataforma (04b)	**código**	Canal de sala por WebSocket, estado no navegador, SAEB em ano não avaliado, número sem origem declarada; seção do DS que o briefing fixou trocada; habilidade do briefing cortada sem pedido do PO; decisão atribuída ao PO sem registro dele.
Gate estrutural (05)	rubrica	Falta mecanismo, há caminho sem tratamento, o ambiente está fora da lista.
Gate final (11)	rubrica	Os três pareceres avaliaram versões diferentes; correção que ficou só no relatório. Não declara mais que o pedido do PO foi atendido: isso é da fidelidade.
Currículo BNCC (14)	**código**	Código inexistente, de outro ano, ou sem letramento digital.
Manifesto (15 e 33)	**código**	Campo do bloco 0 ausente ou fora do vocabulário fechado; **seção antiga do DS em missão nova escolhida pela esteira**, com a sucessora apontada — a escolhida pelo PO é ressalva.
Matriz de evidência (15b)	**código**	Habilidade sem momento em tela, sem medida ligada, ou com grau "pontuado" sem dois sinais nomeados e gabarito no servidor.
Perguntas de produto (15c–15e)	**código** + rubrica	Pergunta sem decisão, sem corte ou sem ação declarada na tabela de eventos; e o que regex não vê — se a pergunta muda mesmo uma decisão nossa, e se é sobre o produto e não sobre a criança.
Rastro da *spec* (15f)	**código**	Lacuna de produto ou de insumo marcada na *spec*: volta para quem a escreveu. Decisão do PO fica ● no parecer de Produto.
Spec × conteúdo aprovado (15g e 20b)	**código**	A *spec* com outras habilidades, outro tempo em tela, sem o podcast, ou decidindo como fato o que o *one-pager* deixou "a definir por produto"; glossário que proíbe um termo que o *one-pager* aprovado usa. Em Design, a *spec* consolidada que piorou desde a de Produto.
Banca nas quatro posições (15h, 29f, 33b, 44)	rubrica + **código**	Critério bloqueante de qualquer dimensão daquela posição, com citação que exista no documento julgado. O veredito é a pior das dimensões, contado em código.
Resumido × aprovado (16e)	**código**	Resumido com habilidade ou tempo diferentes do canônico. Canal B parafraseado fica ●.
Telas no Stitch (29)	**código**	Menos telas no projeto do que no inventário da jornada; tela do inventário com identificador fora do padrão, que a geração deixava de fora em silêncio. Vira *ticket* para o agente de telas.
Lint do protótipo (29a)	**código**	O que o QA não enxerga por ler só o texto: o arquivo HTML em si.

Portão	Tipo	Reprova quando
Coerência entre documentos (29d)	**código**	Jornada, fluxo, *copy* e protótipo usando um nome que o glossário canônico proíbe. Papel ou idade de personagem, número, rótulo de botão e nome de arquivo divergentes ficam ●. A tabela do glossário republicada não conta como uso, nem a variante curta fora de rótulo.
Cobertura de telas (29e)	**código**	Tela do inventário que falta, **por identificador**, no fluxo, na *copy* revisada, na especificação de telas ou nos quadros do protótipo; jornada vazia.
QA de design (30)	rubrica	Construção infiel à *spec*; diagrama ausente ou inválido; visão do professor faltando.
Afirmações do QA (30b)	**código**	"Todas as N telas" ou "N de M" que não batem com o inventário, o Stitch ou o protótipo; caminho de arquivo que a esteira não grava. Reescreve o veredito do QA como bloqueado.
Rastro do design (33)	**código**	Lacuna de Design deixada aberta, enquanto o designer ainda pode refazer. O que o fechamento de lacunas fechou não conta.
Matriz de rastreabilidade (33b e 50)	**código**	Requisito do produto — habilidade, texto literal do PO, tempo, tela, estado, evento, seção do DS — que não chegou a um documento que tinha de carregá-lo.
Fidelidade ao *Refazer* de Design e de Dev (33c e 51)	rubrica + **código**	O pedido feito àquela etapa sem prova nos documentos dela. Sem pedido, não roda. ● recusa o *de acordo*.
Banco de conteúdo (35c)	**código**	Documento, tabela ou roteiro do *brief* sem texto escrito no banco; *status* do *brief* fora de "a produzir", "rascunho" ou "escrito" — homologar é ato humano; chat em que o estudante escreve sem a árvore de intenções. Fala da IA faltando num estado fica ●.
Telemetria (36)	**código**	Evento fora do contrato real da API; medida sem tabela de leitura; WebSocket; e o que o *handoff* afirma da plataforma sem existir nela.
Rastro de lacunas (38)	**código**	Lacuna de Dev — ou de etapa anterior escrita pelo próprio *handoff* ou *brief* — aberta antes do PRD. O resto já foi cobrado na etapa dona.
Coerência dos KPIs (48a)	**código**	KPI sem pergunta, sem decisão, sem corte, ou cujo evento não dispara na tela que ele nomeia.

Portão	Tipo	Reprova quando
Pendências e consistência do PRD (48)	**código**	Marcadores deixados por falta de informação; o documento se contradizendo — dois caminhos para o mesmo módulo, prefixo de tabela fora do padrão, requisito P0 sem teste nem tarefa, evento sem tabela leitora, tarefa que cobre tudo, identificador inválido; **o que ele afirma da plataforma sem existir nela** (ADR que não existe, versão do DS cravada, seção antiga, agendamento); a mesma coluna com dois tipos, no PRD ou contra a *spec*; tarefa que consome conteúdo que o banco não tem; tela, chave de *copy*, *asset* ou estado do design que não chegou; tabela ou evento do *handoff* que o PRD perdeu; e o PRD que ainda não dá para codar (Seção 14.2). Reprovado, devolve ao redator (48b).
PRD × conteúdo aprovado (48c)	**código**	Habilidade do bloco 11 fora da matriz da seção 14; habilidade sem KPI; outro tempo em tela; podcast sem requisito funcional; caminho de arquivo que a sessão não tem; texto aprovado citado com outra redação. Tempo por etapa divergente fica ●.
Coerência com o aprovado (48e e 49e)	rubrica + **código**	O PRD, ou o pedagógico, afirmando o contrário do *one-pager* aprovado — mecânica, número, regra, texto. O juiz cita os dois lados; o código confere as duas citações. Juiz que não responde é ●, não ●.
Pedagógico × aprovado (49b)	**código**	Habilidades, tempo em tela, podcast e caminhos de arquivo do pedagógico contra o aprovado.

13 O que impede o sistema de alucinar

Decompor em 114 passos resolve densidade de instrução e cria um problema novo: contrato de interface [12]. Sete regras de engenharia de contexto sustentam as fronteiras.

1. **Isolamento total de contexto.** Todos os agentes, com a única exceção do 01, rodam com `include_contents="none"`: não herdam o histórico da conversa. Recebem exclusivamente o que foi interpolado no próprio *prompt*, a partir do estado da sessão. Sem isso, a poluição de contexto se acumula e cada agente passa a "lembrar" de versões antigas dos documentos.
2. **Interpolação com degradação declarada.** A variável `{chave}` falha imediatamente se a chave não existir — é o comportamento correto quando a dependência é obrigatória. A variante `{chave?}` degrada com elegância, e é o que permite rodar a etapa de Dev isolada, sobre um estado que não tem Design.
3. **Contrato de envelope JSON.** Todo relatório de portão encerra com o mesmo envelope, e é ele — não a prosa do relatório — que o orquestrador lê para decidir o que fazer:

```
{
  "status": "aprovado | apto_com_ressalvas | bloqueado",
```

```
  "veredito": "LIBERADA PARA DESIGN/DEV",
  "criterios_reprovados": [],
  "bloqueios": []
}
```

4. **Decisões de produto protegidas.** Decisão estratégica que depende de premissa de negócio ausente no briefing é marcada como `[DECISÃO PENDENTE — responsável de produto]`. Os agentes são **proibidos de inventá-la**, e o diagnóstico a endereça a uma pessoa. É o oposto do comportamento padrão de um modelo de linguagem, que é preencher a lacuna com o plausível.
5. **O que um modelo afirma é afirmação, não fato.** Quando um agente escreve algo que tem resposta exata — "todas as 16 telas", "o projeto A vence pelo custo", "a frase está no documento", "arquivo homologado", "ADR-108" —, o código confere contra a fonte: o inventário, a conta refeita, o texto do documento, o vocabulário fechado, o contrato da plataforma. Juiz que diz "atendido" sem uma citação que exista vale como não comprovado. Na dúvida, a conferência falha fechada.
6. **Gate de modelo só bloqueia com citação que existe.** Até 8 de outubro, o micro-gate, o gate estrutural, os três pareceres, o gate final, o juiz das perguntas, o QA de design e a banca decidiam portas reais — o 🔴 deles devolve documento ou recusa *de acordo* — sem que nada conferisse em que trecho estava o problema. Hoje cada achado bloqueante traz o trecho literal do documento avaliado, e o código o procura: **citação que não existe rebaixa o achado a atenção**, e se todos os bloqueios caem o veredito vira 🟡, com o envelope corrigido no relatório. O bloqueio era do juiz, não do documento. Achado bloqueante *sem* citação continua bloqueando, por cautela.
7. **O que o PO aprovou viaja literal.** As seções do *one-pager* aprovado chegam a cada agente de Design e de Dev como o **conteúdo aprovado**, copiadas pela esteira, não resumidas por um agente; e as que o gabarito do PRD e do pedagógico não tinha onde pôr — currículo, papel do professor, tempos e cenários de aula, podcast, contrato de evidência, hipóteses — são **coladas literais no fim** dos dois documentos. O redator remete a elas e não as reescreve (Seção 14).

13.1 E os próprios verificadores, quem confere?

Um verificador que nunca reprova parece igual a um que confere tudo. Por isso a matriz de rastreabilidade tem um **teste de mutação**: o teste pega uma missão real que passa, **planta uma perda** — um texto do PO apagado, uma palavra trocada no fim de um texto aprovado, um "não" inserido, um evento ou um estado renomeado, uma fala do podcast alterada — e exige que a conferência a acuse. As perdas plantadas são as que as auditorias acharam nas sessões reais. Foi a mutação que mostrou que "85% do texto igual" deixava passar a negação no fim de uma frase aprovada: hoje o literal é conferido também no começo, no fim e nas negações do trecho que casou.

O mesmo vale para o gate que **aprova vazio**. Jornada vazia, *copy* revisada vazia com texto aprovado a conferir, PRD conferido sem jornada, *copy*, *brief* ou *spec* — todos davam 🟢, porque não havia nada a reprovar. Hoje reprovam, ou saem 🟡 "não conferido". E "NÃO APROVADO" escrito sem envelope, ou um envelope com *status* aprovado e veredito vermelho, reprovam em vez de serem lidos como aprovação.

14 Do *one-pager* aprovado ao PRD que dá para codar

O PO aprova o *one-pager* e a *spec*. Depois disso, Design e Dev reescrevem, resumem e traduzem aquele conteúdo uma dezena de vezes até o PRD. Cada passagem tinha a sua conferência — ou não tinha —, e ninguém respondia à pergunta que importa: **o que o produto especificou chegou inteiro ao fim?**

As auditorias de 6 e 7 de outubro sobre as sessões do *bucket* responderam que não, e mostraram onde. A *spec* era um segundo autor: aplicava as ressalvas que o PO tinha recusado, e a missão do 6.º ano perdeu uma habilidade e foi de 29 para 24 minutos em tela; o Dev a devolveu por "inconsistência entre PRD e documentação de produto". Dos 153 textos de tela do *one-pager* — a pergunta da aposta, as alternativas, as falas da personagem —, a *spec* levava 81, e o protótipo, de 18% a 45%. O resumido não trazia o Canal B aprovado em 7 de 15 sessões. Um PRD citava 0 das 90 chaves da *copy* revisada e 0 dos 17 estados da *spec*. Cada perda num documento diferente, nenhuma num lugar que o PO lê.

14.1 Três peças, todas em código

O conteúdo aprovado, entregue e colado. As seções do *one-pager* aprovado chegam inteiras, seção por seção, a cada agente de Design e de Dev; e as que os gabaritos não tinham onde pôr são coladas literais no fim do PRD e do pedagógico. O §12a, o contrato de evidência, muda de forma a cada missão — tabela de quatro colunas, de cinco, subseções por habilidade —, e um *parser* dele erraria em silêncio. Inteiro, ele não tem como perder nada.

A conferência contra o aprovado. Só o que tem estrutura firme: os códigos da BNCC (do §12a do *one-pager* ao bloco 11 da *spec*, à seção 14 do PRD e ao pedagógico), o tempo total em tela (§10), o podcast (§11), um KPI por habilidade, os caminhos de arquivo que o documento cita e os textos literais. Ela roda na *spec*, na *spec* consolidada pelo Design, no resumido, no PRD e no pedagógico — e, onde o que se compara é sentido e não texto, um juiz de contradições cita os dois lados e o código confere as duas citações.

A matriz de rastreabilidade. Cada requisito do produto — habilidades, textos literais do PO, tempo em tela, seção do DS fixada, estados, eventos, decisões a revisar, telas do inventário — procurado, por fato, em cada documento a jusante que tem de carregá-lo. Ela sai no fim do Design e de novo no fim do Dev, em `matriz-de-rastreabilidade.md`, e diz onde cada requisito chegou e onde se perdeu, com o dono do conserto. Documento que ainda não existe fica "—", não ●. A fronteira é a mesma do verificador da BNCC: confere-se presença verificável; se a tela *exercita* a habilidade é julgamento, e fica com os pareceres.

14.2 O PRD que dá para codar

Lido como quem vai codar, o PRD da missão do 9.º ano — o último aprovado antes de 8 de outubro — deixava em aberto exatamente o que o gabarito já pedia e nada conferia: `payload: Record<string, unknown>`, *schemas* citados sem definição, `DELETE` para zerar a turma, só um diagrama no lugar do modelo de dados, o nível da criança gravado numa coluna `score_` da tabela da missão, a régua de evidência sem predicado, o chat sem árvore de intenções e a IA do preparo sem *prompt* nem *schema* de saída. Hoje o verificador do PRD confere, por cabeçalho de tabela e bloco de código — não pelo número da subseção, que o redator renumera:

Onde	O que reprova
Rotas (§12)	Rota que não está no molde da plataforma, uma por linha `API-xxx`; zerar com `DELETE` em vez de `POST /zerar`; corpo e resposta que não são `z.object`; tipo frouxo (`Record<string, unknown>`, `any`, `unknown`); falta da tabela de erros, código → *status*. Idempotência e limite de taxa ficam como ressalva.
IA no preparo (§12)	Chamada sem a tabela dela, sem o texto de cada *prompt* num bloco de código, sem o *schema* de saída em Zod.
Modelo de dados (§11)	Sem DDL num bloco `sql` ou *model* Prisma; coluna `score_`, `nivel_` ou `nota_` na tabela da missão — o nível mora na evidência da plataforma. `FORCE RLS` e permissão do papel da aplicação ficam como ressalva.
Ficha de evidência (§14.2)	Falta uma linha por habilidade do bloco 11, com o tipo (ação ou múltipla escolha) e, para a de ação, a regra lendo uma coluna declarada na §11 — ou a proposta ao PO.
Chat do estudante	Chat em que o estudante escreve sem a árvore de intenções no banco de conteúdo — gatilhos fechados e a linha de resposta para o que não casa — e sem a remissão a ela no PRD. Esta é do banco, não do redator.
Design → PRD	Tela do inventário sem *story*; chave da *copy* revisada fora da *story* da tela, ou sem dizer onde vive; *asset* V1 do *brief* fora do PRD; estado da *spec* sem mapa; requisito P0 sem critério EARS ou Gherkin; *schema* citado e não definido; *status* de gate escrito pelo redator — quem publica *status* é a esteira.
***Handoff* → PRD**	Tabela ou evento declarado no *handoff* que o PRD perdeu, salvo registro na §19. Rota trocada é ressalva — foi assim que apareceu o `POST /zerar` do *handoff* virando `DELETE` no PRD.

Os *prompts* anteriores ao PRD — *handoff*, formato da *spec*, modelo de insumo, perguntas de produto, o critério técnico da banca — ensinavam o modelo antigo de tabelas e rotas, e o redator aprendia com eles. Passaram a ensinar o molde de uma missão que já roda na plataforma. Sobre o PRD de 30 de setembro, o verificador acusa nove bloqueios — os pontos que a auditoria tinha achado lendo; um PRD gerado com o código de 8 de outubro passou em todos (Parte VII).

15 As integrações, e por que nenhuma é caminho crítico

Dois servidores MCP participam da etapa de Design, com papéis distintos. Nenhum dos dois é obrigatório: fora do ar, cada um registra a própria indisponibilidade no artefato e a esteira entrega a missão assim mesmo — porque o que vale é sempre o texto versionado.

Stitch — as telas

O modelo **só especifica**: uma linha por tela e estado, com o *prompt*. **Quem gera é o código**, todas, em paralelo, registrando o identificador que o Stitch devolveu. A tela refeita nasce no mesmo projeto como "… · v2"; a folha de links e o PRD listam só a versão atual. Sem chave de acesso, a esteira segue; a especificação em Markdown continua sendo a fonte.

Miro — o espelho dos diagramas

Aqui o modelo **não usa ferramenta nenhuma**. Um passo determinístico publica os mesmos blocos Mermaid como board navegável, fora do turno do agente. O board é **espelho, nunca fonte**: editá-lo não altera a missão, e a ausência dele não é achado de QA.

A razão de o Miro ser publicado por código, e não pelo agente, é concreta e custou uma execução: deixar um modelo construir o board item a item foi o que estourou o teto de contexto quando os diagramas viviam no FigJam. O Stitch seguiu o mesmo caminho em 7 de outubro, por razão parecida. Quando era o agente de telas quem chamava o Stitch, ele decidia quantas telas gerar — e o *prompt* mandava "priorizar": o projeto saía com 27% a 68% do inventário, as telas do professor caíam em quase todas as missões, e na do 9.º ano o agente declarou dez identificadores de tela que o projeto não tinha. Mesmo sem a ordem de priorizar havia um teto: cerca de 25 respostas de ferramenta por agente não cabem em 25 a 46 estados de tela. O Figma, aliás, saiu da esteira pelo mesmo tipo de razão — não há cliente oficial que o sirva neste ambiente, e *board* que ninguém consegue gerar não é entregável.

16 A base da plataforma: lida do *bucket*, não copiada

Para escrever a *spec* e o PRD, os agentes precisam saber como a plataforma Unooria é **hoje**: que seções o *design system* tem, que campos o manifesto aceita, que portas estão ocupadas, qual é a última decisão de arquitetura registrada, como funcionam atividades, equipes e evidência. Isso não é escrito à mão em resumo nenhum. Vem da **base da plataforma**: as regras e os contratos do repositório da plataforma, o inventário de componentes do DS e um **contrato da plataforma** extraído do código.

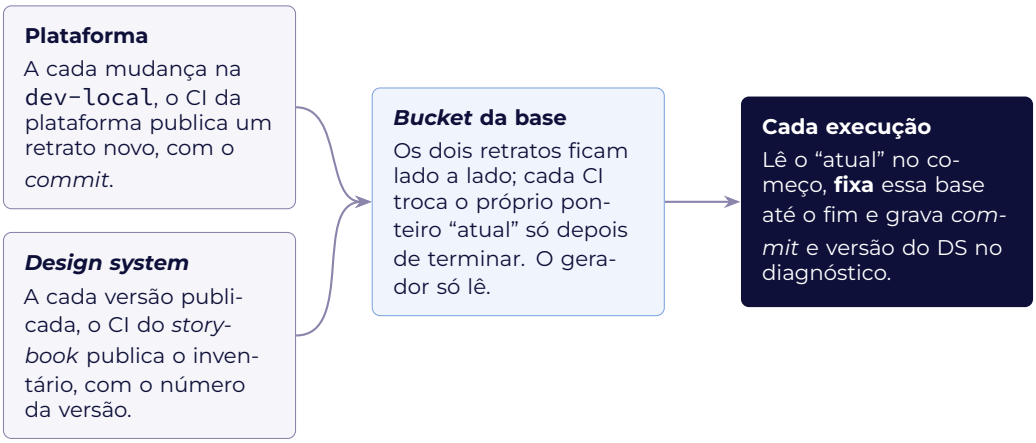


Figura 10: Quem conhece a plataforma a publica; a esteira só lê. Uma mudança na plataforma no meio de uma execução não muda nada nela.

Por que não uma cópia no repositório

Até 6 de outubro, a base era uma cópia guardada no próprio repositório do gerador, atualizada à mão por um *script*. Ela parou em 16 de setembro e ficou **320 *commits*** atrás da plataforma sem ninguém ver. A missão do 9.º ano saiu da esteira pedindo a seção `chatgpt` (que já tinha sucessora), citando uma "ADR-108 proposta" (a última era a 148) e cravando o DS 0.2.2 (era o 0.2.18) — e sem atividades, equipes nem evidência, que a plataforma já tinha. Nada no dossiê dizia de quando era o retrato.

Cópia que depende de alguém lembrar de atualizar envelhece em silêncio. Agora quem muda a plataforma é quem publica o retrato, e **sem base a execução não começa**: cair numa cópia velha sem avisar foi exatamente o que aconteceu.

E por que um *bucket* só dela, que a esteira não consegue apagar

Na primeira versão, a base morava numa pasta do mesmo *bucket* das sessões, que o aplicativo monta com escrita. A barra lateral lista toda pasta como uma missão — e listou a base como

uma missão sem título, com o botão de excluir ao lado. Em 6 de outubro ela foi **apagada duas vezes**: os ponteiros "atual" e os retratos da plataforma e do DS. Os CIs tinham publicado certo; a base foi restaurada da exclusão reversível do *bucket*, antes que uma execução caísse em "base indisponível".
A correção foi em duas camadas. Primeiro, a pasta saiu da lista, e o botão de excluir passou a recusar nome vazio, caminho com barra e qualquer alvo que não seja uma sessão. Depois, a base saiu **do alcance do aplicativo**: mora em `gs://entre-redes-dev-base-plataforma`, montado **só para leitura** em `/app/base-plataforma`, e a conta do gerador só tem permissão de leitura nele. **Proteger com regra na tela é bom; proteger com permissão é melhor.**

Duas consequências práticas:

- **O DS não tem versão cravada.** A plataforma usa sempre a última versão publicada do *design system*; o PRD não escreve número de versão, e o verificador do PRD acusa quando escreve.
- **O que o PRD afirma sobre a plataforma é conferido contra o contrato.** decisão de arquitetura (ADR) com número que não existe, seção antiga em missão nova, tabela própria onde a plataforma já tem uma genérica, "zerar" a turma arquivando o que ela fez quando a plataforma abre uma rodada nova, **agendamento** — que a plataforma não tem: a atividade é posta no ar, não agendada. O mesmo verificador lê o *handoff*. Linha marcada como pendência de plataforma é pedido, não afirmação, e não é cobrada.

O PostHog continua citado nos documentos como o leitor dos indicadores de produto: ele vai entrar na plataforma, e a missão já nasce dizendo o que ele vai ler.

PARTE IV

Por que funciona

Uma rubrica que não diz de onde tira os números acaba defendida por antiguidade.



unoria®

17 Descoberta ativa, e não instrucionismo

A esteira foi construída sobre a aprendizagem pela ação de Dewey [16], a aprendizagem pela descoberta de Bruner [8] e o construcionismo de Papert [35]. Na prática, isso se traduz numa inversão de ordem que toda missão precisa respeitar — e que o micro-gate (agente 02) verifica antes de qualquer outra coisa.

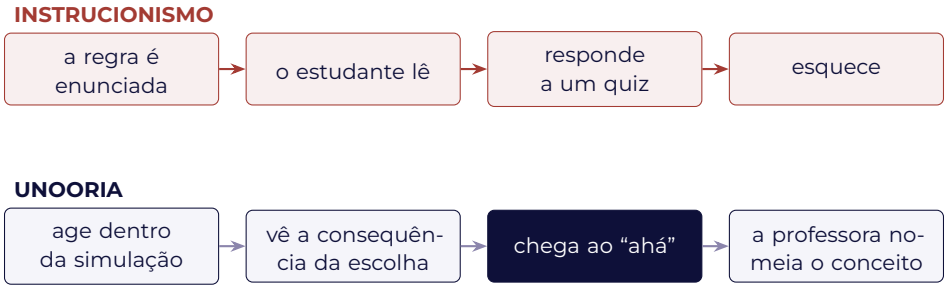


Figura 11: A mesma aula, nas duas ordens. Na segunda, o conceito é nomeado *depois* da experiência — e por uma pessoa, não por uma tela.

O detalhe que se perde com facilidade é o último quadro. A missão **não fecha o conceito sozinha**: ela produz a experiência e entrega à professora os dados de comparação para que a formalização aconteça na roda de conversa. Missão que explica tudo na tela final devolve o estudante ao instrucionismo, com uma camada de jogo por cima.

18 A profundidade da missão: a anatomia do “ahá”

Em 24 de setembro de 2026 o time pedagógico devolveu as missões geradas com uma crítica que nenhum gate tinha detectado: elas eram **rasas**. Não trabalhavam os conceitos do material do bimestre, e o “ahá” não tinha ápice. Pior: o responsável de produto vinha liberando essas missões *confiando no semáforo* — que estava verde, porque media outra coisa.

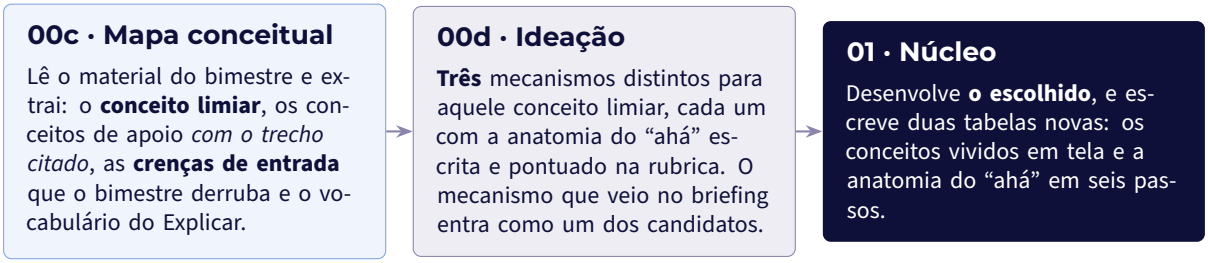
A causa, conferida no código

A esteira tinha 73 critérios sobre estrutura, plataforma, currículo e carga — e **um** sobre o “ahá”: “dá para resumir numa frase?”. Qualquer missão rasa passa nesse teste.
E o núcleo narrativo *recebia* o material do bimestre, mas nenhuma regra pedia que ele o *usasse*. Duas omissões que ninguém tinha notado porque o semáforo verde respondia a uma pergunta diferente da que o time pedagógico fazia.

A resposta foi uma camada inteira, que hoje é a parte mais nova do sistema: três passos antes do núcleo, duas tabelas dentro dele e um juiz próprio no ciclo.

18.1 Ler o material antes de inventar

A ordem importa, e antes ela estava errada: o núcleo inventava o mecanismo e depois procurava no material algo que o justificasse. Hoje há dois passos antes dele.



Uma verificação determinística (01c) confere o núcleo contra o mapa: o conceito limiar vive no Decidir ou no “ahá”; todo conceito do material tem destino; a anatomia tem os seis passos; e **o trecho citado pelo mapa existe mesmo no material** — porque citação inventada é o modo de falha mais fácil de um modelo que precisa fundamentar.

18.2 Os seis passos do “ahá”

Um “ahá” genuíno não é informação nova. É uma crença que o estudante carregava, pôs à prova com uma escolha dele, e viu cair pela consequência dessa escolha.

#	Passo	O teste	A falha vista em execução
1	**Crença de entrada**	Está nomeada, e alguém daquela idade de fato acredita nisso?	Nenhuma crença declarada: a missão supõe que o estudante já sabe, e o “ahá” só confirma.
2	**Aposta comprometida**	A escolha é dele, é sobre o conceito limiar, e **trava** antes da revelação. *Teste de remoção:* tirando o Decidir, o “ahá” muda? Se não muda, é informação, não descoberta.	“A Foto do Mural”: nenhuma aposta, só observar. “O Rastro do Boato”: a escolha era sobre confiabilidade da fonte, e a descoberta, sobre distribuição.
3	**Consequência sentida**	Dois estudantes que escolheram diferente veem coisas diferentes? A consequência é mostrada pelo ambiente, e não dita pelo professor ou por um painel?	Panorama igual para qualquer escolha; consequência que só existia na fala oral do professor; painel com “94% e 6%”.
4	**Virada**	*Teste da crença:* leia o passo 1 e depois a virada. Ela **desmente** a crença, ou só acrescenta informação? A reação esperada é “caramba, eu achava que…”, não “ah, faz sentido”.	Resultado que confirma o que o Desafiar já tinha dito — *spoiler* estrutural que passou pelo micro-gate.
5	**Nomeação**	O termo do material aparece? **Só depois** da virada? Usa o que o próprio estudante fez? Compara ao menos duas escolhas da turma?	Card de texto genérico; conceito explicado antes da vivência; termo técnico sem âncora no que aconteceu.
6	**Transferência**	Um estudante que não fez a missão responde só pela pergunta? A resposta exige o mecanismo (“isso acontece porque…”)?	Refletir binário com resposta óbvia; distrator implausível; “pode ter *fake news*” aceito como resposta.

Duas regras de contorno, que evitam que a rubrica produza o dano oposto:

- **Discriminação, não cinismo.** Em missão sobre desinformação, a lição não pode virar "desconfie de tudo": há sempre ao menos um item confiável que o estudante deve *manter*, e a medida separa o acerto no confiável do acerto no manipulado. Jogos de letramento midiático às vezes só aumentam o ceticismo geral, e isso não é aprendizado.
- **Anos iniciais são diferentes.** Do 2.º ao 5.º ano, "viver primeiro e explicar depois" não tem a mesma evidência que nos anos finais, e aprender com a própria previsão errada depende de controle inibitório ainda em formação. Por isso a discrepância "você escolheu X, aconteceu Y" aparece **explícita na tela**, e um andaime breve antes do Decidir é permitido — desde que não entregue a virada.

18.3 O juiz, e por que ele não decide sozinho

Um agente próprio dá nota de 1 a 4 em seis dimensões — conceito do material vivido, aposta comprometida, ápice do "ahá", explicar depois de viver, transferência e demanda à altura do ano — sempre com **trecho citado** e instrução de reescrita. Nota abaixo de 3 nas três primeiras reprova.

> **Quem conta o veredito é código, não o juiz.** Um passo determinístico relê as notas que o juiz deu e recalcula o resultado, ignorando o que ele escreveu na conclusão. É a mesma desconfiança que sustenta o resto da esteira: avaliadores baseados em modelos tendem à leniência [45], e um juiz que dá nota 2 e conclui "aprovado com ressalvas" é um caso observado, não hipotético.

18.4 O que esta rubrica não faz

- **Não cria número novo.** Os tetos de opções, duração, gestos e abstração continuam vindo da fonte única (Seção 20).
- **Profundidade não é carga.** Nota 4 nunca exige mais telas, mais opções ou mais minutos — a tela do pico continua sendo a mais enxuta da missão. Confundir as duas coisas produziria missões densas e cansativas em nome da profundidade.
- **Não decide pelo time.** Quando o mecanismo vem do responsável de produto, o juiz pontua e aponta; **trocar de mecanismo é decisão humana**.

> **O que ainda falta: a calibragem**
>
> Os cortes — reprovar abaixo de 3 nas três primeiras dimensões — são CONVENÇÃO, e estão declarados assim. Falta calibrá-los contra três a cinco pares "versão gerada → versão revisada pelo time pedagógico", que é o material que diria onde a régua deve de fato ficar.
> Enquanto isso não acontece, `PROFUNDIDADE_ESTRITA=false` rebaixa o bloqueio a ressalva sem mexer nas notas nem nas rodadas. É o interruptor certo para usar durante a calibragem — e o errado para deixar ligado depois dela.

19 Carga cognitiva, e o que a esteira faz com ela

A teoria da carga cognitiva [40, 41] separa três demandas que competem pela mesma memória de trabalho:

Intrínseca

A complexidade real do que se quer ensinar — entender que uma recomendação é inferida do próprio comportamento.

Não se reduz sem reduzir o aprendizado.

Estranha

O esforço desperdiçado com interface confusa, texto longo, botão escondido, instrução ambígua.

É o alvo de metade dos critérios de design da esteira.

Germânica

O esforço que constrói esquema duradouro: comparar, explicar, aplicar a um caso novo.

É o que a etapa Refletir existe para provocar.

Disso sai uma régua concreta que os agentes de design aplicam: a memória de trabalho lida com **dois a três elementos novos e interagentes** por vez [41] — e elemento já automatizado, como o ícone de coração ou o botão de play, *não conta*. Derivando por faixa: cerca de 1 a 2 gestos novos por ato nos anos iniciais, 2 a 3 no 3.º–5.º, 3 a 4 nos anos finais e no Médio. A derivação é explicitamente marcada como extrapolação — ninguém mediu "elementos de interface".

20 A régua por idade

Os tetos que a esteira aplica vivem num arquivo só: `prompts/references/literatura-cognitiva.md`. **Nenhum agente carrega régua própria**; quando um critério precisa de um número, ele vem de lá.

Por que a fonte tem de ser única

Porque números soltos em *prompts* divergem sem que ninguém perceba. Foi o que aconteceu quando um avaliador novo chegou com "3 a 4 elementos" e a rubrica de outro agente tinha uma tabela por faixa dizendo outra coisa. Os dois estavam em produção, os dois julgavam a mesma missão, e nenhum dos dois estava errado segundo o próprio *prompt*.

Cada número no arquivo carrega uma marca: **[DADO DIRETO]** quando a fonte mediu exatamente aquilo, naquela faixa; **[EXTRAPOLAÇÃO]** quando a rubrica deriva um limite prático de um princípio geral; **[CONVENÇÃO]** quando a equipe escolheu por consistência de produto. Extrapolação justifica ressalva com mais frequência que bloqueio.

20.1 Quantas opções o estudante compara de uma vez

Faixa	Teto	De onde vem
1.º–2.º EF (6–7 anos)	**2**	*Span* complexo medido: a criança sustenta entre 1,6 e 2,9 itens *enquanto processa* [19, 13].
3.º–5.º EF (8–10 anos)	**3**	*Span* complexo de 1,9 a 3,7 itens.
6.º–9.º EF (11–14 anos)	**3–4**	Memória visual de trabalho $k = 2{,}4$ aos 11–13 anos.
Ensino Médio (15–17 anos)	**4**	$k = 2{,}72$ aos 16 anos, ainda **abaixo** do adulto jovem (3,16) [23].

Opção visível não é opção lembrada. Se todas as alternativas ficam na tela enquanto o estudante decide, o gargalo deixa de ser memória e passa a ser comparação atencional — e o teto sobe em **+1**. Se ele precisa lembrar de opções que já saíram da tela, vale o teto da tabela. Essa distinção era a maior imprecisão da versão anterior da régua.

20.2 Quanto tempo dura cada ato

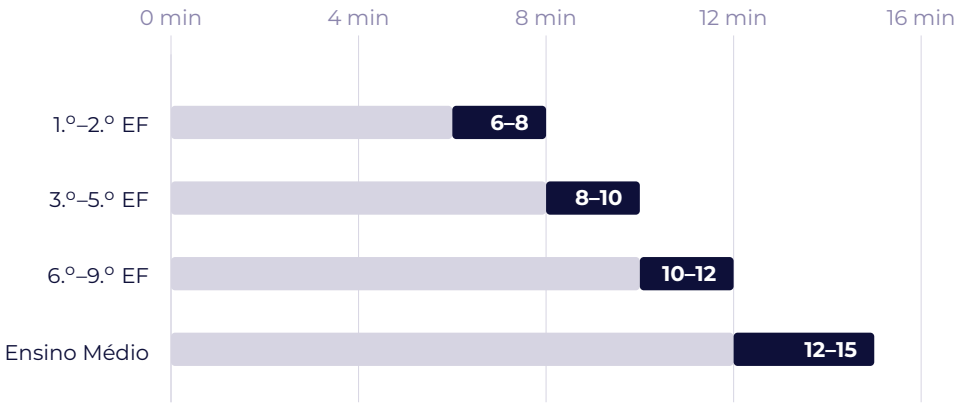


Figura 12: Teto de duração por ato. A escala é deliberadamente achatada: em 52 salas observadas, a atenção cai monotonicamente com a duração e o efeito de série foi equívoco — **a duração importa mais que a idade** [20].

O teto que mais importa, porém, é o **agregado**: no máximo **25 minutos de tempo-aluno** numa aula de 50. Ele é novo, e existe por aritmética: três atos no teto antigo somavam 45 minutos e não sobrava aula. Some-se que apenas 64% do tempo de aula é atividade acadêmica no Brasil [9] — descontando instrução inicial e distribuição de dispositivo, a janela real fica entre 22 e 27 minutos.

E há um fator que pesa mais que o relógio: **o formato**. Atividade individual ou em pequeno grupo sustenta atenção bem melhor que turma inteira (razão de chances até 1,62), e o critério de duração passou a considerar isso.

20.3 O que se pode pedir que o estudante entenda

Missão de letramento digital quase sempre pede raciocínio sobre um sistema invisível — um algoritmo, uma inferência, um viés. Não há uma idade única; há três portas.



Figura 13: As três portas do raciocínio sobre sistemas invisíveis [39, 37]. É o núcleo do letramento digital, e até a revisão de setembro não tinha régua nenhuma.

20.4 O piso da escola pública brasileira

Nenhuma missão é avaliada contra o laboratório ideal. O cenário de piso é declarado, com fonte — e mudou na última revisão, porque a suposição da equipe estava errada em duas direções ao mesmo tempo.

Parâmetro	Piso realista	Observação
Alunos por turma, anos iniciais	**25**	A média oficial é 22,3 — afirmar "30 a 40" **infla** a dificuldade.
Alunos por turma, finais e Médio	**32**	26,6 no Censo; 32,5 na turma de referência do TALIS.
Dispositivo	**nenhum fornecido pela escola**	Só 47% das escolas disponibilizam computador ao aluno. O piso não é "compartilhar", é **não ter**.
Internet	**sinal ausente em sala**	96% das escolas têm internet, mas 39% das estaduais relatam que o sinal não chega à sala.
Leitura, 5.º ano	**39% abaixo do adequado**	Ou seja, **60,7% estão adequados**: tratar os anos iniciais como majoritariamente defasados contraria o Saeb.
Leitura, 9.º ano e Médio	**62% e 65% abaixo**	Aqui a defasagem se confirma.
Professor	**sem formação recente em tecnologia**	46% não tiveram; apenas cerca de um terço decide sobre o uso de recurso digital.

O teste de piso, em uma frase

Uma missão precisa funcionar numa turma de 25 a 32 estudantes, com dispositivo que a escola não forneceu, com sinal que pode não chegar à sala, conduzida por uma professora que não teve formação recente em tecnologia — e ainda assim caber em 25 minutos de tempo-aluno. Tudo que a esteira exige de tela limpa, texto curto e ausência de download pesado deriva daqui, e não de gosto estético.

21 O registro negativo: o que não sustenta a régua

Esta é a seção mais incomum da fonte única, e a mais importante. Sem ela, uma fundamentação refutada tende a reaparecer na revisão seguinte, porque **o argumento de autoridade sobrevive ao artigo que o derrubou**.

Não use	Por quê
Choice overload como base do teto de opções	Meta-análise de 63 condições ($N = 5.036$): efeito médio praticamente nulo. Em adultos, o ponto de inflexão fica entre 12 e 24 opções — não 4. Em crianças, um estudo de 2023 não encontrou o efeito **nem com 53 opções**, e uma revisão de 2024 indica que crianças sofrem *menos* que adultos com arranjos grandes. A lógica "mais nova, menos opções por *overload*" está invertida [11].
Decision fatigue como base do teto de volume	Dois estudos multilaboratório de replicação registrada com efeito nulo ($d = 0{,}04$); o caso dos "juízes com fome" se explica por ordem de apresentação [21]. O sinal real é monotonia, não esgotamento.
"Idade da criança = minutos de atenção"	Não tem origem primária publicada. Circula em três variantes (1×, 2–3×, 5× a idade) que diferem por um fator de cinco. O análogo adulto — "10 a 15 minutos de atenção em aula" — foi formalmente desmontado.
Piaget para vetar abstração por idade	Idades fixas e universais são justamente a parte refutada. Use ciclos de prioridade cognitiva [15] e dependência de suporte [18].
Bloom ou DOK como régua de adequação etária	São instrumentos de alinhamento entre currículo e avaliação, ortogonais à série — Webb prevê DOK 3 e 4 em todos os anos. SOLO [3] é a única das três defensável para calibrar a resposta esperada.
Classificar a tarefa pelo verbo do enunciado	Refutado sobre 940 itens: o verbo não é *proxy* confiável do processo cognitivo. O teste de alinhamento de verbo continua útil como sinal, **nunca** como veredito automático.

21.1 O que mudou na revisão de setembro de 2026

A revisão não ajustou números por gosto: trocou a fundamentação de quatro critérios, porque a pesquisa mostrou que as fontes citadas não sustentavam o que a rubrica afirmava.

Mudança	Antes	Por quê
Teto de opções no Médio	4 a 5	A capacidade visual de trabalho aos 16 anos é 2,72 itens, ainda abaixo do adulto. Tratar o Médio como capacidade adulta superestimava.
Base do teto de opções	*choice overload*	O efeito não replicou. O que sustenta os tetos é *span* complexo de memória de trabalho — outro construto.
Volume de decisões	obrigatório, com teto	Virou desejável e marcado como convenção; o sinal real é monotonia.

Mudança	Antes	Por quê
Tetos de duração	12 / 15 / 20 min	Observação de 52 salas: queda monotônica com a duração, sem minuto mágico. A escala foi achatada.
Teto agregado da missão	não existia	Três atos no teto antigo somavam mais que uma aula inteira.
Formato da atividade	ignorado	Individual ou em pequeno grupo sustenta atenção bem melhor que turma inteira.
Justificativa de abstração	Piaget	A parte citada era a refutada. Substituída por Demetriou e Fischer.
Raciocínio sobre sistema invisível	não existia	É o núcleo do letramento digital e não tinha régua. Entrou com três portas.
Cenário da escola pública	suposição	Substituído por dados de INEP, CETIC e Saeb.

22 O papel do professor

A falha mais comum em EdTech é desenhar como se a professora não existisse — ou pior, como se ela fosse um obstáculo a contornar. Aqui ela é o centro da experiência, e isso é auditado por critério próprio na avaliação pedagógica.

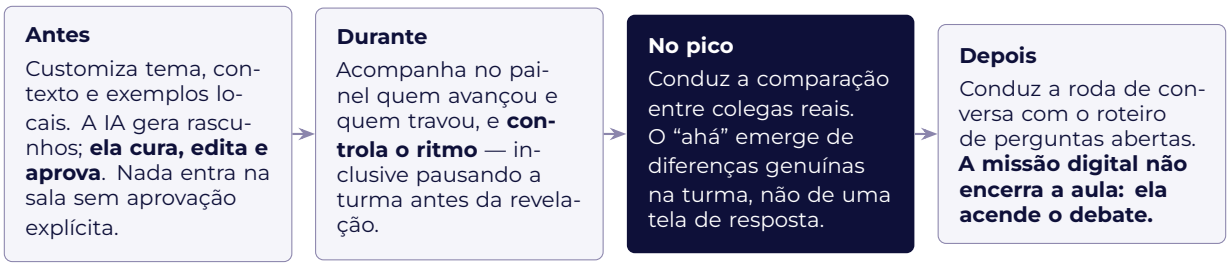


Figura 14: Os quatro momentos em que a autonomia docente é exercida — e auditada.

Há um quinto ponto, menos nobre e igualmente decisivo: **ritmos diferentes**. Toda missão precisa prever o que fazem os estudantes que terminam cedo, sob pena de eles darem *spoiler* para o resto da turma e destruírem a virada para todo mundo.

PARTE V

O que protege

Desenhar para crianças impõe uma responsabilidade que nenhum software comum tem. Aqui ela está escrita em código.



uncoria®

23 Alinhamento curricular e o primado do letramento digital

Em outubro de 2022, o Conselho Nacional de Educação homologou a **Resolução CNE/CEB n.º 1/2022**, que instituiu as normas sobre Computação na Educação Básica como complemento oficial à BNCC [7]. A norma organiza o ensino de computação em três eixos:

Pensamento computacional
Resolver problemas por algoritmos, decomposição, abstração e reconhecimento de padrões.
avalia e alerta

Mundo digital
Compreender a infraestrutura: *hardware*, *software*, codificação de dados, inteligência artificial.
avalia e alerta

Cultura digital
Uso ético, crítico, responsável e reflexivo das tecnologias digitais na sociedade.
É A ÚNICA QUE BLOQUEIA

A regra de ouro do gate curricular

Na esteira Unooria, a **única** habilidade da BNCC com poder de bloquear a missão é o cumprimento **ativo** de letramento digital — o eixo Cultura Digital.
Habilidades de componentes tradicionais (Língua Portuguesa, História, Matemática), socioemocionais ou do eixo Pensamento Computacional recebem nota, sugestão e alerta ●, mas **nunca travam a esteira**. Sem letramento digital comprovado na prática, a missão recebe ●.
Esta é uma **decisão de governança de produto**, não um achado de pesquisa — e está declarada como tal na própria rubrica, para que ninguém a defenda como se fosse ciência.

A decisão se apoia em três razões, e vale enunciá-las porque elas são contestáveis — e devem ser contestadas com argumento, não com antiguidade:

1. **Contra o tecnologismo vazio.** Muitas plataformas de programação infantil ensinam a arrastar blocos para mover um boneco e param aí. Criar um robô sem saber discernir uma notícia falsa, ou sem entender o valor dos próprios dados, não forma cidadão para este século [10, 43].
2. **Urgência de proteção.** Crianças brasileiras passam horas conectadas. A escola tem dever cívico e legal de prepará-las contra assédio digital, vazamento de imagem, golpe e manipulação algorítmica. Letramento digital é, antes de tudo, competência de autoproteção [29].
3. **Vivência, não teoria.** O critério exige que o estudante **pratique** o letramento digital na interface. "Ler uma dica de segurança" reprova. Ele precisa identificar, configurar, reportar ou avaliar — com o dedo na tela.

23.1 Como o verificador de BNCC funciona

Modelos de linguagem inventam códigos curriculares com naturalidade e confiança. Por isso a esteira **não confia na memória do modelo**: o agente 14 é código Python puro, que consulta a lista oficial do MEC em `prompts/references/bncc-computacao.json`.

1. Existência real
Todo código citado (ex.: `EF06C006`) tem de existir na lista oficial. Código inventado bloqueia.

2. Ano compatível
O código tem de corresponder ao ano da missão. Missão de 3.º ano não cita habilidade exclusiva de 9.º.

3. Letramento digital presente
No Fundamental, a missão sem ao menos uma habilidade de Cultura Digital é reprovada.

Figura 15: As três regras do verificador determinístico de currículo. Nenhuma delas passa por um modelo de linguagem.

A escolha de fazer isso em código, e não por avaliador, é deliberada: é regra mecânica, e avaliador baseado em modelo tem viés documentado de leniência [45]. Um avaliador aceitaria um código plausível; uma comparação literal, não.

Duas consequências práticas apareceram em outubro. A primeira: o verificador lê o bloco 11 da *spec* inteiro, inclusive as subseções numeradas (`11.1 Habilidades e momentos`). Até 6 de outubro ele cortava o bloco no primeiro título numerado, e a *spec* que abria o currículo com uma subseção ficava "sem nenhum código de habilidade" — o falso positivo que o PO contestou na missão do 6.º ano com razão. A segunda: **o texto oficial de cada habilidade é citado uma vez só** em cada documento. O filtro de recitação do modelo corta a resposta que transcreve texto publicado por extenso, e o *one-pager* que repetia a mesma habilidade três vezes saía vazio (Parte VII).

23.2 Quando nenhuma habilidade do ano serve

Há um caso que a primeira edição deste manual não cobria, e que o time encontrou na prática: a missão é boa, o material é bom, e **nenhuma habilidade de Cultura Digital daquele ano descreve a ação que o estudante pratica**. Aconteceu com uma missão de 6.º ano sobre filtragem algorítmica — a Cultura Digital do 6.º ano simplesmente não fala disso.

A decisão do time foi **bloquear e devolver**, não acomodar. A esteira grava um marcador de decisão pendente com dono nomeado, e a missão não segue.

Por que não afrouxar aqui

Afrouxar teria duas saídas, e as duas são piores. Aceitar uma habilidade de outro eixo transformaria o único critério curricular bloqueante em decorativo. Aceitar uma habilidade de Cultura Digital que *não* descreve a ação produziria um documento que afirma cumprir o currículo sem cumprir — e alguém adiante confiaria nessa afirmação.
O que a esteira faz em vez disso é nomear a decisão: mudar a missão de ano, ancorar em outro componente, ou aceitar que a missão não é de letramento digital. Essas são escolhas de produto, e a máquina não tem autoridade sobre nenhuma delas.

23.3 O contrato de evidência: o que o número sobre uma criança pode afirmar

Declarar a habilidade não basta; é preciso declarar **como ela é medida**. A *spec* traz uma tabela de medidas que um portão determinístico lê, e cada medida tem um grau:

Grau	O que significa, e o que ele autoriza afirmar
`scored`	Dois ou mais sinais independentes. Tem número, e o número entra nas médias.
`indicative`	Um sinal só. Vira texto, e **fica fora das médias** — porque um sinal não sustenta uma nota.
`zero`	O estudante teve a chance e não fez. Entra na média.
`none`	**A oportunidade não chegou até ele.** Sai da conta.

A distinção entre `zero` e `none` é uma regra de proteção

Tratar "não teve chance" como "escolheu não fazer" faz uma criança **parecer ter recusado o que nunca lhe foi oferecido**. E o número será lido por um adulto que decide sobre ela.
A esteira não confia na declaração: das 25 medidas marcadas como `scored` nas primeiras execuções, **só 10 nomeavam os dois sinais** que o próprio grau exige. Hoje um portão determinístico rebaixa ou reprova cada uma que não nomeia — o grau inflado era invisível justamente por parecer rigor.

24 Inteligência artificial com humano no controle

A literatura sobre segurança em IA é clara quanto aos riscos de alucinação, viés e manipulação quando crianças interagem diretamente com modelos generativos abertos [4]. A diretriz da plataforma é inegociável e tem três partes:

Salvaguarda	O que significa na prática
Geração na preparação, nunca ao vivo sem filtro	A IA é ferramenta de produtividade **para a professora**, no planejamento. Textos, personagens e posts simulados são revisados, ajustados e aprovados por ela antes de chegarem à turma.
A criança interage com conteúdo curado	Em sala, o estudante vê um banco fechado e aprovado. Isso elimina por construção a exposição a resposta imprópria, discurso de ódio ou conteúdo de risco.
Personagem com IA opera fechado	Quando a missão usa um personagem interativo, ele roda sob restrições rígidas de sistema, que o impedem de sair do tópico pedagógico ou emitir opinião própria.

25 Privacidade: o artigo 14 da LGPD

O tratamento de dados de menores no Brasil é regulado pelo **art. 14 da Lei n.º 13.709/2018** [6], lido em conjunto com o Estatuto da Criança e do Adolescente [5]:

> *"O tratamento de dados pessoais de crianças e adolescentes deverá ser realizado em seu melhor interesse, considerados os direitos fundamentais à vida, à privacidade e ao pleno desenvolvimento."* (art. 14, § 1.º)

A esteira converte isso em três exigências verificadas — duas delas por código, na dimensão de Dados e LGPD da banca:

1. **Nada de captura abusiva de mídia.** É vedado exigir que o estudante grave vídeo do próprio rosto, foto de casa ou áudio aberto sem justificativa pedagógica indispensável e consentimento dos responsáveis.
2. **Isolamento por turma e escola.** Todo acesso a banco de dados roda sob política estrita de isolamento (`FORCE ROW LEVEL SECURITY`). Dado de criança de uma escola não pode, sob nenhuma hipótese de falha de software, ser visto por outra instituição.
3. **Telemetria de progresso, não de pessoa.** Os eventos registram o percurso na missão — que etapa foi concluída, quantas tentativas — e nunca localização geográfica ou identificador de publicidade. O contrato de evento é conferido literalmente pelo agente 36: verbo, objeto, resultado e contexto, com tabela de leitura declarada.

26 Ética de acolhimento e não-rotulação

Proibido

Classificar o estudante — "aluno lento", "perfil desatento", qualquer categorização psicológica, moral ou ideológica. Também é proibido fazê-lo sentir-se culpado ou incapaz por uma escolha dentro da simulação.

O que a missão faz no lugar

Julga **a escolha, nunca a pessoa**, e mostra a consequência dentro da história, com humor e empatia: "Opa! O computador travou. Que pista deixamos passar?" O erro é informação sobre o sistema, não sobre a criança.

Essa regra tem um efeito de projeto que costuma passar despercebido: ela **proíbe a tela de resposta certa**. Se o sistema declarasse quem acertou, ele estaria classificando — e a comparação entre colegas, que é o coração do momento "ahá", viraria ranking. Por isso a revelação é conduzida pela professora com dados de comparação, e não por uma tela de pontuação.

PARTE VI

Os agentes, um a um

O que cada um faz, e em que se apoia para decidir. Sem essa coluna, uma rubrica acaba defendida por antiguidade.



uncoria®

27 Como ler este catálogo

Esta parte existe por uma razão específica: uma regra que não diz de onde veio é indefensável quando alguém a contesta — e indefensável quando alguém deveria contestá-la e não o faz. Por isso cada agente aparece aqui com três informações: o que faz, e sobre que natureza de fundamentação a sua regra se apoia.

EVIDÊNCIA
Deriva de pesquisa publicada, com citação. Pode ser derrubada por pesquisa melhor — e já foi, quatro vezes.

CONTRATO
Deriva de documento normativo: BNCC, LGPD, manifesto da plataforma. Não se pesquisa: confere-se.

CONVENÇÃO
A equipe escolheu por consistência de produto. Legítimo, desde que declarado — e sujeito a mudar sem pesquisa nova.

São **114 posições numeradas**: 43 agentes com modelo de linguagem e **71 verificações em código** — Python, marcadas em azul claro nas tabelas. Quase dois terços da esteira não usam modelo para decidir, e isso é o desenho, não uma economia: toda regra que *pode* ser mecânica foi tirada dos avaliadores. Seis delas — as de fidelidade (03a, 16c, 33c e 51) e as de coerência com o aprovado (48e e 49e) — chamam por dentro um juiz de modelo, mas quem dá o veredito é o código, que confere a citação do juiz.
O revisor do *one-pager* aparece duas vezes, como 07 (movido pelo gate estrutural) e como 10c (movido pelo parecer de adequação); a ficha do manifesto é conferida em Produto (15) e de novo em Dev (33), contra o mesmo vocabulário. Alguns números se repetem: 33 é o rastro do design e a ficha em Dev; 33b, a matriz do design e o veredito da banca de entrada; 50 e 51 são a matriz final e a fidelidade do Dev, e também o diagnóstico e o aprendizado; as três peças de cada banca de etapa dividem um número (15h, 29f). E a ordem da lista não é a dos números: a conta da virada (03aa) e a fidelidade (03a) rodam dentro do ciclo do núcleo, e a banca de entrada (39–41) roda antes do 33.

28 Produto (00b–16f)

Passo	O que faz	Em que se apoia
00b Missões já geradas	Lê o índice e entrega ao núcleo as missões mais próximas do ano do briefing — as geradas e as publicadas na plataforma —, para a nova não repetir uma anterior.	CONTRATO interno. A semelhança é lexical e só *recupera e ordena* — quem reprova é o micro-gate.
00c Mapa conceitual do material	Conceito limiar, conceitos núcleo e de apoio com trecho citado, crenças de entrada que o bimestre derruba, vocabulário do Explicar.	CONVENÇÃO com fundamentação em conceitos limiares. Existe porque o núcleo *recebia* o material e nada pedia que o usasse — ver Seção 18.

Passo	O que faz	Em que se apoia
00d Ideação de mecanismos	Três mecanismos distintos para o conceito limiar, cada um com a anatomia do "ahá", pontuados na rubrica de profundidade. Dispensada quando o PO anexa o *one-pager*.	CONVENÇÃO. O mecanismo que veio no briefing entra como candidato, não como decisão — e trocá-lo continua sendo ato humano.
01 Núcleo narrativo	Descoberta, conceitos vividos em tela, anatomia do "ahá", narrativa e as seis etapas, a partir do briefing, dos anexos, do mapa e do mecanismo escolhido. É o único agente que lê a mensagem e os anexos como chegaram.	CONVENÇÃO — o *framework* de seis etapas é desenho de produto. A exigência de *gap* de informação tem lastro em curiosidade epistêmica [31].
01b Missões já geradas, pelo núcleo	Refiltra o índice pelo ambiente, eixo e descoberta do núcleo recém-escrito. É esta lista que o micro-gate julga.	CONTRATO interno.
01c Cobertura conceitual	Núcleo × mapa: o limiar vive no Decidir ou no "ahá", todo conceito tem destino, a anatomia tem os seis passos, e **o trecho citado existe mesmo no material**.	CONTRATO interno. A última regra existe porque citação inventada é o modo de falha mais fácil de um modelo obrigado a fundamentar.
02 Micro-gate do núcleo	*Spoiler*, *gap* de informação, escolha significativa, *job-to-be-done* e semelhança com missão anterior — a mesma missão em outra versão não conta.	CONVENÇÃO derivada de 01. Separar quem escreve de quem critica reduz autopreferência [45]. Só bloqueia com citação que exista.
02b Juiz de profundidade	Nota de 1 a 4 em seis dimensões, com trecho citado e instrução de reescrita.	EVIDÊNCIA e CONVENÇÃO declarada: a anatomia do "ahá" tem lastro em previsão e conflito cognitivo; **os cortes estão por calibrar**.
03aa Conta da virada	Refaz em código o ranking da tabela 1.3 do núcleo — opções, critérios, pesos, valores antes e depois do dado que faltava — e confere que os vencedores são os declarados, e diferentes. Missão sem ranking declara "não se aplica".	CONTRATO interno. Nasceu do 9.º ano: o juiz de profundidade deu nota 4 a uma virada que, feita a conta, não existia (Parte VII).
03a Fidelidade ao pedido do PO (núcleo)	O núcleo contra todos os pedidos de *Refazer* e o briefing. O juiz cita, o código confere; com ●, o núcleo volta ao autor com o relatório.	CONTRATO com quem pediu: o pedido do PO é a especificação. A citação conferida em código existe contra a leniência de avaliadores [45].

Passo	O que faz	Em que se apoia
03 · **03b** · **03c** Vereditos do núcleo	Registram o micro-gate; **recontam o veredito de profundidade nas notas do juiz**, não no que ele escreveu na conclusão; e encerram o ciclo quando as quatro conferências aprovam.	CONTRATO interno, contra a leniência de avaliadores [45].
04 Seções de apoio	Completa o *one-pager* canônico (seções 2 e 5–18) e o *brief* de escopo. Com *one-pager* do PO, parte dele e declara cada alteração com o pedido citado.	CONVENÇÃO — formato de produto. A separação em relação a 01 é deliberada: densidade de instrução degrada o cumprimento [24].
04d · **04e** *One-pager* do PO preservado	Cada linha e célula do *one-pager* do PO no gerado; trecho perdido volta ao 04 com a lista. Sem anexo, não se aplica.	CONTRATO com o PO. Nasceu da missão do 6.º ano: 254 de 304 trechos reescritos e todos os gates verdes (Parte VII).
04f *One-pager* existe	Sem *one-pager* depois das duas tentativas do 04, a etapa para com uma mensagem que diz quem ficou mudo e por quê.	CONTRATO interno. Antes, o gate seguinte quebrava com o nome de uma variável.
04b Contrato de plataforma	*Lint* a cada rodada do gate: canal de sala por consulta periódica, estado no servidor, SAEB só nos anos avaliados, número com origem; seção do DS e habilidades que o briefing fixou; decisão atribuída ao PO sem registro dele.	CONTRATO da plataforma e do briefing. O pedido mais recente do PO vence o briefing original.
05 Gate estrutural	Critérios de estrutura, ambiente, logística, evidências, viabilidade e infraestrutura. Lê o briefing. Escreve a correção pronta para colar.	CONVENÇÃO com critérios de EVIDÊNCIA: teto de decisões e duração vêm da fonte única. Rubrica com teste prático e contraexemplo sustenta concordância [25].
06 Veredito do gate	Lê o envelope JSON e decide repetir, seguir ou bloquear. "NÃO APROVADO" sem envelope reprova.	CONTRATO interno.
07 Revisor do *one-pager*	Aplica o complemento que o gate escreveu; não inventa decisão de produto, não desfaz restrição do briefing — registra o conflito.	CONVENÇÃO. Separa julgar de consertar.
08 Avaliação pedagógica	Currículo, papel do professor, *guardrails* e a relação entre o que a interface permite e o que ela protege.	CONTRATO: BNCC Computação [7]. A regra de que só letramento digital bloqueia é decisão de governança, declarada como tal.

Passo	O que faz	Em que se apoia
09 Experiência do estudante	Carga cognitiva, faixa etária, linguagem, duração e agência.	EVIDÊNCIA — o passo mais ancorado da esteira [19, 13, 20, 2, 33, 26]. Todos os números vêm da fonte única.
10 Adequação pedagógica	Parecer em seis eixos com ajustes acionáveis. Recebe o briefing.	EVIDÊNCIA [3, 34, 39]; CONTRATO para a BNCC; CONVENÇÃO para o placar. Não carrega régua própria.
10b Veredito do parecer	Parecer sem ajuste exigido encerra o ciclo; com ajuste, devolve ao revisor.	CONTRATO interno.
10c Revisor movido pelo parecer	O mesmo agente 07, aplicando os ajustes que o parecer exigiu.	CONVENÇÃO.
10d · **10e** *One-pager* do PO, depois dos pareceres	A conferência final, depois de gate, pareceres e revisores. Trecho perdido sem pedido vai ao restaurador; se sobrar, Produto fica ●.	CONTRATO com o PO.
10f Restaurador do *one-pager* do PO	Devolve, literais, os trechos do PO que a esteira reescreveu; o que um gate ou parecer pedia vira `[PROPOSTA — validar com PO]`.	CONVENÇÃO. Quem decide mudar o *one-pager* do PO é o PO.
11 Gate final de produto	Coerência entre os três pareceres e veredito de liberação. Não declara que o pedido do PO foi atendido — isso é da fidelidade.	CONVENÇÃO. Existe porque avaliadores paralelos podem julgar *versões diferentes* do mesmo documento [12].
12 Veredito final	Lê o envelope do gate final.	CONTRATO interno.
13 *Spec* da missão	Os 12 blocos do contrato contra o qual Design e Dev constroem, mais o **glossário canônico**. **Traduz o *one-pager* aprovado, não o revisa**: tempos e cenários no bloco 6, podcast no 8, habilidades no 11; ressalva que o PO não incorporou vai para uma tabela própria do bloco 12.	CONTRATO — manifesto da plataforma, isolamento por linha, contrato de eventos. O glossário é CONVENÇÃO de coerência, conferida em código adiante (29d).
14 Conformidade BNCC	Código existe, é do ano, e há letramento digital. Confere também **o que a *spec* diz sobre a habilidade**, e lê as subseções numeradas do bloco.	CONTRATO, verificado em código [7].

Passo	O que faz	Em que se apoia
15 Ficha do manifesto	Dez campos do bloco 0 contra o vocabulário fechado. Seção antiga do DS escolhida pela esteira bloqueia e aponta a sucessora; a escolhida pelo PO é ressalva; a que o briefing fixa é trava.	CONTRATO. Roda em Produto porque o conserto ali custa uma reescrita da *spec*, não o descarte do Design.
15b Matriz de evidência	Grau `scored` com dois sinais nomeados, toda habilidade com momento em tela e medida ligada, gabarito no servidor.	CONTRATO. Das 25 medidas `scored` das primeiras execuções, só 10 nomeavam dois sinais.
15c Perguntas de produto	Quantidade fechada, decisão com limiar, corte do vocabulário, e a ação que responde cada pergunta declarada na tabela de eventos.	CONTRATO interno.
15d Juiz das perguntas	O que regex não vê: a pergunta muda uma decisão nossa, é sobre o produto e não sobre a criança, e não é uma transversal repintada.	CONVENÇÃO.
15e Veredito do juiz	Reprovação em pergunta declarada pelo responsável de produto bloqueia; em proposta, segue como ressalva para ele decidir.	CONTRATO interno.
15f Rastro de lacunas da *spec*	Marcadores de lacuna na *spec*, por dono: o de produto ou de insumo volta para quem escreveu a *spec*; a decisão do PO fica ●; o de Design e o de Dev ficam para a etapa delas.	CONTRATO interno. Cobra cada lacuna na etapa que ainda pode consertá-la.
15g *Spec* × *one-pager* aprovado	As mesmas habilidades do contrato de evidência, o mesmo tempo em tela, o podcast; parâmetro “a definir por produto” que a *spec* decidiu; glossário que proíbe termo aprovado. Divergência volta ao autor da *spec*.	CONTRATO com o PO. Nasceu da *spec* como segundo autor (Seção 14).
15h Banca no fim de produto	As dimensões de produto e de pedagogia sobre a *spec*, com o conteúdo aprovado ao lado; a consolidação, em código, dá o pior das duas.	CONTRATO e EVIDÊNCIA, como a banca de Dev. O achado volta ao autor da *spec* antes do parecer do PO — ver Seção 11.

Passo	O que faz	Em que se apoia
16 Veredito da *spec*	Currículo, manifesto, evidência, perguntas, lacunas, conteúdo aprovado e banca conformes encerram o ciclo.	CONTRATO interno.
16c Fidelidade ao pedido do PO e ao briefing	Núcleo, *one-pager* e *spec* juntos contra os pedidos feitos a Produto e o briefing. Pedido sem prova é ● e recusa o *de acordo*; conflito com regra da esteira vira decisão do PO (●).	CONTRATO com quem pediu. O juiz é modelo (`61_juiz_de_fidelidade`); o veredito é do código.
16b *One-pager* resumido	O formato curto do time de produto: ficha, ideia, componente curricular, socioemocionais, seis etapas, professor, contrato de evidência e *takeaway*. Sai em três formatos; vale o canônico.	CONVENÇÃO de comunicação, com regras explícitas de registro — ver Seção 31.1.
16e · **16f** Resumido conferido	As habilidades, o tempo em tela e o Canal B do resumido contra o canônico; divergente volta ao autor.	CONTRATO interno. Em 7 de 15 sessões o resumido não trazia o Canal B aprovado.

29 Design (16d–33c)

Agente	O que faz	Em que se apoia
16d Conteúdo aprovado	Grava as seções do *one-pager* aprovado, literais, para os agentes de design que não leem o *one-pager*. Depois de um *Refazer* de Produto, cada autor recebe também o que mudou nelas.	CONTRATO com o PO.
17 Fechamento de lacunas	Varre a *spec* célula a célula, registra decisões com base declarada e pedidos de re-*spec*, e consolida.	CONVENÇÃO. Preenche ausência, nunca altera presença — a regra existe para que Design não redecida produto.
18 Separação e consolidação	Copia a *spec* consolidada, preserva a original e separa o relatório de preenchimento.	CONTRATO interno.

Agente	O que faz	Em que se apoia
19 Ambiente simulado	Condicional: decide o ambiente quando a *spec* cala — app clonado, app novo ou mini-jogo. Dormente, emite só a ficha que os demais consomem.	CONTRATO (a lista fechada de seções do *design system*) mais CONVENÇÃO para o mini-jogo dos anos iniciais.
20 *Spec* com o bloco do ambiente	Copia a ficha do ambiente para a *spec* e registra o estado.	CONTRATO interno.
20b *Spec* consolidada × aprovado	A *spec* que o fechamento de lacunas e o ambiente reescreveram, contra o *one-pager* aprovado. Só o que piorou desde a de Produto vira *ticket*, pelo QA.	CONTRATO com o PO. Linha da *spec* perdida na consolidação bloqueia.
21 Briefing de ofício	Aposta, forma do arco, pico, *beat* memorável, mapa de *beats* e não-negociáveis.	CONVENÇÃO de ofício narrativo. **É a parte menos fundamentável da esteira, e está declarada assim**: são heurísticas de dramaturgia, não achados.
22 Jornada · **23** Fluxo	Raias por etapa, cenários de dispositivo e inventário de telas, com identificadores no padrão; grafo de estados com condições, gates e cinco testes de fluxo. Ambos em Mermaid.	CONVENÇÃO. O diagrama existe porque representação externa reduz a carga de manter a estrutura na cabeça [41].
24 · **27** *Copy* (duas passadas)	Dicionário de *strings* por tela, calibrado à faixa; depois revisão dentro da tela já construída.	EVIDÊNCIA para a calibragem de linguagem por faixa (fonte única) e CONVENÇÃO para o formato de duas passadas.
25 Camada CAA	Símbolos, modos alternativos de resposta e pranchas de comunicação. Só roda quando a faixa ou a *spec* pedem.	CONTRATO de acessibilidade. A regra de que o símbolo vem *junto* do texto, nunca no lugar dele, é redundância deliberada e benéfica para quem não decodifica com fluência [28].
26 Telas	Especificação por tela e estado sobre a seção do *design system* do ambiente, com a tabela "Geração no Stitch": uma linha por tela e estado, com o *prompt*.	CONTRATO (tokens do *design system*) mais EVIDÊNCIA para o teto de elementos novos por tela [41].
26b Telas no Stitch	Gera no Stitch, fora do modelo, toda tela e estado da tabela e do inventário da jornada, em paralelo, e registra o identificador que o Stitch devolveu. A refeita nasce "· vN".	CONTRATO interno. O modelo gerava de 27% a 68% do inventário (Seção 15).

Agente	O que faz	Em que se apoia
27b *Copy* preservada	Reanexa à *copy* revisada, sem mudança, as seções da primeira passada que a revisão descartou — *prompts*, banco curado, roteiro do podcast.	CONTRATO interno.
28 Protótipo	Arquivo HTML único, sem dependências, com as telas navegáveis — um quadro marcado por estado —, o painel da etapa e as cadências do pico.	CONVENÇÃO. É o artefato que a revisão humana abre; a exigência de arquivo único e offline é operacional.
29 Artefatos e links	Separa o HTML do relatório, colhe os links das telas — só da versão atual — e monta a folha de diagramas. **Conta as telas do projeto contra o inventário da jornada**: faltando tela, ou tela com identificador fora do padrão, bloqueia com dono no agente de telas.	CONTRATO interno.
29a *Lint* do protótipo	Confere o arquivo HTML em si — o que o QA de entregáveis não vê, porque lê só o texto.	CONTRATO interno. Existe porque o QA auditava a *descrição* do protótipo e dava por bom um arquivo que não abria como descrito.
29d Coerência entre documentos	Jornada, fluxo, *copy* revisada e protótipo contra o glossário canônico da *spec*. Variante proibida de um nome bloqueia; papel ou idade de personagem, número, rótulo de botão e nome de arquivo divergentes ficam ●.	CONTRATO interno. Cada documento, sozinho, pode estar bem escrito; a contradição só aparece lendo todos juntos.
29b Espelho no Miro	Publica no Miro os mesmos diagramas Mermaid, como board navegável.	CONVENÇÃO. O texto continua sendo a fonte e a integração indisponível não interrompe a esteira.
29e Cobertura de telas	Toda tela do inventário no fluxo, na *copy* revisada, na especificação e nos quadros do protótipo, **por identificador**; falta vira *ticket* com o dono.	CONTRATO interno. Contar telas não basta: cinco quadros com três estados da mesma tela aprovavam com três telas faltando.
29f Banca no fim de design	Produto realizado em tela, e design e imersão, sobre os documentos de design; a consolidação dá o pior das duas. O bloqueio vira *ticket* nesta etapa.	CONTRATO e CONVENÇÃO. Ver Seção 11.

Agente	O que faz	Em que se apoia
30 QA de entregáveis	Fidelidade à *spec*, *diff* de *strings*, currículo, faixa, fluxo, **visão do professor**, briefing, ambiente e CAA.	CONVENÇÃO, com uma regra de EVIDÊNCIA: é proibido reportar número que não foi medido. Só bloqueia com citação que exista.
30b Afirmações do QA	As contagens de telas que o QA escreveu (“todas as N telas”, “N de M”) contra o inventário, o Stitch e o protótipo, e os nomes de arquivo contra os que a esteira grava. Divergência reescreve o veredito do QA como bloqueado.	CONTRATO interno. O QA é um modelo: o que ele escreve sobre contagem é afirmação, não fato.
31 Veredito de design	Lê o envelope do QA.	CONTRATO interno.
33 Rastro de lacunas do design	Marcadores de lacuna na *spec* consolidada e nos documentos de design: o de Design bloqueia a etapa; o que o fechamento de lacunas fechou não conta; o de Produto já aprovado e o de Dev ficam ●.	CONTRATO interno.
33b Matriz de rastreabilidade	Cada requisito do produto procurado em cada documento de design; o que se perdeu sai com o dono.	CONTRATO interno — ver Seção 14.
33c Fidelidade ao *Refazer* de design	Cada pedido feito ao Design conferido na *spec* consolidada, no *brief*, na jornada, no fluxo, na *copy*, na CAA e nas telas. Sem pedido de Design, não roda.	CONTRATO com quem pediu. O juiz cita, o código confere.
32 Retrabalho de design	Reexecuta só os agentes com *ticket*, na ordem de precedência — telas antes da *copy* —, cada um com a própria versão anterior, e refaz o protótipo, o *lint*, a coerência e a cobertura depois.	CONTRATO interno. O despacho é seletivo porque redundância com falhas correlacionadas não aumenta cobertura [27].

30 Dev (33a–49c) e transversal (50–52)

Agente	O que faz	Em que se apoia
33a Conteúdo aprovado	As mesmas seções literais do Design, para o banco de conteúdo, a banca e o PRD.	CONTRATO com o PO.
39–41 · **33b** Banca de entrada	Produto, design e pedagogia sobre o que as duas etapas aprovaram, com a visão cruzada. Não reescreve nada; ● impede o Dev de rodar e lista os bloqueios por dono.	CONTRATO e EVIDÊNCIA. A segunda verificação existe para pegar quem aprovou sem ler e a correção que não pegou.
33 Ficha do manifesto	Bloco 0 contra as listas fechadas: produto, séries, unidade, público, *viewport*, permissões, *capabilities*, seção do DS, liberação por turma. Não conforme impede o PRD.	CONTRATO da plataforma.
34 *Handoff* para dev	Modelo de insumo preenchido, *stories* com critérios fechados, eventos, escopo V1/V2 e hipóteses.	CONTRATO da plataforma: assinatura de evento, papéis, canal de sala.
35 *Brief* de assets	Inventário, especificação por tipo, critérios de reprovação e ordem de produção. O *status* de cada item tem vocabulário fechado: "a produzir", "rascunho" ou "escrito".	CONVENÇÃO de produção. Homologar e dar "pronto" é ato humano, e um agente que o escreve afirma o que ninguém fez.
36 Contrato de telemetria	Evento = verbo · objeto · resultado · contexto; medida com tabela de leitura; sem WebSocket; seis papéis; quota declarada. E o que o *handoff* afirma da plataforma, contra o contrato da base.	CONTRATO. Comparação literal contra o contrato real da API.
37 Veredito do *handoff*	Telemetria conforme encerra o ciclo; reprovada devolve o *handoff* com o relatório.	CONTRATO interno.
35b Banco de conteúdo	O texto integral de cada item do *brief* — documentos fictícios, atas, tabelas, transcrições, roteiros —, as falas da IA simulada por estado e, quando o estudante escreve no chat, a **árvore de intenções**: gatilhos fechados e a resposta para o que não casa. Lê o núcleo aprovado.	CONVENÇÃO de produção. Existe porque o banco padrão saía só *descrito*, e o PRD mandava codificar sobre conteúdo que não existia.

Agente	O que faz	Em que se apoia
35c Banco de conteúdo conferido	Item do *brief* sem texto no banco bloqueia; *status* fora do vocabulário bloqueia; chat sem árvore de intenções bloqueia; estado em que a IA fala sem fala escrita fica ●. Em ciclo com o 35b.	CONTRATO interno.
38 Rastro de lacunas	Marcadores em *spec*, design, *handoff* e assets, **por dono**. Só impede o PRD a lacuna de Dev — ou a de etapa anterior escrita pelo próprio *handoff* ou *brief*. O que depende da plataforma vira pendência humana ●.	CONTRATO interno. A categoria *humano* foi criada depois da primeira execução completa — ver Seção 32.
42 · 43 Banca de dados e técnica	Dados, telemetria e LGPD; técnica e arquitetura. Em paralelo, sobre os documentos de Dev.	CONTRATO. A separação existe porque um avaliador único com 70 critérios erra de forma sistemática [24] e porque falhas correlacionadas não se compensam [27].
44 Consolidação da banca	Veredito único ●/●/●: **a pior das dimensões**, com os bloqueantes lidos do arquivo de critérios e só os achados com citação que exista.	CONTRATO: regra determinística. A consolidação deixou de ser feita por um modelo justamente porque ele arredondava para cima.
45 Veredito de dev	Lê o envelope da banca.	CONTRATO interno.
46 Retrabalho da banca	Reexecuta o autor do documento de Dev que o achado aponta — *handoff*, *brief* ou banco — e reconfere o que ele reescreveu. Achado sobre a *spec* ou o design vira pendência nomeada para a etapa dona.	CONTRATO interno. Até 8 de outubro, este passo reescrevia jornada, fluxo, telas e *copy* depois do parecer de Design.
47 PRD padrão ouro	As 23 seções que alimentam quem codifica, com o contexto da plataforma lido da base fixada. Remete ao conteúdo aprovado em vez de reescrevê-lo; não crava versão do DS; não escreve *status* de gate.	CONTRATO da plataforma e da LGPD (dado de criança e adolescente).
48 Pendências e consistência do PRD	Marcadores, consistência interna, o PRD contra a plataforma, o design, o *handoff* e o banco, e o PRD que ainda não dá para codar (Seção 14.2).	CONTRATO. Determinístico porque é comparação literal — e porque o PRD é o documento com mais restrições simultâneas da esteira.

Agente	O que faz	Em que se apoia
48a Coerência KPI ↔ evento ↔ tela	Todo indicador com pergunta, decisão, corte e um evento que dispara *na tela que ele nomeia*; os sinais universais das perguntas transversais presentes na tabela de eventos.	CONTRATO interno. Um indicador que ninguém consegue calcular promete um número que não existe.
48c PRD × *one-pager* aprovado	As habilidades na matriz da seção 14, um KPI por habilidade, o tempo em tela, o podcast com requisito funcional, só caminhos de arquivo que existem, e o texto aprovado sem outra redação.	CONTRATO com o PO.
48e · **49e** Coerência com o aprovado	Um juiz aponta onde o corpo do PRD — e, no ciclo seguinte, o do pedagógico — afirma o contrário do conteúdo aprovado, citando os dois lados; o código confere as citações.	CONTRATO com o PO. O pedagógico era o único documento aprovado por pessoas sem juiz de contradições.
48b Veredito do PRD	PRD conferido encerra o ciclo; reprovado devolve ao redator com a própria versão, até o teto.	CONTRATO interno. Foi o último ciclo a ganhar volta — ver Seção 33.
48d · **49c** Anexos do conteúdo aprovado	Colam no fim do PRD e do pedagógico, literais, as seções do *one-pager* aprovado que o gabarito não tinha onde pôr.	CONTRATO com o PO. Inteiro, o texto não tem como perder nada.
49 *One-pager* pedagógico	A missão inteira em linguagem de sala de aula, com o papel do professor, as telas do Stitch debaixo de cada passo, os links do protótipo e as perguntas de aprovação. Sai em Markdown, `.docx` e `.html`.	CONVENÇÃO de comunicação, com uma regra que vale citar: **nenhuma ressalva sobrevivente pode ser omitida**, mesmo com veredito verde [45].
49b · **49r** Pedagógico conferido	Habilidades, tempo em tela, podcast e caminhos de arquivo contra o aprovado; divergente volta ao redator.	CONTRATO interno.
50 Matriz de rastreabilidade final	A mesma matriz do design, agora até o PRD e o pedagógico. Entra no semáforo de Dev.	CONTRATO interno.
51 Fidelidade ao *Refazer* de dev	Cada pedido feito ao Dev conferido no *handoff*, no *brief*, no banco, no PRD e no pedagógico. Sem pedido de Dev, não roda.	CONTRATO com quem pediu.

Agente	O que faz	Em que se apoia
50 Diagnóstico da execução	O dossiê: como cada portão se saiu, o que trava, de quem é cada conserto, o que saiu provisório e que documento faltou — e, no cabeçalho, de qual base da plataforma a missão foi escrita. Sem modelo de linguagem.	CONTRATO interno. **Roda sempre**, inclusive quando tudo o mais falhou — é a peça que sustenta a decisão de não abortar.
51 Aprendizado	A leitura em prosa do diagnóstico, mais padrões aprendidos, lacunas com dono e candidatos a critério.	CONVENÇÃO.
52 Registro no catálogo	A memória entre execuções: registra a missão no índice, mostra as mais próximas e acusa identificador repetido — que é decisão humana, não aviso. *Status* derivado dos gates.	CONTRATO interno. **Promover é ato humano**. Missão bloqueada entra no índice de propósito: o que se quer evitar é reinventá-la.

PARTE VII

O que a operação ensinou

As execuções completas sobre material real expuseram defeitos que nenhum teste unitário pegaria — porque não eram erros de código.



uncoria®

31 O refino dos entregáveis

Uma revisão dedicada ao *one-pager* pedagógico e ao PRD perguntou o que a literatura recomendaria acrescentar a cada um. A resposta, nos dois casos, foi a mesma: **nada de conteúdo novo**. Os dois já estavam acima da média do gênero. O que faltava era **ordem, navegação e verificação**.

Um terceiro documento nasceu depois dessa revisão, e a razão dele é diferente das outras duas — é sobre *quem lê*.

31.1 O *one-pager* resumido: quando o documento certo é o curto

Em 21 de setembro comparamos o *one-pager* que o time de produto escrevia à mão — cerca de 2.550 palavras — com o que a esteira gerava: cerca de 8.700. O veredito não foi que um era melhor.

O gerado, de 8.700 palavras

Superior em **cobertura**: passa por todos os gates, documenta cada decisão, sustenta auditoria.

E é o documento que ninguém lê inteiro para aprovar uma missão.

O manual, de 2.550 palavras

Superior em **leitura** — e trazia quatro coisas que o gerado não tinha: componente curricular do bimestre, diferenciação frente às missões já feitas, contrato de evidência por habilidade e o *takeaway*.

A decisão foi ter os dois, e não escolher: o resumido é o que o time de produto de fato lê para aprovar; o canônico é insumo de auditoria. Os quatro itens ausentes entraram — e o contrato de evidência, que nasceu aqui, virou portão determinístico (Parte V).

A segunda reclamação foi sobre tom, e o diagnóstico vale como método. Com o formato certo, o responsável de produto reprovou a *linguagem*: "muito acadêmica e difícil de ler". A causa não estava no conteúdo. O *prompt* regulava fidelidade, códigos, tabelas e tamanho — e **não dizia nada sobre como soar**. Sem regra de registro, o modelo cai no português pedagógico formal e, pior, imita o registro da própria instrução que recebeu.

A correção foram duas seções novas no *prompt*: regras de linguagem com tabela de substituição de jargão, e calibragem de tom com cinco pares de exemplo extraídos do documento que o time considerava padrão-ouro. A estrutura de itens foi preservada intacta — só os textos mudaram.

> **Reclamação de leitura é problema de regra de registro no *prompt*, não de conteúdo.** O diagnóstico é direto: procure no *prompt* se há alguma regra sobre *como soar*. Se não houver, é isso.

31.2 *One-pager*: o problema era onde as coisas estavam

A pesquisa sobre adoção de material didático converge num ponto: a professora decide por **praticidade** — o material diz concretamente o que fazer, cabe na turma que ela tem, e compensa o esforço [17]. E em adoção de tecnologia escolar, a preocupação mais intensa é *operacional*, não teórica [22]. Três consequências:

- **A decisão pedida subiu para o topo.** Ela aparecia a 80% do documento. Quem lê precisa saber na primeira tela o que está sendo pedido; se parar ali, ainda sabe o que fazer.

- **Entrou "o que sua sala precisa ter"**, em lista fechada — aparelhos, projetor, internet, material impresso, minutos de preparo. Antes isso estava disperso em prosa, e era justamente a primeira pergunta que a professora faz.
- **As perguntas de aprovação viraram instrumento**, com opção de resposta e assinatura, em vez de convite a opinião livre. Material sem expectativa clara de implementação não é usado.

E um defeito concreto que só a execução real expôs: o documento dizia "o protótipo pode ser aberto no navegador" **sem nenhum endereço**, e o teste que deveria pegar isso não bloqueava. O protótipo é o único item que deixa a professora simular a aula antes de aprovar — o teste passou a ser bloqueante. Saíram também as URLs longas de servidor de arquivo coladas no corpo do texto: são ruído de leitura e vencem em poucos dias.

31.3 PRD: o problema era que nada verificava o que ele afirmava

Aqui a literatura foi mais direta ainda. Especificação bem escrita ajuda pouco; especificação **verificada contra o repositório** ajuda de forma mensurável [38], e o que reduz alucinação em geração de código é símbolo e caminho verificáveis, não prosa mais detalhada [14]. Somado a isso, documento longo não é vantagem: a recuperação piora no meio de contextos grandes [30].

As seções ficaram como estavam. O que entrou foi um **gate determinístico de consistência interna**, porque o gabarito já exigia essas regras em prosa e ninguém as conferia. Ele barra:

O que ele barra	Por que isso quebra quem codifica
Mais de um prefixo de tabela no mesmo documento	O prefixo nomeia o esquema inteiro. Dois prefixos significam dois bancos possíveis, e a escolha vira sorteio.
Requisito P0 fora da matriz e sem tarefa	Um requisito bloqueante sem rastro é um requisito que ninguém vai implementar, e ninguém vai notar que não implementou.
Evento com leitura em fila, sem tabela própria	Contradiz o contrato real de telemetria: o dado ficaria sem onde ser lido.
Tarefa que declara cobrir "todos os requisitos"	Não é decomposição. Uma tarefa assim não pode ser estimada, revisada nem dada como concluída.
Nome de módulo que não é identificador válido	Quebra no primeiro *import*.
Dois caminhos diferentes para o mesmo diretório	Metade do código vai para um lugar, metade para o outro.

Nenhum desses é lacuna de informação — são **contradições internas**. Um PRD pode não ter marcador de pendência nenhum e ainda assim fazer quem codifica escolher um caminho ao acaso; a partir daí, o resto do documento deixa de valer contra o que foi escrito.

32 O que a primeira e a segunda execução ensinaram

Rodar a esteira inteira sobre um material real expôs três defeitos que nenhum teste unitário pegaria, porque não são erros de código: são lacunas de contrato. Valem como princípio, não como conserto pontual.

1. **Ato que só uma pessoa pratica não é lacuna de construção.** Homologar autoria, indexar no material editorial, licenciar um acervo gráfico — nenhum agente produz isso. Os gates de rastro tratavam esses marcadores como impedimento, o que reprovava a missão por algo que nenhuma rodada poderia resolver. Pior: como o texto do marcador não nomeava uma etapa da esteira, o achado caía na etapa padrão de quem o leu, e uma homologação editorial aparecia endereçada ao time de desenvolvimento. Criou-se a categoria **humano**: severidade *decisão humana*, não bloqueio, e dono correto no dossiê.
2. **O prefixo da tabela é o identificador inteiro, sem separador interno.** Com o identificador novo carregando ano e bimestre, referências que ainda ensinavam o formato antigo levaram um avaliador a pedir um prefixo híbrido, que não corresponde a nenhum dos dois padrões. A regra passou a estar escrita no gabarito da *spec*, com contraexemplo.
3. **Insumo incompleto deve produzir pendência nomeada, não reprovação genérica.** Quando o material de origem não traz autoria, revisão ou indexação curricular, isso é informação ausente de quem forneceu o insumo — e o relatório precisa dizer exatamente isso, com o dono, em vez de devolver a missão como malfeita. **A régua não baixa; o endereço do problema é que muda.**

Uma quarta, achada na segunda execução: citação do formato não é pendência. Um agente escreveu, como afirmação de conformidade, a frase "nenhuma pendência `[ABERTO — <dono>]` sobrevive nesta *spec*". O varredor de lacunas leu a citação como pendência real e ainda propagou o *placeholder* como se fosse o responsável, produzindo no dossiê a linha sem sentido "comece por: `[ABERTO — <dono>]`". A guarda contra citação de formato existia, mas não cobria *placeholder* entre sinais de menor e maior — que nunca é dono real.

32.1 O efeito medido

As correções foram verificadas rodando a mesma missão duas vezes, com a mesma entrada.

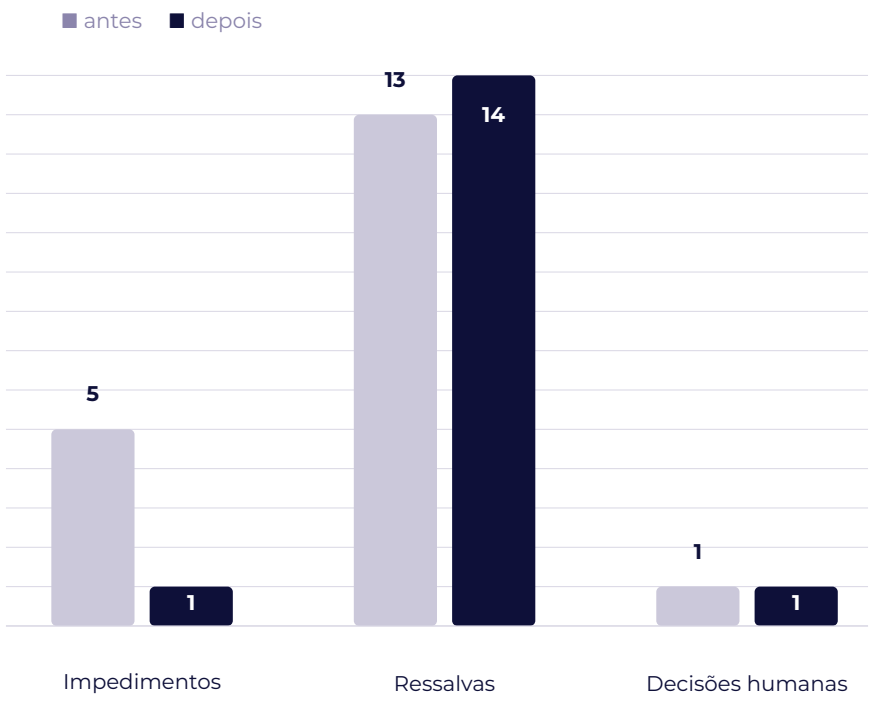


Figura 16: Mesma missão, antes e depois das correções. Os quatro impedimentos que sumiram eram atos humanos que a esteira cobrava de si mesma.

Nenhuma das correções baixou a régua. O que mudou foi o endereço do problema, não a exigência — e a contagem de ressalvas até subiu, porque impedimento falso virou ressalva verdadeira.

33 O que a terceira e a quarta execução ensinaram

A terceira execução não trouxe mais nenhum falso impedimento: os cinco achados eram todos defeitos reais do documento final, e vinham do mesmo lugar — o portão de consistência do PRD. Isso expôs um problema estrutural que as execuções anteriores escondiam, porque antes esse portão não existia.

33.1 Um portão que julga precisa poder devolver

O PRD é o último documento da esteira, e o seu portão era o único que **detectava sem poder corrigir**: apontava o defeito e seguia para o *one-pager* com o defeito intacto, deixando para uma pessoa aquilo que tem correção certa e mecânica. Os outros seis ciclos já devolviam o achado ao autor. O do PRD passou a devolver também, com o mesmo padrão e teto próprio de rodadas.

O princípio, que vale muito além deste caso

Gate sem ciclo de volta transfere para a pessoa um trabalho que a máquina sabe fazer. Um achado determinístico com correção única — prefixo de tabela fora do padrão, identificador inválido, requisito bloqueante sem rastro — não é uma decisão: é uma reescrita. O que deve chegar ao humano é a pendência de insumo e a decisão de escopo, nunca o erro de contrato que o próprio documento permite localizar.

33.2 Por que o redator errou com a regra escrita no *prompt*

As três regras violadas estavam todas escritas de forma explícita no gabarito do redator, com contra-exemplo. Não é caso de instrução ausente, e sim de **densidade de instrução**: o cumprimento cai à medida que o número de regras cresce no mesmo *prompt*, e o PRD é o documento com mais restrições simultâneas da esteira. Acrescentar ênfase ao *prompt* não resolveria — por isso a resposta foi um verificador determinístico *fora* do modelo, mais o ciclo que devolve o relatório para reescrita.

33.3 Um achado que não faz sentido é pior que nenhum achado

A quarta execução derrubou os impedimentos de cinco para um, e o que sobrou era **falso**: o verificador acusava um requisito P0 "sem rastro" que estava declarado, estava na matriz e estava coberto por duas tarefas. A regra contava ocorrências do identificador e exigia três; mas um PRD bem escrito cita o requisito *duas* vezes — na declaração e na matriz — porque é a matriz que nomeia a tarefa. E a cobertura costuma vir por intervalo (`RF-001..RF-014`), que não repete o identificador do meio.

O mais instrutivo é o que essa cegueira **escondia**. Ao passar a ler intervalos, o verificador enxergou cinco tarefas cobrindo os catorze requisitos cada uma — ou seja, o plano de implementação não estava decomposto, defeito bem mais sério que o falso positivo apontado. A regra de atomicidade existia, mas só reconhecia a ausência de decomposição quando escrita por extenso ("todos os RFs"); escrita como intervalo, passava batido.

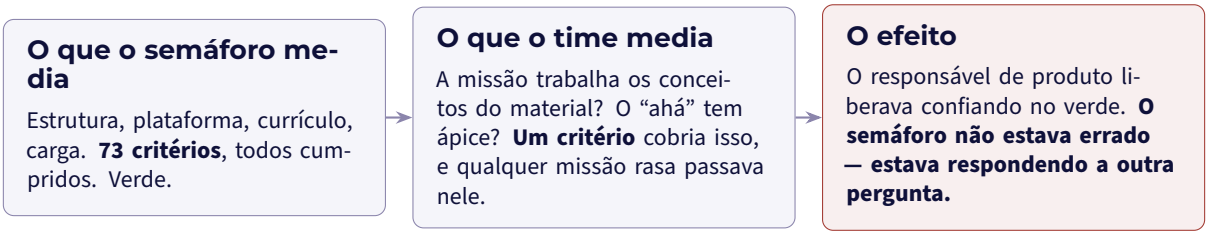
Dois princípios de escrita de verificador determinístico

Contar ocorrências é uma aproximação de rastreabilidade, não rastreabilidade. O teste certo não é quantas vezes o identificador aparece, e sim se existe uma linha que ligue o requisito a uma tarefa. Trocar a contagem pela relação corrigiu o falso positivo e, no mesmo movimento, *apertou* a régua: o impedimento inventado virou cinco ressalvas verdadeiras.

Achado que não carrega a evidência obriga quem lê a refazer a investigação. Um dos achados citava a linha defeituosa cortada em 90 caracteres, e o corte caía justamente antes do identificador do evento: o relatório dizia que havia um evento errado sem dizer qual. O achado passou a nomear o identificador e a carregar a linha inteira. Relatório assim é indistinguível de nenhum relatório.

34 A devolutiva que mudou a arquitetura

As correções descritas até aqui vieram de rodar a esteira e ler o que ela produzia. A de 24 de setembro veio de fora: **o time pedagógico devolveu as missões**. E ela é a mais instrutiva das quatro, porque o sistema não tinha errado — ele estava respondendo corretamente a uma pergunta que não era a certa.



Daí saíram as duas mudanças estruturais da 2.ª edição deste manual, e elas se sustentam mutuamente.

34.1 Medir o que o time mede

A primeira foi a camada de profundidade (Seção 18): mapa conceitual, ideação, anatomia do "ahá" e um juiz próprio. O princípio é simples de enunciar e fácil de esquecer:

> **Um gate que não mede o que a revisão humana mede não protege — ele autoriza.** O verde não era um erro de cálculo: era uma afirmação verdadeira sobre uma pergunta irrelevante, lida como se respondesse à pergunta que importava.

34.2 Pôr uma pessoa na passagem

A segunda foi o parecer humano entre etapas (Seção 4.2). Ela responde a uma objeção óbvia à primeira: se o gate automático pode estar medindo a coisa errada, então nenhuma régua nova resolve o problema de fundo — ela só o move.

Por isso a esteira deixou de correr sozinha do briefing ao PRD. Hoje cada etapa termina com uma pessoa que **responde por ela**, com comentário obrigatório que segue para as etapas seguintes. A máquina continua fazendo o que faz bem — conferir o que é mecânico, em escala, sem se cansar — e parou de ser a última palavra sobre o que é bom.

Três decisões de desenho que valem além deste caso

O comentário do *de acordo* é obrigatório e circula. Não é registro de auditoria: ele entra no contexto dos agentes seguintes. Aprovação silenciosa não transporta o que a pessoa sabia.
Devolver não refaz nada sozinho. Quem recebe uma etapa e discorda devolve *o parecer* para quem o deu — e é essa pessoa que decide entre refazer e reafirmar. A alternativa, disparar o retrabalho automaticamente, transfere a decisão para quem não respondeu por ela.
A exceção fica marcada. Com gate vermelho não há *de acordo*; só a conta de administração libera, e a liberação entra no dossiê como exceção. O sistema não impede que se passe por cima de um bloqueio — ele impede que isso passe despercebido.

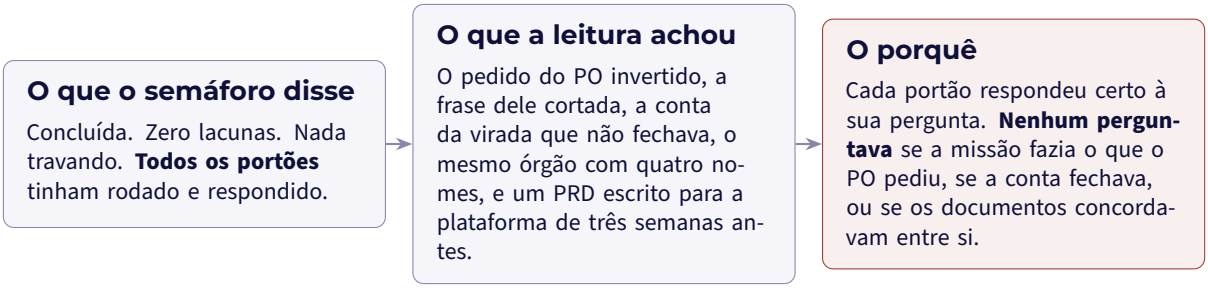
34.3 E uma coisa que a esteira passou a saber

Até 22 de setembro, o gerador não tinha memória entre execuções: o agente que inventava a missão não via nenhuma das anteriores, e duas missões parecidas só eram percebidas por acaso. Hoje há um índice, alimentado automaticamente ao fim de cada execução, que o micro-gate consulta.

A decisão de desenho que vale registrar é a mais contraintuitiva: **missão bloqueada entra no índice de propósito**. O que se quer evitar é reinventar uma missão que já foi tentada — e uma tentativa que falhou é exatamente o que ninguém deve repetir sem saber.

35 A missão que saiu verde e estava errada

A missão do 9.º ano — `exp09ef1b`, "O Dossiê Invisível" — saiu da esteira no começo de outubro marcada como **concluída**. O rastro dizia **"0 lacunas"**. O diagnóstico dizia **"Nada travando"**. Os três pareceres humanos tinham sido dados. E a leitura humana, feita com calma depois, achou uma lista de defeitos que nenhum portão tinha visto.



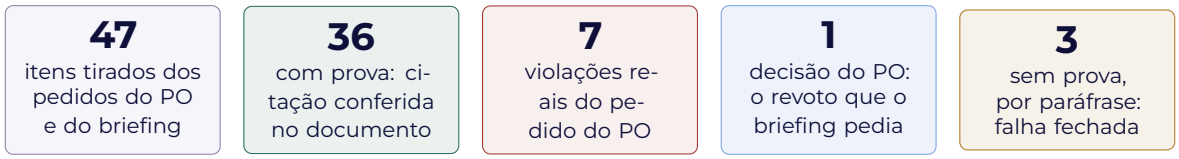
35.1 O que a leitura humana achou, e o que entrou por causa de cada coisa

O defeito	Por que nenhum portão viu	O que entrou
Caminhos A e B invertidos em relação ao que o PO pediu.	O PO pediu *Refazer* duas vezes; o segundo pedido apagou o primeiro, que era o que definia A e B. E o gate final escreveu que o pedido estava "integralmente atendido".	Os pedidos se acumulam e chegam a todas as etapas; a fidelidade confere cada um, com citação (03a e 16c).
A frase da descoberta cortada — a parte final, a que dava sentido, sumiu.	Nada comparava o texto entre aspas do PO com o documento.	Frase entre aspas nas TRAVAS é conferida letra por letra, sem juiz.
As perguntas de um canal inteiro, reescritas pelo Dev no *de acordo*, não entraram em documento nenhum.	O *de acordo* só registra. E o do Dev, última etapa, não tem etapa seguinte para lê-lo.	*De acordo* com conteúdo é recusado, com o caminho do *Refazer*.
O projeto "invisível" trocado (Ana virou João), e com isso **a conta do custo por vaga não fechava**: o projeto que devia perder já vencia antes da virada.	O juiz de profundidade deu nota 4 à virada citando justamente o custo. Ninguém fez a conta.	A tabela 1.3 do núcleo e a conta refeita em código (03aa); a troca de nome é item de fidelidade.
O banco padrão e as falas da IA só descritos: atas, tabelas e cadernos apareciam no *brief* com peso e tela, sem uma linha escrita; a IA tinha quatro falas na missão inteira; o *brief* marcava "Homologado" em arquivo inexistente.	Ninguém tinha a tarefa de escrever o conteúdo, e o PRD leu o *status* como fato.	O banco de conteúdo (35b) e a conferência dele (35c); *status* com vocabulário fechado.
Quatro nomes para o mesmo órgão municipal; beneficiários de 14 anos que viraram coordenadores na *copy*; "2 dias por semana" que virou "180 dias de registros".	Cada documento, sozinho, estava bem escrito. Nenhum agente lia os cinco juntos, e não havia lista do que é canônico.	O glossário canônico na *spec* e a coerência entre documentos (29d).
O QA aprovou "todas as 16 telas" ao lado de um Stitch com 6, e apontou o protótipo num arquivo que a esteira não grava.	O QA é um modelo; o que ele escreve sobre contagem ninguém conferia.	As afirmações do QA conferidas (30b) e o Stitch contado contra o inventário (29) — e, depois, gerado inteiro fora do modelo (26b).

O defeito	Por que nenhum portão viu	O que entrou
O PRD preso à plataforma de 16 de setembro: seção `chatgpt`, uma "ADR-108 proposta", DS 0.2.2 — e sem atividades, equipes nem evidência.	A cópia da plataforma no repositório estava 320 *commits* atrás, e nada dizia de quando ela era.	A base no *bucket*, publicada pela própria plataforma; o PRD conferido contra o contrato; a base no cabeçalho do diagnóstico (Seção 16).
"Nada travando" no dossiê, com a banca pedindo uma pessoa e o identificador da missão colidindo com outro.	O diagnóstico não lia o "precisa de humano" dos gates, e o catálogo rodava depois dele.	Decisão pendente aparece listada; colisão é decisão humana; o verde do rastro diz "nenhum marcador de lacuna".

35.2 O juiz de fidelidade, com a IA de verdade

O verificador de fidelidade foi rodado com o modelo real sobre o núcleo, o *one-pager* e a *spec* da própria `exp09ef1b`. O resultado é o melhor argumento a favor dele, e também o retrato honesto do que ele ainda não faz:



As sete violações eram todas reais, e incluíam o que a leitura humana tinha achado: **A e B trocados em quatro itens**, a frase da descoberta cortada — e mais o contrato de estados da missão. O conflito entre o revoto do briefing ("confirma ou muda a escolha") e a regra da escolha irreversível saiu como decisão do PO, que é onde ele deveria ter estado desde o começo.

E houve um achado sobre o próprio verificador. Na primeira passada, **15 itens que o juiz dava como atendidos caíam como "sem prova"** — não por erro do juiz, mas da conferência: uma marcação de código que deixava espaço onde não havia, citações com reticências no meio, uma vírgula a mais, e citações que o juiz copiava do próprio pedido em vez do documento. Os quatro foram corrigidos, e o *prompt* do juiz passou a dizer que a citação vem do documento, nunca do item. Sobram três sem prova, de paráfrase — e esses **devem** falhar.

35.3 As lições

Quatro princípios que esta missão deixou

Verde é uma resposta à pergunta do portão, não à pergunta de quem lê. "0 lacunas" queria dizer "nenhum marcador de lacuna"; "nada travando" ignorava o que os gates tinham pedido. A esteira passou a dizer o que mede — e quem dá o parecer continua respondendo pelo que ela não mede.

Um pedido não deixa de valer porque veio outro. O que o PO pediu é a especificação da missão. Guardar só o último pedido é perder, em silêncio, a parte da especificação que não foi repetida.

O que tem resposta exata se confere; não se julga. Uma conta, uma contagem de telas, um nome de arquivo, uma citação, a versão de uma biblioteca: o modelo escreve, o código confere.

Juiz de modelo que dá nota alta a uma conta que ninguém fez é o caso observado, não o hipotético.
Cópia que depende de alguém lembrar envelhece sem avisar. A base da plataforma passou a ser publicada por quem a muda, e cada missão diz de qual retrato foi escrita.

Nenhuma dessas correções baixou a régua, e nenhuma tirou a pessoa do parecer. O que mudou foi o que a máquina confere antes de pedir o parecer — para que a leitura humana gaste o tempo dela no que só ela vê.

36 O *one-pager* do PO que a esteira reescreveu

A missão do 6.º ano — `exp06ef1b` — já tinha *one-pager* aprovado pelo time. Em 8 de outubro o PO pediu três correções pontuais: tirar a habilidade EF06CO07, reduzir o Canal B de quatro para três perguntas "como estão" e mudar o foco para outra habilidade. E escreveu, com todas as letras: **não alterar a narrativa, o mecanismo e a estrutura**. O pedido foi feito numa conversa nova, com o *one-pager* anexado.

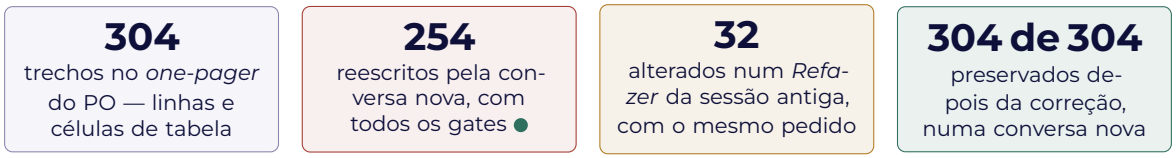


Figura 17: O mesmo *one-pager*, medido pela conferência que nasceu deste caso. A versão do PO só reformatada dá zero perdas: a régua não acusa formatação.

A conversa nova trocou a Etapa 5 por outro mecanismo, mudou os tempos de 29 e 21 minutos para 25 e 25, pôs o Unoovista no lugar da seção que o PO escolhera, reescreveu o podcast, o roteiro e as perguntas do Canal B — e a *spec* atribuiu as mudanças ao "PO no *one-pager* v1.8.0". Todos os gates saíram verdes. A leitura do código explicou por quê:

- **Só o núcleo via o anexo.** O núcleo narrativo é o único agente que lê a mensagem e os anexos como chegaram. As seções de apoio, os revisores e os pareceres trabalhavam sobre o que a esteira tinha escrito.
- **As conferências comparavam a esteira com ela mesma.** A *spec* e o resumido eram conferidos contra o *one-pager gerado*. Nada comparava o resultado com o que o PO mandou.
- **O pedido em prosa não virava item.** O verificador de fidelidade só extraía itens de briefing em blocos; com "não alterar a narrativa" em prosa, ele achou um item e o juiz nem rodou.
- **A atribuição ao PO passava.** "PO no *one-pager*" não era reconhecido como atribuição, e a decisão do agente chegou ao parecer com o nome do PO.

O que entrou é a Seção 3.5: o *one-pager* anexado vira a base, conferido trecho a trecho duas vezes, com restaurador, e o que um parecer quiser mudar vira proposta ao PO. Junto, duas correções vizinhas que o mesmo caso expôs. Num *Refazer* da sessão antiga, o PO mandou tirar a EF06CO07 **e a própria marcação** `[PROPOSTA — validar com PO: tirar EF06CO07]`; a marcação voltava a cada rodada, porque o *lint* lia só o briefing de 28 de setembro, que dizia "obrigatoriamente EF06CO07". E a seção do DS do *one-pager* do PO era uma seção antiga, que a ficha do manifesto bloqueava: com o *one-pager* do PO

como base, isso empurraria a *spec* para a sucessora contra a escolha dele. As duas foram resolvidas a favor do PO (Seções 4.3 e 2).

Depois da correção, a missão foi refeita como o PO quer usá-la: conversa nova, o *one-pager* corrigido anexado, e a mensagem só "use o *one-pager* anexado; não alterar nada". A conferência contou **304 trechos preservados de 304**.

> **Conferir a esteira contra ela mesma não é conferência.** Todo gate respondia certo: a *spec* batia com o *one-pager*, o resumido batia com a *spec*. Só que o *one-pager* já não era o do PO. A referência de uma conferência tem de ser o que a pessoa aprovou, não o que a máquina escreveu a partir disso.

37 A resposta que o modelo cortou

Na mesma manhã, a missão do 3.º ano — `exp03ef1b` — quebrou duas vezes seguidas num *Refazer* de Produto, com a mensagem "*Context variable not found: one_pager in agent A05GateEstrutural*". A mensagem aponta o gate estrutural; o defeito estava um passo antes.

Reproduzido sobre o estado salvo, o agente das seções de apoio terminava com o motivo `RECITATION` e nenhum texto. O filtro de recitação do Gemini corta a saída que transcreve texto publicado — e o *one-pager* transcrevia o texto oficial da habilidade EF03CO03 três vezes. Repetir com a mesma instrução dava o mesmo corte; tirar a versão anterior da instrução também. **Com um aviso no topo — "não transcreva texto publicado por extenso" — saiu o *one-pager* inteiro**: cerca de 60 mil caracteres, seções 0 a 18, com o pedido do PO aplicado.

O que entrou está na Seção 6 e na Parte V: todo agente registra o corte e o motivo, e a tentativa seguinte recebe o aviso; o *one-pager* vazio faz o ciclo chamar o autor de novo; se ainda vier vazio, a etapa para antes do gate, dizendo quem ficou mudo e por quê; e, como prevenção, o texto oficial de cada habilidade passou a ser citado uma vez só. A primeira tentativa ainda pode ser cortada; a segunda, com o aviso, resolve.

> **O erro aparece onde o dado faltou, não onde ele deixou de ser escrito.** A mensagem nomeava a variável e o agente que tentou lê-la; o agente que ficou mudo não aparecia em lugar nenhum. Silêncio de modelo é um resultado, e precisa ser registrado como tal — com o motivo.

38 A primeira volta do padrão ouro

As conferências das Seções 14 e 11 entraram em produção na madrugada de 8 de outubro. A primeira execução real com elas — o Design da missão do 1.º ano, `exp01ef1b` — terminou ●, com **21 lacunas de coerência**. Reproduzidas sobre o estado salvo, só uma era real: o Canal B encurtado na camada de acessibilidade. Treze eram falsos positivos — o detector de variante proibida casava a própria tabela do glossário republicada e locuções comuns fora de rótulo. Sete eram conflitos: o glossário da *spec* proibia termos que o *one-pager* aprovado usava no título e no podcast.

E **nenhum conserto rodou** em duas rodadas: o orçamento contava a cadeia inteira de passos em código atrás de cada dono, e um único *ticket* custava 11 de um orçamento de 8. Corrigidos o detector, o glossário — termo que o PO aprovou não é proibido, e o conflito volta à *spec* ainda em Produto — e a conta do orçamento, a mesma execução fica com a lacuna real.

O Dev foi medido do outro lado. A etapa de Dev da missão do 9.º ano foi reexecutada localmente, numa cópia, com o código novo: o PRD gerado passou em todas as conferências de "dá para codar" (Seção 14.2), contra nove bloqueios no PRD de 30 de setembro. Dois achados que sobraram eram do gate, não do PRD: o redator escreveu o *prompt* da IA como uma constante de código, num bloco *TypeScript*, e o verificador só procurava bloco de texto; e o rótulo "Ainda não:" do nível de evidência, no pedagógico, era lido como a negação de um texto aprovado. Os dois foram corrigidos.

O princípio, que vale para cada verificador novo

Gate novo que gera ruído não protege: ele trava o retrabalho que deveria consertar. Treze falsos positivos consumiram o orçamento que a lacuna real precisava. Por isso cada verificador desta edição foi calibrado nas sessões reais do *bucket* antes de bloquear — a conferência do *one-pager* do PO dá zero perdas na versão só reformatada; a régua estrita de literais não criou falso positivo novo nas missões do 1.º, do 6.º e do 9.º ano — e o teste de mutação confere que ele ainda pega a perda que deve pegar.

39 A consequência de método

Há uma mudança que vale mais que qualquer número corrigido: a fonte única da régua passou a ter uma seção de **registro negativo** — o que *não* sustenta a régua e não deve voltar (Parte IV). Sem ela, uma fundamentação refutada tende a reaparecer na revisão seguinte, porque o argumento de autoridade sobrevive ao artigo que o derrubou.

A missão do 9.º ano acrescentou uma segunda: **a cada defeito que só a leitura humana achou, a pergunta é qual afirmação de algum documento era conferível e ninguém conferia**. Quase sempre havia uma — e cada verificador novo é a resposta a uma delas.

As missões do 6.º e do 1.º ano acrescentaram uma terceira, que é a outra face da segunda: **conferir contra o que a pessoa aprovou, e calibrar em dado real antes de bloquear**. Um verificador que compara a esteira com ela mesma dá verde para o documento errado; um que não foi medido nas sessões reais dá vermelho para o documento certo — e os dois tiram da pessoa a confiança de que o semáforo diz alguma coisa.

O mesmo vale para este manual. Ele descreve um sistema que muda, e a única forma honesta de descrevê-lo é dizendo, a cada regra, de onde ela veio — para que a próxima pessoa saiba o que precisa refutar para mudá-la.

Apêndices

A árvore de artefatos, as seções do PRD, o glossário e as fontes.



uncoria®

A A árvore de artefatos de uma execução

Toda missão criada pelo aplicativo persiste uma árvore isolada em `output/<data-hora>-session_<id>/` — no servidor, dentro do *bucket* das sessões. O identificador da missão não está no nome da pasta, e pode se repetir entre missões; para achar uma, leia o `conversa.json`. Os nomes abaixo são os que a esteira grava (`pipeline/artifacts.py`).

```
output/20261008-101500-session_0a1b2c3d/
|-- PRD.md # o PRD padrao ouro (23 secoes + anexo aprovado)
|-- one-pager-pedagogico.{md,docx,html} # o documento de validacao humana
|-- diagnostico-execucao.md # o dossie da execucao (com a base da plataforma)
|-- matriz-de-rastreabilidade.md # cada requisito do produto, em cada documento
|-- aprendizado.md # padroes, lacunas e candidatos a criterio
|-- catalogo-ficha.md # a ficha desta missao no indice geral
|-- briefing-original.md # o insumo que voce enviou
|-- estado.json # o estado completo da sessao
|-- execucao.json # progresso da etapa em segundo plano
|-- conversa.json # historico, pareceres, pedidos e passagens
|-- manifesto-artefatos.json # metadados de todos os arquivos gerados
|
|-- produto/
| |-- 00-material-anexado.md # o material do bimestre, como a esteira o leu
| |-- 00g-mapa-conceitual.md # conceito limiar, crencas, vocabulario
| |-- 00h-ideacao-mecanismos.md # os tres mecanismos candidatos
| |-- 00a-nucleo-narrativo.md # descoberta, anatomia do aha, conta da virada
| |-- 00i-cobertura-conceitual.md # nucleo x mapa
| |-- 00b-micro-gate.md # spoiler, gap, escolha, job e semelhanca
| |-- 00j-juiz-profundidade.md # as seis notas, com trecho citado
| |-- 00-one-pager.{md,docx,html} # one-pager canonico (18 secoes)
| |-- 00c-one-pager-original.md # a versao antes do revisor
| |-- 00d-one-pager-resumido.{md,docx,html} # o formato do time de produto
| |-- 00e-contrato-produto.md # lint de contrato de plataforma
| |-- 00f-brief-escopo.md # o brief de escopo, separado
| |-- 01-gate-estrutural.md # os criterios e o envelope
| |-- 02-avaliacao-pedagogica.md # BNCC, professor e guardrails
| |-- 03-experiencia-aluno.md # carga cognitiva, faixa e agencia
| |-- 03b-adequacao-pedagogica.md # parecer em seis eixos
| |-- 04-gate-final.md # liberacao formal de produto
| |-- 05-spec-missao.md # o contrato de construcao + glossario canonico
| |-- 05a-spec-missao-gate0.md # a spec antes do fechamento de lacunas
| |-- 05b-conformidade-bncc.md # bloco 11 x lista oficial do MEC
| |-- 05c-matriz-evidencia.md # grau, sinais e gabarito por habilidade
| |-- 05d-perguntas-de-produto.md # as perguntas, a decisao e o corte
| |-- 05e-juiz-perguntas-de-produto.md
| |-- 05f-rastro-lacunas-spec.md # lacunas de produto, cobradas aqui
| |-- 05g-conteudo-aprovado.md # spec x one-pager aprovado
| |-- 05h-conteudo-aprovado-resumido.md # resumido x canonico
| +-- 05i-banca-produto.md # a banca no fim de produto
|
|-- design/
| |-- 00-fechamento-lacunas.md # decisoes e pedidos de re-spec
| |-- 00a-relatorio-lacunas.md # o mesmo, em PT-BR, para produto
| |-- 00b-ambiente-simulado.md # a ficha do ambiente e a classe
| |-- 00c-spec-consolidada-x-one-pager.md # o que a consolidacao piorou
| |-- 01-briefing-oficio.md # arco, pico e beat memoravel
| |-- 02-jornada-telas.md # raias x 6 etapas + visao do professor
| |-- 03-fluxo-arestas.md # grafo do estudante + grafo do professor
| |-- 04-copy-strings.md # dicionario de strings (passada 1)
| |-- 04b-camada-caa.md # simbolos, modos e pranchas
| |-- 05-wireframes-cobertura.md # telas, estados e a tabela do Stitch
| |-- 06-copy-strings-revisada.md # revisao em contexto de tela
```

```
| |-- 06b-lint-prototipo.md # o que o QA nao ve por ler so o texto
| |-- 06c-coerencia-documentos.md # tudo contra o glossario canonico
| |-- 07-qa-entregaveis.md # auditoria de fidelidade
| |-- 07b-qa-afirmacoes.md # as contagens e os arquivos que o QA citou
| |-- 07c-banca-design.md # a banca no fim de design
| |-- 08-prototipo.html # <- abre com duplo clique
| |-- 08-prototipo.md # o relatorio do prototipo
| |-- 09-diagramas.html # jornada e fluxo remontados
| |-- 10-stitch-telas.md # telas da versao atual e link do projeto
| |-- 11-miro-boards.md # link do board espelho
| |-- 12-rastro-lacunas-design.md # lacunas de design, cobradas aqui
| +-- 13-fidelidade-refazer-design.md # o pedido feito ao design, conferido
|
+-- dev/
   |-- 00-ficha-manifesto.md # bloco 0 x listas fechadas
   |-- 00b-banca-entrada.md # a segunda verificacao, por dono
   |-- 01-handoff-dev.md # stories fechadas e telemetria
   |-- 01b-brief-assets.md # o que produzir e o que reprova
   |-- 01c-telemetria-check.md # contrato de evento e arquitetura
   |-- 01d-rastro-lacunas.md # o que faltou, por dono
   |-- 01e-banco-conteudo.md # o texto integral, as falas e a arvore do chat
   |-- 01f-banco-conteudo-check.md # o banco conferido contra o brief
   |-- 02a..02e-banca-*.md # as dimensoes da banca
   |-- 03-banca-consolidada.md # veredito unico: a pior das dimensoes
   |-- 04-pendencias-do-prd.md # consistencia, plataforma, design, da para codar
   |-- 04b-kpis-check.md # coerencia KPI <-> evento <-> tela
   |-- 04c-conteudo-aprovado-prd.md # PRD x one-pager aprovado
   |-- 04d-coerencia-prd.md # as contradicoes, com as duas citacoes
   |-- 05-conteudo-aprovado-pedagogico.md # pedagogico x aprovado
   +-- 06-fidelidade-refazer-dev.md # o pedido feito ao dev, conferido
```

Os relatórios da conta da virada, da fidelidade ao pedido do PO em Produto e da conferência do *one-pager* do PO não viram arquivo próprio: aparecem no semáforo e no passo correspondente da tela, e ficam no `estado.json`. As telas do Stitch que entram no pedagógico ficam numa pasta oculta da sessão, fora do manifesto. Os arquivos baixados pela tela levam o identificador da missão no nome — `PRD_exp09ef1b.md`, `spec_exp09ef1b.md`, `artefatos_exp09ef1b.zip`.

B As seções do PRD padrão ouro

O `PRD.md` segue um gabarito único (`prompts/references/prd-padrao-ouro.md`), escrito contra o código da plataforma e contra uma missão que já roda nela, e que sintetiza métodos de especificação de produto e de engenharia — EARS e Gherkin para critérios de aceite, ADR para decisões — e os requisitos do art. 14 da LGPD. São 23 seções, numeradas de 0 a 22, depois do *frontmatter*; no fim, a esteira cola o conteúdo aprovado do *one-pager*. As edições anteriores deste manual listavam 25 seções de um gabarito mais antigo: a lista abaixo é a do arquivo.

#	Seção	O que traz
—	***Frontmatter***	Identificador, produto, série, unidade, ambiente, seção do DS e os caminhos que a sessão tem. Sem *status*: quem publica o *status* dos gates é a esteira.
0	**Resumo executivo**	Problema, para quem, a descoberta numa frase, o número que a missão existe para mover.

#	Seção	O que traz
1	**Contexto e problema**	O problema de aprendizagem e como a missão se diferencia das que já existem.
2	**Objetivos e métricas**	A tabela de KPIs que um gate lê — pergunta, decisão, corte, tela, evento, baseline e alvo com origem —, com um KPI de evidência por habilidade.
3	**Não-objetivos**	O que fica de fora, com motivo; toda ausência declarada na *spec*.
4	**Personas e *jobs***	Estudante, professor e gestor, e os cenários de dispositivo.
5	**Experiência**	As seis etapas, com os tempos e os cenários de aula do *one-pager* e o podcast no momento dele.
6	**Fluxo e máquina de estados**	Os estados do contrato da *spec*, com o nome que o banco grava, a trava de cada gesto e quem destrava.
7	***User stories***	Uma por tela do inventário da jornada, com todas as chaves da *copy* e onde cada uma vive.
8	**Requisitos funcionais**	Todo P0 com critério EARS ou Gherkin e o teste que o prova.
9	**Requisitos não funcionais**	Meta numérica com origem — ou proposta ao PO — por categoria, LGPD para menores incluída.
10	**UX, telas e design**	Inventário, direção, acessibilidade e CAA por tela.
11	**Modelo de dados**	DDL, sem nível de evidência em tabela da missão.
12	**Contratos de API**	Rotas no molde da plataforma, corpo e resposta em Zod, tabela de erros, e a IA do preparo com *prompt* e *schema*.
13	**Eventos e telemetria**	A tabela de eventos e os sinais universais das perguntas transversais.
14	**Pedagogia e conformidade**	A matriz de habilidades, a ficha de evidência que o servidor calcula e o que cada adulto vê.
15	**Dependências e pré-requisitos**	O que a plataforma precisa ter antes, e o inventário de conteúdo.
16	**Arquitetura da solução**	Componentes e comunicação entre a missão e a plataforma.
17	**Plano de implementação**	Arquivos e convenções em caminhos reais do monorepo; manifesto, migração, *hub* do professor.
18	**Tarefas**	Decompostas e ordenadas, cada uma com o comando que a verifica.
19	**Decisões e pendências**	ADR, pendências e as decisões do agente que o PO precisa revisar.
20	**Testes e *Definition of Done***	Estratégia de testes e critérios objetivos de conclusão.
21	**Plano de lançamento**	Fases, sem data que ninguém homologou.
22	**Apêndices e *changelog***	O que mudou nesta versão do documento.

C Glossário

Agente Uma posição numerada da esteira. Pode ser um modelo com *prompt* próprio ou um passo determinístico em Python.

Anatomia do "ahá" Os seis passos que um "ahá" genuíno tem: crença, aposta, consequência, virada, nomeação e transferência. É tabela obrigatória do núcleo narrativo.

Ambiente simulado A casca da missão: a seção do *design system* que reproduz a estrutura de um app real com a identidade da Unooria — UnooriaAI no lugar do ChatGPT, Unoovista no do Instagram, Unoochat no do WhatsApp.

Banca Os avaliadores paralelos de cinco dimensões — produto, design, pedagogia, dados/LGPD e arquitetura —, com os mesmos critérios em quatro posições: fim de produto, fim de design, entrada do Dev e dentro do Dev.

Banca de entrada A segunda verificação de produto e design, antes de qualquer agente de Dev. Não reescreve nada; ● impede o Dev de rodar.

Banco de conteúdo O texto integral de cada documento, tabela e roteiro que a missão mostra, e as falas da IA simulada por estado. Escrito por agente, com *status* "escrito"; homologar é da curadoria.

Base da plataforma O retrato da plataforma e do *design system* que a execução leu, publicado pelo CI de cada um num *bucket* próprio, que o gerador só lê, e fixado no começo da execução. O diagnóstico diz qual foi.

CAA Comunicação aumentativa e alternativa: símbolos, pranchas e modos de resposta para estudantes que não respondem por texto.

Citação conferida O trecho literal que um juiz de modelo traz para sustentar um achado, procurado pelo código no documento. Citação que não existe rebaixa o achado.

Conceito limiar O conceito do material do bimestre que, uma vez compreendido, muda como o estudante vê o resto. Sai do mapa conceitual e tem de viver no Decidir ou no "ahá".

Conteúdo aprovado As seções do *one-pager* aprovado, literais, entregues a cada agente de Design e de Dev e coladas no fim do PRD e do pedagógico. É contra ele que os documentos a jusante são conferidos.

Conta da virada A tabela 1.3 do núcleo: opções, critérios, pesos e valores antes e depois do dado que faltava. O código refaz o ranking e confere que a virada existe.

Corte do modelo Resposta vazia que o próprio modelo interrompeu — por recitação de texto publicado ou por tamanho. Fica registrada com o motivo, e a tentativa seguinte recebe o aviso.

Decisão do PO Marcada ●: conflito entre o pedido do PO e uma regra da esteira. Nem o juiz nem o autor escolhem; vai para a lista do que só uma pessoa decide.

Descoberta central O que o estudante sabe ao fim e não sabia no início — e que a missão jamais pode enunciar antes de ele agir.

Envelope JSON O bloco padronizado que fecha todo relatório de portão, e que o orquestrador lê para decidir o fluxo.

Esteira estrita Modo padrão: reprovação marca herança degradada e o PRD sai bloqueado, em vez de virar ressalva.

Fidelidade A conferência de que os documentos fazem o que o PO pediu e o briefing fixou, item a item, com citação conferida em código.

Grau de medida `scored`, `indicative`, `zero` ou `none`. Define o que um número sobre uma criança pode afirmar — e a distinção entre `zero` e `none` é regra de proteção, não de contabilidade.

Gate, portão O passo que julga o trabalho de outro agente. Determinístico (código) ou avaliador (rubrica).

***Gap* de informação** A lacuna deliberada entre o que o estudante sabe e o que a missão vai revelar. É o que faz a virada funcionar.

Glossário canônico O bloco da *spec* com o nome único de cada coisa da missão — entidades e as variantes proibidas, personagens, números, rótulos de botão, arquivos. Todo documento seguinte usa só esses nomes, e o código confere.

Impedimento Achado que impede construir. Distinto de ressalva (risco aceito) e de decisão humana (fora da autoridade da esteira).

Matriz de rastreabilidade Cada requisito do produto procurado em cada documento que tem de carregá-lo, com o dono de cada perda. Sai no fim do Design e no fim do Dev.

Micro-gate O portão do agente 02, que audita o núcleo narrativo antes de qualquer construção.

Ofício O papel que o estudante assume na simulação, e que justifica as ferramentas que ele usa.

One-pager O documento canônico de 18 seções (Produto) e, no fim da esteira, o documento de validação humana (agente 49).

***One-pager* do PO** O *one-pager* que o PO anexa a uma conversa nova. Vira a base da missão: a esteira completa e confere trecho a trecho, e não o redesenha.

Passagem O momento entre duas etapas: semáforo dos gates, parecer do dono, recebimento pelo dono seguinte. Registrada no histórico da missão.

Pedido do PO O comentário de um *Refazer*. Todos se acumulam e valem para todas as etapas; TRAVAS, AJUSTE e VALIDAÇÃO são os blocos que o verificador de fidelidade sabe conferir.

Pergunta de produto Pergunta que, respondida, muda uma decisão nossa. Tem três partes — a pergunta, a decisão que dispara e o corte. Sem decisão, é curiosidade.

Orquestrador O código que decide a ordem, os ciclos e as repetições. Não é um agente e não usa modelo de linguagem.

PRD *Product requirements document*: o entregável de 23 seções, mais o *frontmatter* e o conteúdo aprovado anexo, que alimenta quem codifica.

`[PROPOSTA]` `[PROPOSTA — validar com PO: …]`: decisão que um agente tomou, declarada como dele. Tira a etapa do verde e chega ao parecer.

`[PROVISÓRIO]` Marcador de documento construído sobre algo que um portão reprovou.

Recorte da aula O que a professora parametriza: disciplina, tema, assuntos, contexto e foco. Não é "um tema" — é o que torna a mesma missão utilizável em Português, Ciências ou História.

Re-*spec* Pedido formal de Design para que Produto redecida algo que a *spec* decidiu mal — em vez de Design decidir por conta própria.

***Spec* da missão** O contrato de 12 blocos (mais dois acrescentados em Design), seguido do glossário canônico, contra o qual tudo é construído e auditado.

Teste de mutação O teste que planta numa missão real uma perda conhecida e exige que a conferência a acuse. É como se confere o verificador.

D Fontes

Referências

[1] ANDERSON, L. W.; KRATHWOHL, D. R. *A Taxonomy for Learning, Teaching, and Assessing*. Longman, 2001.

[2] BETTS, J. et al. The development of sustained attention in children. *Child Neuropsychology*, v. 12, n. 3, p. 205–221, 2006.

[3] BIGGS, J.; COLLIS, K. *Evaluating the Quality of Learning: the SOLO Taxonomy*. Academic Press, 1982.

[4] BOMMASANI, R. et al. *On the Opportunities and Risks of Foundation Models*. arXiv:2108.07258, 2021.

[5] BRASIL. *Lei n.º 8.069, de 13 de julho de 1990* — Estatuto da Criança e do Adolescente.

[6] BRASIL. *Lei n.º 13.709, de 14 de agosto de 2018* — Lei Geral de Proteção de Dados Pessoais.

[7] BRASIL. Conselho Nacional de Educação. *Resolução CNE/CEB n.º 1, de 4 de outubro de 2022*: normas sobre Computação na Educação Básica — complemento à BNCC. Brasília: MEC, 2022.

[8] BRUNER, J. S. The act of discovery. *Harvard Educational Review*, v. 31, p. 21–32, 1961.

[9] BRUNS, B.; COSTA, L.; CUNHA, N. Through the looking glass. *Economics of Education Review*, v. 64, p. 214–250, 2018.

[10] BUCKINGHAM, D. *Digital Media Literacies: Rethinking Media Education in the Age of the Internet*. 2007.

[11] CASTELO, N.; KIM, S.; CARLSON, S. Choice overload em crianças. *Frontiers in Developmental Psychology*, 2023. DOI 10.3389/fdpys.2023.1317426

[12] CEMRI, M.; PAN, M. Z.; YANG, S. et al. *Why Do Multi-Agent LLM Systems Fail?* NeurIPS 2025, Datasets and Benchmarks Track. arXiv:2503.13657.

[13] COWAN, N. Working memory maturation. *Perspectives on Psychological Science*, v. 11, n. 2, p. 239–264, 2016.

[14] *De-Hallucinator*: recuperação iterativa de referências de API do próprio projeto para reduzir alucinação em geração de código. arXiv:2401.01701.

[15] DEMETRIOU, A. et al. Cognitive developmental cycles. *Frontiers in Psychology*, v. 13, 954971, 2022.

[16] DEWEY, J. *Experience and Education*. Kappa Delta Pi, 1938.

[17] DOYLE, W.; PONDER, G. The practicality ethic in teacher decision-making. *Interchange*, v. 8, n. 3, p. 1–12, 1977.

[18] FISCHER, K. A theory of cognitive development. *Psychological Review*, v. 87, n. 6, p. 477–531, 1980.

[19] GATHERCOLE, S. et al. The structure of working memory from 4 to 15 years of age. *Developmental Psychology*, v. 40, n. 2, p. 177–190, 2004.

[20] GODWIN, K. et al. Off-task behavior in elementary school children. *Learning and Instruction*, v. 44, p. 128–143, 2016.

[21] HAGGER, M. et al. A multilab preregistered replication of the ego-depletion effect. *Perspectives on Psychological Science*, v. 11, n. 4, 2016.

[22] HALL, G.; HORD, S. *Implementing Change: Patterns, Principles and Potholes* — Concerns-Based Adoption Model. 2015.

[23] ISBELL, E. et al. Visual working memory continues to develop through adolescence. *Frontiers in Psychology*, v. 6, 696, 2015.

[24] JAROSLAWICZ, D.; WHITING, B.; SHAH, P.; MAAMARI, K. *How Many Instructions Can LLMs Follow at Once?* arXiv:2507.11538, 2025.

[25] JONSSON, A.; SVINGBY, G. The use of scoring rubrics: reliability, validity and educational consequences. *Educational Research Review*, v. 2, n. 2, p. 130–144, 2007.

[26] KALYUGA, S. et al. The expertise reversal effect. *Educational Psychologist*, v. 38, n. 1, p. 23–31, 2003.

[27] KNIGHT, J. C.; LEVESON, N. G. An experimental evaluation of the assumption of independence in multiversion programming. *IEEE Transactions on Software Engineering*, v. SE-12, n. 1, p. 96–109, 1986.

[28] KNOOP-VAN CAMPEN, C.; SEGERS, E.; VERHOEVEN, L. The modality and redundancy effects in dyslexia. *Dyslexia*, v. 24, n. 2, p. 140–155, 2018.

[29] LIVINGSTONE, S.; THIRD, A. Children and young people's rights in the digital age. *New Media & Society*, v. 19, n. 5, 2017.

[30] LIU, N. F.; LIN, K.; HEWITT, J. et al. Lost in the middle: how language models use long contexts. *TACL*, v. 12, p. 157–173, 2024.

[31] LOEWENSTEIN, G. The psychology of curiosity. *Psychological Bulletin*, v. 116, n. 1, p. 75–98, 1994.

[32] MERRILL, M. D. First principles of instruction. *ETR&D*, v. 50, n. 3, p. 43–59, 2002.

[33] MORENO, R.; MAYER, R. Verbal redundancy in multimedia learning. *Journal of Educational Psychology*, v. 94, n. 1, p. 156–163, 2002.

[34] OSTERHAUS, C.; KOERBER, S. The development of advanced theory of mind. *Child Development*, v. 92, n. 5, p. 1872–1888, 2021.

[35] PAPERT, S. *Mindstorms: Children, Computers, and Powerful Ideas*. Basic Books, 1980.

[36] PARNAS, D. L. On the criteria to be used in decomposing systems into modules. *Communications of the ACM*, v. 15, n. 12, p. 1053–1058, 1972.

[37] ROZENDAAL, E. et al. Children's understanding of advertisers' persuasive tactics. *International Journal of Advertising*, v. 30, n. 2, p. 329–350, 2011.

[38] *Spec Kit Agents*: grounding e validação contra o repositório em desenvolvimento dirigido por especificação. arXiv:2604.05278.

[39] STARKS, D.; REICH, J. "What about data?": youth understanding of algorithmic systems. *Information and Learning Sciences*, v. 125, n. 9, p. 673–692, 2024.

[40] SWELLER, J. Cognitive load during problem solving: effects on learning. *Cognitive Science*, v. 12, n. 2, p. 257–285, 1988.

[41] SWELLER, J.; VAN MERRIËNBOER, J.; PAAS, F. Cognitive architecture and instructional design: 20 years later. *Educational Psychology Review*, v. 31, n. 2, p. 261–292, 2019.

[42] TRAN, D.; KIELA, D. *Single-Agent LLMs Outperform Multi-Agent Systems on Multi-Hop Reasoning Under Equal Thinking Token Budgets*. arXiv:2604.02460, 2026.

[43] UNESCO. *A Global Framework of Reference on Digital Literacy Skills for Indicator 4.4.2*. UNESCO Institute for Statistics, 2018.

[44] VYGOTSKY, L. S. *Mind in Society*. Harvard University Press, 1978.

[45] ZHENG, L.; CHIANG, W.-L.; SHENG, Y. et al. Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. NeurIPS 2023. arXiv:2306.05685.

Manual do Gerador de Missões

Outubro de 2026 · 4.ª edição · documento de trabalho, uso interno

Unooria Educação e Tecnologia